SMART BEINGS · WOOHOO PLATFORM

Agentic Voice Platform **Architecture**

End-to-end system design for the in-room hospitality voice assistant — migrating from Dialogflow intent matching to a Gemini-powered agentic orchestrator over MCP-exposed hotel, room, device and service capabilities.

Docker → AWS | Gemini + MCP | Multi-tenant | Production-grade | Safety & Compliance

DOCUMENT	VERSION	DATE	STATUS
Woohoo V2 Target Architecture	1.0 — for review	29 July 2026	Draft for architecture sign-off

FRONT MATTER

How to read this document

This document is the complete target architecture for **Woohoo V2** — the second generation of the in-room hospitality voice assistant, rebuilt around an agentic core. It is written to be reviewed and then executed against: every significant choice is stated as an explicit *decision* with the reasoning and the rejected alternatives beside it, so a reviewer can disagree with a specific line rather than the whole design.

Structure

Parts I–II	Context and the end-to-end picture. Read these even if you read nothing else — Part II contains the full system map from guest utterance to physical actuation and back.
Parts III–VIII	The build. Service decomposition, the voice pipeline, the agentic core, MCP, data, and the integration layer that talks to Honeywell / Interel / PMS.
Parts IX–XI	The things that make it a product rather than a demo: agentic safety, security, compliance and governance.
Parts XII–XV	Operating it: scale and unit economics, reliability, observability and evaluation, and the Docker → AWS deployment path.
Part XVI + Appendices	Roadmap, team shape, and the concrete contracts (tool schemas, event schemas, config schema) that the implementation starts from.

Conventions

- DECISION boxes are load-bearing. Changing one changes the build plan.
- NEW marks components that do not exist today and must be built.
- Risk tiers T0 T1 T2 T3 T4 are defined in Part IX and referenced throughout.
- Prices and latency figures are **indicative planning numbers** current to early 2026. They are accurate enough to choose an architecture and wrong enough that you must re-quote before signing a contract. Every such figure is marked with a †.

> **SCOPE EXCLUSIONS**
>
> Guest-facing and staff-facing **UI is out of scope** (owned by a separate workstream), but this document does specify the *contracts* the UI depends on — the staff task API, the notification event stream, and the device screen state protocol — because those are architecture, not presentation. Hardware/industrial design of the device enclosure is also out of scope; the device *software* architecture is in scope.

Contents

Part I — Context & Executive Summary

1 What Woohoo is and what breaks today
2 What actually changes in V2
3 Executive summary of the target design
4 Decision register
5 Principles that drive the design

Part II — End-to-End Architecture

6 The complete system map
7 The critical path, narrated
8 Component inventory (every box, owner, build/buy)
9 Trust and network zones

Part III — Service Decomposition

10 Modular monolith vs microservices — the honest answer
11 The service catalogue

12 Phased decomposition and split triggers
13 Synchronous vs event-driven boundaries

Part IV — Voice & Edge Layer

14 Wake word
15 The STT placement decision: device vs cloud
16 ASR vendor selection
17 TTS and response audio
18 Cascaded vs speech-to-speech (Gemini Live)
19 Device↔cloud transport
20 Latency budget
21 Device software architecture
22 Hotel network reality

Part V — The Agentic Core

23 Orchestration pattern
24 The three-lane router
25 Model selection and routing
26 Prompt architecture and context assembly
27 The agent loop, step by step
28 Session, memory and disambiguation
29 Deterministic degraded mode

Part VI — MCP & Tool Architecture

30 Why an MCP gateway is mandatory
31 MCP server topology
32 Tool surface design
33 The capability model
34 Authorization, scoping and idempotency

Part VII — Data Architecture

35 The domain model
36 Datastore selection per workload
37 Device state: cache, twin and truth
38 The event backbone

Part VIII — Integration Layer

39 Room automation vendor adapters
40 The on-premise edge gateway
41 PMS integration
42 Service request and ticketing

Part IX — Agentic Safety

43 Threat model for a physical-world agent
44 The action risk tier model
45 Blast radius containment
46 Prompt injection defence
47 Grounding and hallucination control
48 Duty of care and emergency escalation
49 Abuse, privacy between guests, and misuse
50 Kill switches and staff override

Part X — Security Architecture

51 Device identity and lifecycle
52 Network and cloud security
53 Tenant isolation and blast radius
54 Secrets, keys and credentials

Part XI — Compliance & Governance

55 The consent problem
56 Controller / processor and the DPA
57 Data classification and retention
58 Residency and regional topology
59 AI-specific regulation
60 PCI, accessibility, life safety
61 Certification roadmap and procurement
62 Governance operating model

Part XII — Scale & Unit Economics

63 Demand model
64 Capacity model
65 Cost model and cost per room
66 The optimisation ladder

Part XIII — Reliability

67 SLIs and SLOs
68 The degradation ladder
69 Failure modes and responses

Part XIV — Observability & Evaluation

70 Tracing an agent turn
71 Quality metrics
72 The evaluation harness
73 Red teaming

Part XV — Deployment

74 Phase 1: Docker
75 Phase 2: AWS production
76 The cross-cloud question
77 CI/CD and prompt release management
78 Environments and property canaries

Part XVI — Delivery

79 Roadmap
80 Team shape
81 Open questions requiring your input

Appendices

A Tool / MCP schemas
B Event schemas
C Property configuration schema

D Repository and module layout

E Glossary

PART I

Context & Executive Summary

1. What Woohoo is, and what breaks today

Woohoo is an in-room voice assistant for hotels. A guest says *"Hello Voice Genie"* and then asks for something — dim the lights, set the temperature, send housekeeping, order a club sandwich, when is checkout, what's the Wi-Fi password. The device is an Android hub with a screen (`com.sb.woohoo`), deployed one per guest room, and it fronts whichever building-automation system that particular property runs — Honeywell INNCOM, Interel, or another — plus the hotel's service and F&B operations.

The V1 system is a Dialogflow application. The device performs wake-word detection locally with the Sensory TrulyHandsfree SDK, opens a gRPC stream to Google for speech recognition, receives a matched intent with parameters, and the backend maps that intent onto an API call. Text-to-speech is synthesised on the device using AWS Polly. There is a socket.io channel to the backend for push, Twilio for video calling, Spotify for music, and QR-code enrolment for device provisioning.

That design has four structural problems, and they are the reason V2 exists.

1.1 Intent matching cannot represent what guests actually say

Dialogflow's model is: one utterance → one intent → a fixed parameter set → one action. Guests do not speak that way. *"It's freezing in here and can you get someone to look at the shower, it's dripping"* is two intents, one of which is a service ticket with a free-text description that no entity extractor was designed to produce. *"Make it a bit warmer"* requires reading the current setpoint before deciding the target — Dialogflow has no notion of reading state before acting. *"Same as yesterday"* requires memory. Every one of these becomes a fallback intent, and fallback rate is the single number that determines whether guests keep using the device after day one.

1.2 Every new hotel is an integration project, not a configuration change

Because intents are wired to concrete API calls, a property running Interel needs a different intent-to-action mapping than one running INNCOM, and a property without motorised curtains needs the curtain intents disabled. In V1 that difference lives in code and in branching. This is the reason onboarding a property takes weeks. It is also the reason the system cannot be sold as a product with a price per room — the marginal cost of a customer is engineering time.

1.3 The device holds credentials and makes cloud decisions

The V1 device ships with `google-services.json`, a Google service-account auth path (`google-auth-library-oauth2-http`), and AWS Polly SDK credentials. A device physically stolen from a hotel room — which happens — is a credential compromise, and the credential is not scoped to that room. This is the finding that will fail the first serious hotel-chain security review.

1.4 There is no audit trail of *why* the system did something

When a guest complains that the lights turned off during their shower, or that a $180 room-service order appeared on their folio, V1 can show that an API was called. It cannot show what was heard, what was inferred, what state was read, what was decided and on whose authority. For an agentic system that takes physical actions and spends money, that trail is not optional — it is the product's licence to operate.

2. What actually changes in V2

Dimension	V1 (Dialogflow)	V2 (Agentic)	Why it matters
Understanding	Intent classification with fixed slots	Gemini reasoning over a tool surface, multi-intent, multi-turn, with state read before action	Fallback rate collapses; compound requests work
Action binding	Intent hard-wired to a vendor API	Abstract capability tools; a per-property adapter resolves them to the vendor	New property = config, not code
State	Write-only, fire and forget	Read current state → reason → act → verify → confirm	"Make it warmer" works; failed actions are detected
Data access	Bespoke REST per feature	MCP servers behind a governed gateway	One integration contract; tenant scoping enforced centrally
Device role	Smart client with cloud credentials, does STT and TTS itself	Thin, credential-less audio endpoint with a per-device X.509 identity	Theft is contained; pipeline is upgradable without an OTA
Authority	Implicit — whatever the intent was wired to	Explicit policy engine outside the model, with risk tiers and confirmation rules	The model cannot be talked into an action policy forbids
Evidence	API access logs	Immutable per-turn decision record: transcript, context, reasoning summary, tool calls, policy verdicts, outcome	Dispute resolution, incident response, regulatory defence
Deployability	One environment	Docker Compose for pilots and on-prem → AWS multi-tenant production, same artefacts	Pilot velocity without a rewrite

> **THE REFRAMING THAT MATTERS MOST**
>
> V1 asked *"which of my known intents is this?"* V2 asks *"what does this guest want, what am I permitted to do about it in this room, and what is the current state of the world?"* The second question is answerable by a model. The first never was.
>
> But the corollary is the whole reason this document is long: once the system can decide, **everything the system is not allowed to decide must be enforced outside the model**. Roughly two-thirds of this architecture is that enforcement.

3. Executive summary of the target design

Woohoo V2 is a **multi-tenant, event-backed agentic platform** with a thin edge, a governed tool plane, and a policy layer that sits between the model's intent and the physical world.

3.1 The shape in one paragraph

The device does wake-word detection, voice activity detection, acoustic echo cancellation and endpointing locally, then streams Opus audio over a single authenticated WebSocket to a **Device Gateway**. The gateway terminates device identity, resolves `device_id → tenant / property / room`, and hands the session to a **Session Orchestrator** that owns the turn.

The orchestrator streams audio to a cloud **ASR** provider, and the resulting transcript enters a **three-lane router**: a deterministic fast lane for the ~60% of utterances that are high-frequency and unambiguous, an agentic lane on Gemini for everything else, and a clarification lane when confidence is low. The agent reasons over a **capability-shaped tool surface** served by an internal **MCP Gateway**, which fans out to the property catalogue, device state, service/ticketing, F&B, PMS and knowledge MCP servers. Every tool call the agent proposes passes through a **Policy Decision Point** that checks tenancy scope, risk tier, quiet hours, spend limits and consent before it is allowed to execute. Execution reaches physical devices through a per-vendor **Automation Adapter**, which in most properties runs partly on an **on-premise Edge Gateway** because INNCOM and Interel are LAN-resident systems. The spoken reply is synthesised by streaming TTS and returned over the same WebSocket. Every step emits a span into a per-turn trace and an immutable record into the audit store.

3.2 The eight decisions that define the system

#	Decision	Choice	Principal alternative rejected
D1	STT location	**Cloud streaming ASR**, with wake word, VAD, AEC and endpointing on device	Full on-device ASR — fails on accent breadth and hotel-specific vocabulary at acceptable hardware cost
D2	Pipeline shape	**Cascaded** ASR → agent → TTS, with a Gemini Live pilot behind a flag	Native speech-to-speech as the primary path — loses the intermediate text that safety, audit and evals depend on
D3	Orchestration	**Three-lane router**: deterministic fast path, agentic path, clarify path	Pure single ReAct agent — too slow and too expensive for "turn on the lights"
D4	Service granularity	**Modular monolith with four carve-outs** at Phase 1; documented split triggers	Full microservices on day one — buys distributed-systems cost before there is scale to justify it
D5	Tool plane	**Internal MCP Gateway**; agent never speaks to a downstream MCP server directly	Agent holding N direct MCP connections — no central scoping, no tenant enforcement, no uniform audit
D6	Authority	**Policy Decision Point outside the model**; risk tiers T0–T4; scope injected server-side, never from model output	Prompt-level guardrails as the primary control — a prompt is a request, not a boundary
D7	Property connectivity	**On-prem Edge Gateway per property**, outbound-only, with local automation fallback	Direct cloud→BMS connectivity — not reachable, and hotel IT will not open inbound ports
D8	Data posture	**Ephemeral by default** — no raw audio retention, redacted transcripts, tokenised guest identity, zero-retention LLM/ASR contracts	Retain-everything-for-analytics — creates a consent and residency problem that hotels will not underwrite

3.3 What this buys, in numbers

Target	V2 goal	Basis
Wake→first audio response, p50	**≤ 1,100 ms**	Fast lane; see the latency budget in §20
Wake→first audio response, p95	**≤ 2,400 ms**	Agentic lane with one tool round-trip
Task success rate (top-50 intents)	**≥ 95%**	Measured by the eval harness in §72
Containment (resolved without staff)	**≥ 80%**	Excludes intentional escalations
New property onboarding	**≤ 5 working days** for a 300-room property	Config-driven; §12 and Appendix C
Cost per room per month at 10k devices	**< $1.10** † all-in cloud+AI	Cost model in §65
Unsafe-action rate (T3+ executed without required confirmation)	**0** — enforced structurally	PDP is in the execution path, not advisory

4. Decision register

The complete set of architectural decisions, each with its section reference. This table is the review surface — challenge rows here.

ID	Area	Decision	§	Reversibility
D1	ASR placement	Cloud streaming ASR; device does wake/VAD/AEC/endpoint only	15	Medium — hybrid path pre-planned
D2	Pipeline	Cascaded STT→LLM→TTS; Live API behind a per-property flag	18	High — flag-gated
D3	ASR vendor	Primary Deepgram Nova-3 or Google STT v2 (chirp_2); dual-vendor abstraction from day one	16	High — adapter interface
D4	TTS vendor	Google Chirp3-HD or Amazon Polly Generative primary; cached earcons and canned phrases	17	High
D5	Transport	Single WebSocket per device, Opus 16 kHz 20 ms frames, mTLS, exponential reconnect	19	Low — device firmware
D6	Wake word	Keep Sensory THF; abstract behind a `WakeWordEngine` interface; evaluate Picovoice as second source	14	Medium
D7	Orchestration	Three-lane router (fast / agentic / clarify)	24	High
D8	LLM	Gemini 2.5 Flash default; Flash-Lite for classification; Pro for offline eval and complex escalation only	25	High
D9	Tool plane	Internal MCP Gateway; downstream MCP servers never exposed to the agent directly	30	Medium
D10	Capability model	Abstract capability tools + per-property capability manifest; vendor differences never reach the model	33	Low — core abstraction
D11	Authority	PDP with T0–T4 risk tiers, evaluated on every proposed tool call	44	Low — core abstraction
D12	Scoping	Tenant/property/room scope injected server-side from the device identity; model output can never widen it	45	Low
D13	Decomposition	Modular monolith + Device Gateway + Edge Gateway + Adapters carved out; split triggers documented	10–12	High — that is the point
D14	Property connectivity	On-prem Edge Gateway, outbound-only mTLS tunnel, local fallback automation	40	Low
D15	Session state	Redis (ElastiCache) with 30-minute TTL; nothing durable in session store	28	High
D16	Catalogue store	PostgreSQL (Aurora Serverless v2) with row-level security by tenant	36	Medium
D17	Event backbone	Phase 1 Redis Streams / NATS; Phase 2 EventBridge + SQS; Kinesis only for telemetry firehose	38	Medium
D18	Audit store	S3 with Object Lock (WORM) + OpenSearch index; per-tenant KMS keys	57	Low
D19	Compute	ECS Fargate for stateless; EKS only if/when the fleet justifies it; no Lambda on the voice path	75	Medium
D20	Cross-cloud	Agent runtime on AWS calling Vertex AI over the internet with WIF; regional Vertex endpoints; measured, not assumed	76	Medium
D21	Data posture	Ephemeral by default; no raw audio at rest; PII redaction before LLM; zero-retention contracts	57	Low
D22	Consent	Hotel is controller, Woohoo is processor; notice at check-in + in-room + device-level; hard mic mute	55–56	Low
D23	Residency	Regional cells: `eu`, `in`, `me`, `us`; no cross-cell guest data movement	58	Low
D24	Payments	Woohoo never touches PAN; folio posting via PMS only; PCI scope reduced to SAQ-A-equivalent	60	Low
D25	Door locks	No lock actuation capability exists in the tool surface, at any tier, in V2	44	Deliberately hard to reverse
D26	Emergency	Device is never presented as an emergency channel; distress detection routes to staff and states its limits aloud	48	Low

ID	Area	Decision	§	Reversibility
D27	Prompt release	Prompts, tool schemas and model IDs are versioned deploy artefacts with staged rollout and one-click rollback	77	High
D28	IaC	Terraform; one module per environment cell; no console changes in production	75	Medium
D29	Repo	Monorepo with independently buildable service targets	App. D	High
D30	Evals	Golden + adversarial suites gate every prompt/model/tool change in CI; no manual “looks fine” releases	72	Low

5. Principles that drive the design

P1 — The model proposes, the platform disposes

Gemini never executes. It emits a proposed tool call; the Policy Decision Point authorises it; the executor performs it. Every constraint that must hold is code, not prose in a prompt. If a guest could talk the model into it, it was never a control.

P2 — Scope is ambient, never argued

The agent is never asked which room to act on and cannot supply one. Tenant, property and room are derived from the authenticated device identity and injected into every tool call server-side. A tool call for a room the device is not in is structurally impossible.

P3 — Latency is a feature, not a metric

A voice assistant that answers in three seconds is perceived as broken regardless of accuracy. Every design choice is evaluated against the latency budget first. This is why the fast lane exists and why the device speaks an acknowledgement before the answer is ready.

P4 — Vendor differences stop at the adapter

The agent knows about “a thermostat in the bedroom zone with a setpoint range”. It never knows about INNCOM or Interel. Every vendor-specific fact lives in an adapter and a per-property capability manifest.

P5 — A new property is data

Onboarding a hotel must be: load a property config, load a room/device inventory, deploy an edge gateway, run a conformance suite. If it requires a code change, the abstraction failed and the abstraction gets fixed.

P6 — Store the decision, not the person

The system retains enough to reconstruct why it acted — redacted transcript, context digest, tool calls, policy verdicts — and as little as possible about who the guest was. Raw audio is never written to disk anywhere.

P7 — Degrade in defined steps

Gemini down, ASR down, internet down, BMS down are four different failures with four defined behaviours, ending in local-only automation that still turns the lights on. The device never becomes a brick with a logo.

P8 — Everything on the guest path is traced

One trace ID from the wake word to the actuation acknowledgement, spanning device, gateway, ASR, agent, MCP, adapter and vendor API. If it is not traced, it cannot be debugged at 3 a.m. across 500 properties.

THE SINGLE MOST IMPORTANT PRINCIPLE

P1. Everything else is engineering. P1 is the difference between a product a hotel chain will install in 40,000 rooms and a demo that gets pulled after the first incident. An LLM in the loop of physical actuation and guest billing is only acceptable if the set of things it can cause is bounded by something that does not reason.

PART II

End-to-End Architecture

6. The complete system map

This is the whole system, from a guest speaking in room 412 to a relay closing in a fan-coil unit and a confirmation coming back out of the speaker. Everything else in this document is a zoom into one of these boxes.

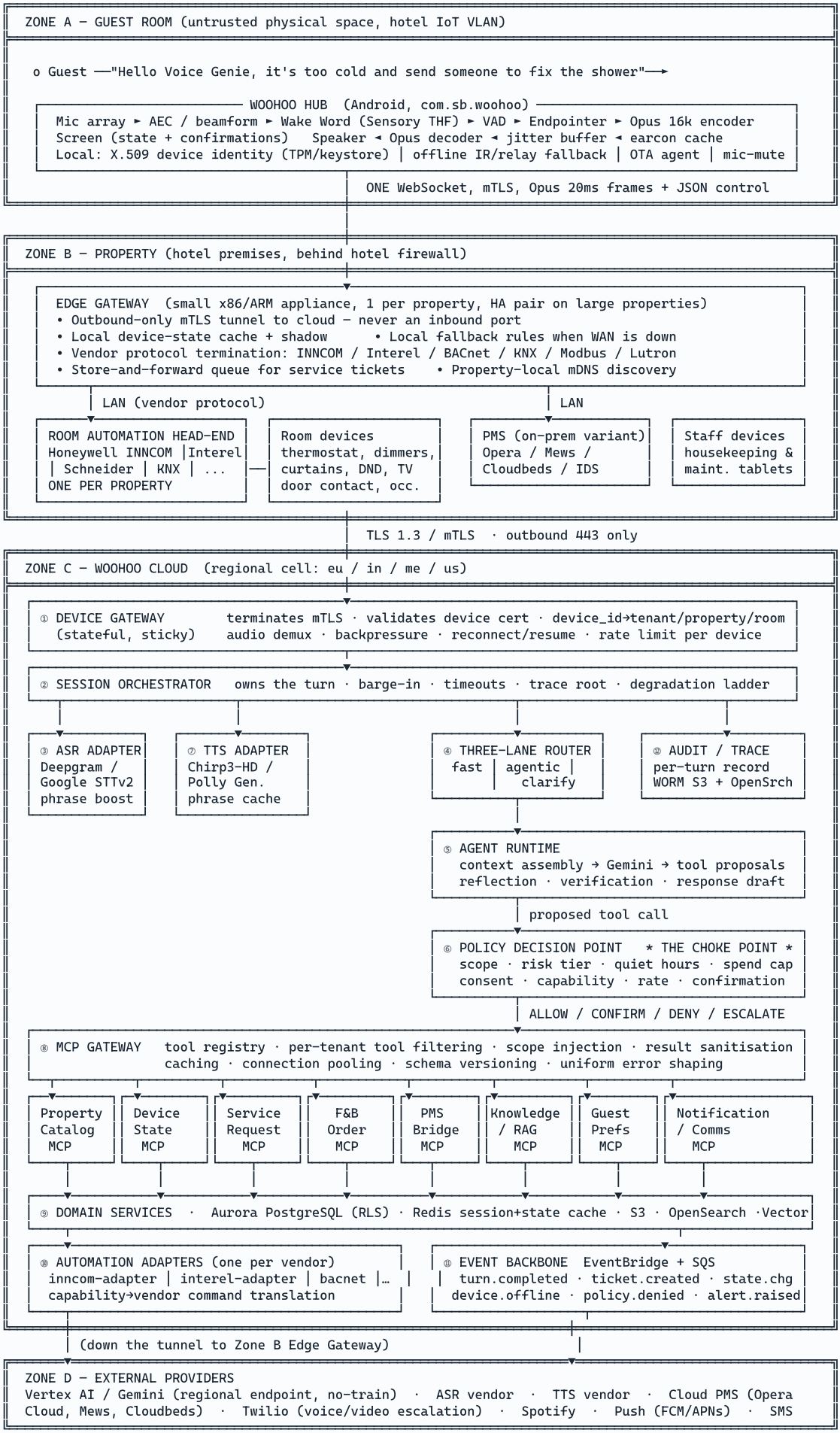

ZONE A — GUEST ROOM (untrusted physical space, hotel IoT VLAN)
o Guest ——"Hello Voice Genie, it's too cold and send someone to fix the shower"——►
WOOHOO HUB (Android, com.sb.woohoo)
Mic array ► AEC / beamform ► Wake Word (Sensory THF) ► VAD ► Endpointer ► Opus 16k encoder
Screen (state + confirmations) Speaker ◄ Opus decoder ◄ jitter buffer ◄ earcon cache
Local: X.509 device identity (TPM/keystore) | offline IR/relay fallback | OTA agent | mic-mute
ONE WebSocket, mTLS, Opus 20ms frames + JSON control
ZONE B — PROPERTY (hotel premises, behind hotel firewall)
EDGE GATEWAY (small x86/ARM appliance, 1 per property, HA pair on large properties)
• Outbound-only mTLS tunnel to cloud — never an inbound port
• Local device-state cache + shadow • Local fallback rules when WAN is down
• Vendor protocol termination: INNCOM / Interel / BACnet / KNX / Modbus / Lutron
• Store-and-forward queue for service tickets • Property-local mDNS discovery
LAN (vendor protocol)
LAN
ROOM AUTOMATION HEAD-END
Honeywell INNCOM |Interel
| Schneider | KNX | ...
ONE PER PROPERTY
Room devices
thermostat, dimmers,
curtains, DND, TV
door contact, occ.
PMS (on-prem variant)
Opera / Mews /
Cloudbeds / IDS
Staff devices
housekeeping &
maint. tablets
TLS 1.3 / mTLS · outbound 443 only
ZONE C — WOOHOO CLOUD (regional cell: eu / in / me / us)
① DEVICE GATEWAY terminates mTLS · validates device cert · device_id→tenant/property/room
(stateful, sticky) audio demux · backpressure · reconnect/resume · rate limit per device
② SESSION ORCHESTRATOR owns the turn · barge-in · timeouts · trace root · degradation ladder
③ ASR ADAPTER
Deepgram /
Google STTv2
phrase boost
⑦ TTS ADAPTER
Chirp3-HD /
Polly Gen.
phrase cache
④ THREE-LANE ROUTER
fast | agentic | clarify
⑫ AUDIT / TRACE
per-turn record
WORM S3 + OpenSrch
⑤ AGENT RUNTIME
context assembly → Gemini → tool proposals
reflection · verification · response draft
proposed tool call
⑥ POLICY DECISION POINT * THE CHOKE POINT *
scope · risk tier · quiet hours · spend cap
consent · capability · rate · confirmation
ALLOW / CONFIRM / DENY / ESCALATE
⑧ MCP GATEWAY tool registry · per-tenant tool filtering · scope injection · result sanitisation
caching · connection pooling · schema versioning · uniform error shaping
Property Catalog MCP
Device State MCP
Service Request MCP
F&B Order MCP
PMS Bridge MCP
Knowledge / RAG MCP
Guest Prefs MCP
Notification / Comms MCP
⑨ DOMAIN SERVICES · Aurora PostgreSQL (RLS) · Redis session+state cache · S3 · OpenSearch ·Vector
⑩ AUTOMATION ADAPTERS (one per vendor)
inncom-adapter | interel-adapter | bacnet |…
capability→vendor command translation
⑪ EVENT BACKBONE EventBridge + SQS
turn.completed · ticket.created · state.chg
device.offline · policy.denied · alert.raised
(down the tunnel to Zone B Edge Gateway)
ZONE D — EXTERNAL PROVIDERS
Vertex AI / Gemini (regional endpoint, no-train) · ASR vendor · TTS vendor · Cloud PMS (Opera Cloud, Mews, Cloudbeds) · Twilio (voice/video escalation) · Spotify · Push (FCM/APNs) · SMS

Figure 1 — Woohoo V2 end-to-end architecture. Numbered components ①–⑫ are catalogued in §8. The Policy Decision Point ⑥ is deliberately drawn as a full-width choke point: no path reaches the world without it.

7. The critical path, narrated

Take the compound utterance from Figure 1 — *"it's too cold and send someone to fix the shower"* — and follow it. This is the walkthrough to argue with; if this sequence is right, the rest of the document is detail.

```
t=0ms       Guest speaks the wake phrase. Sensory THF fires on-device. Device plays a 60ms earcon
            immediately (perceived responsiveness is bought here, not later) and opens/reuses the WS.

t=+40ms     Device begins streaming Opus frames. A 500ms pre-roll ring buffer is flushed first so the
            first syllable after the wake word is never clipped. AEC is already active.

t=+40ms     ① DEVICE GATEWAY validates the client cert (already established on the persistent WS),
            looks up device_id → {tenant_id, property_id, room_id, capability_manifest_version,
            language, consent_state}. This resolution is CACHED — it must not cost a DB hit per turn.
            The resolved scope becomes an immutable "TurnContext" that follows the turn everywhere.

t=+45ms     ② SESSION ORCHESTRATOR creates turn_id, starts the root trace span, loads session state
            from Redis (last 3 turns, pending clarification, last-referenced entity for anaphora).

t=+50ms     ③ ASR ADAPTER opens a streaming recognition session, injecting phrase hints assembled from
            the property's own vocabulary: F&B menu items, amenity names, staff-facing terms, local
            landmarks, and the room's own device labels. This is worth several points of WER and is
            the single cheapest accuracy win available.

...         Guest finishes speaking. Device-side endpointer detects end-of-utterance (~380ms of
            trailing silence, adaptive) and sends an explicit end-of-turn control frame — the cloud
            does NOT wait for its own silence timer. This saves 200–400ms versus server-side VAD.

t=+2,900ms Final transcript: "it's too cold and send someone to fix the shower, it's dripping".
(~150ms      Confidence 0.91. PII redactor runs (no PII here). Transcript enters ④ THE ROUTER.
 after
 speech
 ended)

t=+2,910ms ④ ROUTER: fast-lane matcher scores this. It is compound and contains a free-text
           maintenance description → no deterministic match → AGENTIC LANE. (Had the guest said
           "turn on the lights", the fast lane would have executed it by t=+3,000ms with no LLM call.)

t=+2,915ms ⑤ AGENT RUNTIME assembles context:
             · system prompt (versioned artefact, cached)
             · TurnContext: property brand voice, room 412, language en-IN, time 23:14 local
             · capability manifest for THIS room, rendered as the available tool set
             · current room state snapshot (pre-fetched in parallel at t=+50ms — see §27)
             · last 3 turns of conversation
             · pending-ticket summary ("do not create a duplicate")
           Sends to Gemini 2.5 Flash with structured tool-calling enabled.

t=+3,380ms Gemini returns TWO proposed tool calls in one turn:
             (a) climate.set_temperature { zone: "bedroom", target_c: 23.0 }
                 — chosen because state showed setpoint 19.5°C and the guest said "too cold"
             (b) service.create_request { category: "maintenance", subcategory: "plumbing",
                   description: "shower dripping", priority: "normal" }

t=+3,385ms ⑥ POLICY DECISION POINT evaluates each independently:
             (a) climate.set_temperature → tier T1. Scope check: zone belongs to room 412 ✓.
                 Capability check: property has HVAC control ✓. Bounds check: 23.0 within the
                 property's permitted 18–26°C band ✓. Quiet hours: no restriction on climate ✓.
                 → ALLOW
             (b) service.create_request → tier T2. Duplicate check: no open plumbing ticket for
                 412 ✓. Rate limit: 3rd ticket this stay, cap is 10 ✓. It is 23:14 — property
                 policy says non-urgent maintenance after 22:00 is queued for 08:00 dispatch,
                 not suppressed. → ALLOW WITH ANNOTATION {dispatch_at: 08:00}
           Both verdicts are written to the audit record BEFORE execution.

t=+3,390ms ⑧ MCP GATEWAY injects tenant/property/room scope server-side (the model's arguments are
           merged UNDER the injected scope — model-supplied scope fields are stripped, not trusted),
           then dispatches.

t=+3,400ms (a) → ⑩ inncom-adapter → down the tunnel → Zone B Edge Gateway → INNCOM head-end →
                 fan-coil setpoint write. Adapter returns ack + verified new state.
           (b) → Service Request domain service → row in Aurora → event on ⑪ → staff app push +
                 ticket appears on the maintenance tablet queue for 08:00.

t=+3,760ms Both results return to ⑤. The agent composes ONE reply, not two:
           "I've set it to 23 degrees, it'll warm up in a few minutes. I've also logged the dripping
            shower for maintenance — they'll come by after 8 in the morning so you're not disturbed
            tonight. Anything else?"

t=+3,790ms ⑦ TTS ADAPTER begins streaming synthesis. First audio byte at ~+3,900ms.
           ! BUT the guest has not been sitting in silence: at t=+3,400ms the orchestrator emitted a
           filled-pause acknowledgement ("Sure, one moment—") from the CACHED earcon/phrase set the
           instant the first tool call was authorised. Perceived latency is ~500ms, not ~3.9s.

t=+3,900ms Audio streams to the device. Screen shows: 23°C · Maintenance request logged #4471.

t=+4,400ms ⑫ AUDIT: the complete turn record is sealed and written — redacted transcript, TurnContext
           digest, model+prompt version, reasoning summary, both tool calls, both policy verdicts,
           both outcomes, latencies per span, cost. Raw audio was never written to disk at any point.
```

THREE THINGS TO NOTICE IN THAT TRACE

1. **The acknowledgement is emitted before the answer exists.** This is the single highest-leverage UX decision in the system and it costs nothing. See §20.4.
2. **Room state was pre-fetched at t=+50ms**, in parallel with the guest still speaking. By the time the model needs it, it is already in the context. This removes an entire tool round-trip — roughly 400–600 ms — from the majority of turns. See §27.2.
3. **The policy verdicts were written before execution, not after.** If the process dies mid-turn, the audit record still shows what was authorised. Log-after-the-fact loses exactly the records you need most.

8. Component inventory

Every box in Figure 1, with what it owns and whether it is built, bought or already exists. NEW means it does not exist in V1.

#	Component	Responsibility (owns)	Scaling profile	Build / Buy	Status
—	**Woohoo Hub** (device)	Audio capture, AEC, wake word, VAD, endpointing, Opus codec, playback, screen state, device identity, OTA, offline fallback	N/A — per unit	Build (Android, exists)	Refactor: strip cloud credentials, replace direct gRPC STT with WS to gateway
—	**Edge Gateway**	Property-local protocol termination, state shadow, offline fallback rules, store-and-forward, outbound tunnel	1–2 per property	Build (Go/Rust, container on appliance)	NEW — hard requirement, see §40
①	**Device Gateway**	WS termination, mTLS + device cert validation, identity→scope resolution, backpressure, per-device rate limits, reconnect/resume	Connection-bound; memory & FD heavy, low CPU	Build (Go or Node)	NEW
②	**Session Orchestrator**	Turn lifecycle, barge-in, timeouts, the degradation ladder, trace root, filled-pause emission	IO-bound, long-lived	Build (Python/TS)	NEW
③	**ASR Adapter**	Vendor-neutral streaming recognition interface, phrase-hint assembly, failover, confidence normalisation	IO-bound, stream-concurrency	Build wrapper / Buy ASR	NEW (V1 called Google directly from device)
④	**Router**	Lane selection: deterministic fast path, agentic path, clarification path; confidence thresholds	CPU-light	Build	NEW
⑤	**Agent Runtime**	Context assembly, Gemini invocation, tool-call loop, reflection, response drafting, token budget enforcement	IO-bound (waiting on LLM), bursty	Build	NEW — replaces Dialogflow
⑥	**Policy Decision Point**	Scope validation, risk tiering, confirmation requirements, quiet hours, spend caps, consent gating, capability gating, rate limits	CPU-light, must be <5 ms	Build (OPA/Cedar or hand-rolled)	NEW — the most important new component
⑦	**TTS Adapter**	Streaming synthesis, brand voice per property, SSML, phrase cache, failover	IO-bound	Build wrapper / Buy TTS	NEW (V1 did Polly on-device)
⑧	**MCP Gateway**	Tool registry, per-tenant tool filtering, scope injection, result sanitisation & untrusted-content tagging, caching, pooling, schema versioning	IO-bound, fan-out	Build	NEW
—	**Property Catalog MCP**	Hotel→property→floor→room→device inventory, zones, labels, capability manifests, property policy config	Read-heavy, cacheable	Build	Partially exists as DB; needs MCP facade
—	**Device State MCP**	Current and desired state of every controllable device; shadow reconciliation; state history	Read-heavy + write bursts	Build	NEW
—	**Service Request MCP**	Housekeeping / maintenance / amenity requests: create, status, cancel; SLA timers; duplicate detection	Balanced	Build	Exists in some form; needs MCP facade
—	**F&B Order MCP**	Menu retrieval, availability, cart, order placement, folio posting request, order status	Balanced	Build	`feature-fnb` exists on device; server side needs building
—	**PMS Bridge MCP**	Guest-stay context (tokenised), checkout time, folio posting, wake-up calls, late checkout, DND sync	IO-bound, external-latency-bound	Build adapters per PMS	NEW
—	**Knowledge MCP**	Property FAQ, amenity hours, policies, local recommendations; retrieval with citations	Read-heavy	Build (pgvector)	NEW
—	**Guest Preference MCP**	Opt-in cross-stay preferences, in-stay learned preferences; consent-gated; erasable	Light	Build	NEW — consent-gated, see §55
—	**Notification MCP**	Staff push, escalation paging, guest SMS/app notification, on-screen device messages	Bursty	Build + Buy transport	Partially exists (FCM)

#	Component	Responsibility (owns)	Scaling profile	Build / Buy	Status
⑩	**Automation Adapters**	One per vendor. Capability→vendor command translation, connection management, verification reads, error normalisation	IO-bound, per-property connections	Build (one per vendor)	NEW as a clean abstraction
⑪	**Event Backbone**	Async fan-out for tickets, state changes, alerts, analytics, staff notification	Throughput	Buy (EventBridge/SQS)	NEW
⑫	**Audit & Trace Store**	Immutable per-turn decision records, WORM retention, search, replay, cost attribution	Write-heavy, append-only	Build on S3+OpenSearch	NEW — compliance-critical
—	**Tenant Admin & Provisioning**	Property onboarding, device enrolment, capability manifest authoring, feature flags, kill switches	Light	Build	NEW (UI by the other workstream; API here)
—	**Eval & Replay Harness**	Golden/adversarial suites, CI gates, production turn replay, regression detection	Batch	Build	NEW — see §72

9. Trust and network zones

The zone boundaries in Figure 1 are the security architecture in miniature. Each crossing has a defined authentication mechanism, a defined direction, and a defined failure behaviour.

Crossing	From → To	Auth	Direction	If it fails	Key risk
X1	Guest voice → Device	None (physical presence)	In	N/A	Anyone in the room can act. Mitigation: no T3+ action is available without an explicit confirmation, and no action can affect another room (P2).
X2	Device → Edge Gateway or Cloud	mTLS, per-device X.509 from a private CA	Outbound only	Device enters offline mode: local IR/relay fallback, on-screen notice	Device theft. Mitigation: cert bound to hardware keystore, revocable, scoped to one room.
X3	Edge Gateway → Cloud	mTLS + per-property identity	Outbound only, persistent tunnel	Edge runs autonomously on cached rules; queues tickets for replay	Compromised edge = one property. Mitigation: property-scoped credentials only.
X4	Edge Gateway → BMS head-end	Vendor-specific (often weak — assume it is)	LAN, bidirectional	Automation unavailable; agent says so and offers to call staff	The BMS is usually the weakest link. Mitigation: dedicated IoT VLAN, edge is the only speaker to it.
X5	Cloud → Vertex AI	Workload Identity Federation (no long-lived keys)	Outbound	Degraded mode: fast lane only (§29)	Data egress and residency. Mitigation: regional endpoint pinning, no-train contract, redaction before send.
X6	Cloud → ASR/TTS vendor	API key in Secrets Manager, rotated	Outbound	Failover to secondary vendor; then text-only on-screen mode	Audio leaving the region. Mitigation: regional endpoints, zero-retention contract.
X7	Cloud → PMS	Per-property OAuth client / API cred, tenant-scoped	Outbound	Guest-context-free operation; folio posting queued or refused	Guest PII, folio. Mitigation: tokenised guest refs, no PAN ever, minimum scopes.
X8	Staff app → Cloud	OIDC, per-property staff identity, RBAC	Inbound (the only inbound path)	Staff fall back to existing hotel tooling	Privilege escalation across properties. Mitigation: RLS + tenant claim validation on every request.

THERE IS EXACTLY ONE INBOUND PATH

X8 — the staff/admin API — is the only place where traffic originates outside Woohoo cloud and terminates inside it. Devices, edge gateways and vendor systems are all **outbound-only**. This is not stylistic: hotel IT will not open inbound firewall rules for a vendor, and if they would, you should not ask. Design accordingly from the first line of code, because retrofitting outbound-only onto a system that assumed inbound polling is a rewrite of the integration layer.

PART III

Service Decomposition

10. Modular monolith vs microservices — the honest answer

You asked whether this should be modular, whether it should be microservices, and which components must be split. The honest answer has three parts, and the middle one is the one most teams get wrong.

10.1 It must be modular. That is not the same as distributed.

Modularity is non-negotiable here, because the whole product thesis is that a new hotel is configuration rather than code. That requires hard internal boundaries: the agent must not be able to reach into the INNCOM adapter, the F&B service must not read the property catalogue's tables directly, and vendor knowledge must be physically incapable of leaking into the reasoning layer. But every one of those boundaries can be enforced by a module system, a package structure and a lint rule — they do not require a network hop.

10.2 Most of it should *not* be separate services at Phase 1.

Splitting a system into microservices buys you three things: independent scaling, independent deployment, and failure isolation. It costs you distributed transactions, network failure modes between every pair of components, N× observability plumbing, N× CI pipelines, and — the one that actually kills small teams — the inability to change an interface without a coordinated multi-repo release.

At Phase 1 you have one pilot property and a handful of engineers. The scaling asymmetries that justify a split do not exist yet, and the interfaces are going to change weekly because nobody has yet learned what the agent actually needs from the catalogue. Splitting now means paying the full cost of distribution to buy benefits you cannot use.

10.3 But four things must be separate *from day one*, for reasons that are not about scale.

Must be separate	Why — and it is never "for scale"	What breaks if you don't
Device Gateway	Fundamentally different runtime shape: thousands of long-lived stateful WebSocket connections with strict memory-per-connection behaviour, versus request/response business logic. It also has a different deploy cadence — you must be able to ship agent logic ten times a day without dropping every device connection in the fleet.	Every backend deploy disconnects 10,000 devices. Reconnect storms. Guests hear "I'm having trouble" during every release.
Edge Gateway	It runs on someone else's premises, on hardware you ship, on a release cadence measured in months, and it must function when the cloud is unreachable. It is a different *product*.	You cannot control room lights when the hotel's WAN drops — which, in hotels, is weekly.
Automation Adapters	Isolation of blast radius and of dependency hell. The INNCOM integration will need a specific old TLS stack or a vendor SDK; the Interel one will need something incompatible. More importantly, when one vendor's head-end hangs, it must not exhaust the thread pool serving every other property.	One hotel's flaky BMS takes down voice control for every hotel. This *will* happen; it is the most likely production incident.
Policy Decision Point	Not for scale — for auditability and review. The PDP is the component a hotel chain's security team will ask to see. It must be independently versionable, independently testable, and small enough that a human can read all of it. Keeping it separate makes "the agent bypassed policy" a structurally impossible class of bug rather than a code-review question.	Policy logic diffuses into the agent runtime, becomes untestable, and the "the model cannot decide its own authority" guarantee quietly stops being true.

> **DECISION D13 — SERVICE GRANULARITY**
>
> **Phase 1 ships five deployable units:** (1) `woohoo-core`, a modular monolith containing the orchestrator, router, agent runtime, MCP gateway and all domain services; (2) `device-gateway`; (3) `policy-service`; (4) one `automation-adapter-*` per vendor; (5) `edge-gateway` (ships to properties). Everything inside `woohoo-core` communicates through explicit in-process interfaces that are *shaped like network calls* — typed request/response, no shared mutable state, no cross-module database access — so that promoting any module to a service later is a transport change, not a redesign.

11. The service catalogue

Every module and service, with the data it owns, its scaling profile, and whether it starts inside the monolith.

Module / Service	Owns (data & decisions)	Sync/Async	Scaling profile	Phase 1	Phase 2	Split trigger
Device Gateway	Connection registry, per-device rate state. Owns no business data.	Sync + stream	Memory & FD per connection; ~5k conns/instance	Service	Service	— already separate
Session Orchestrator	Turn lifecycle state (in Redis). Owns the degradation ladder.	Sync	IO-bound, long-lived per turn	Module	Service	>2,000 concurrent turns, or deploy cadence conflicts with agent runtime
Router	Lane rules, fast-path grammar, confidence thresholds.	Sync	CPU-light	Module	Module	Never — too small to justify
Agent Runtime	Prompt assembly, tool loop, token budgets. Owns nothing durable.	Sync	Bursty, IO-bound; long tail on LLM latency	Module	Service	When prompt/model releases need to ship independently of domain code — likely within 6 months
Policy (PDP)	Risk tier table, property policy, quiet hours, spend caps, confirmation rules.	Sync	CPU-light, p99 <5 ms required	Service	Service	— separate by design
MCP Gateway	Tool registry, per-tenant tool visibility, sanitisation rules.	Sync	IO fan-out	Module	Service	When third-party / customer-supplied MCP servers are admitted — then it becomes a security boundary
ASR Adapter	Vendor selection, phrase-hint assembly, confidence normalisation.	Stream	Stream concurrency; vendor connection limits	Module	Service	When you self-host an ASR model (GPU nodes ≠ web nodes)
TTS Adapter	Voice config per property, phrase cache.	Stream	IO-bound	Module	Module	Same as ASR — only on self-hosting
Property Catalog	**Source of truth:** tenants, properties, floors, rooms, zones, devices, labels, capability manifests, property config.	Sync	Read-heavy, highly cacheable	Module	Module	Rarely — but it is the hottest read path; cache aggressively instead
Device State	**Source of truth:** reported & desired state per device, shadow, state history.	Sync + async ingest	Read-heavy + high-volume writes from telemetry	Module	Service	When state-change ingest exceeds ~500 events/s — write path starves reads
Service Request	**Source of truth:** tickets, SLA timers, assignment, status history.	Sync + async	Balanced	Module	Module	When the staff-facing product grows its own roadmap and team
F&B Order	**Source of truth:** menus, availability, carts, orders.	Sync + async	Balanced, spiky at meal times	Module	Service	When POS integrations (Micros, Toast) multiply — same argument as automation adapters
PMS Bridge	Per-property PMS credentials, guest-stay token mapping, folio posting.	Sync	External-latency-bound, flaky upstreams	Module	Service	Immediately if a PMS upstream shows >1 s p95 — it must not block the agent thread pool
Knowledge / RAG	Property content, embeddings, retrieval config.	Sync	Read-heavy; vector search CPU	Module	Module	When corpus >10 M chunks or per-property fine-tuning appears
Guest Preference	Consent records, opt-in preferences, erasure state.	Sync	Light	Module	Module	Never — but its data is the most sensitive; separate *schema* and key, not process

Module / Service	Owns (data & decisions)	Sync/Async	Scaling profile	Phase 1	Phase 2	Split trigger
Notification	Delivery state, staff routing rules, escalation chains.	Async	Bursty	Module	Service	When escalation paging needs its own on-call reliability guarantees
Automation Adapters	Vendor connection state, command translation tables.	Sync + async	Per-property connections; blocking IO	Service ×N	Service ×N	— separate by design (bulkhead)
Edge Gateway	Local state cache, fallback rules, outbox queue.	Both	1–2 per property	Appliance	Appliance	— separate by design
Audit / Trace	Immutable turn records.	Async	Write-heavy append-only	Module + async sink	Service	When retention search becomes a product feature for hotels
Tenant Admin API	Onboarding workflows, enrolment, flags, kill switches.	Sync	Light	Module	Service	When the UI team wants independent release cadence — likely early
Eval Harness	Suites, baselines, results.	Batch	Batch/CI	Separate repo target	Same	— never in the serving path

12. Phased decomposition



SPLIT TRIGGERS — THE RULE THAT KEEPS THIS HONEST

A module is promoted to a service when **one of these is measurably true**, and never because it feels cleaner:

- **Scaling asymmetry:** the module's resource curve diverges >3× from its host (e.g. device-state write volume forcing the whole core to scale out).
- **Deploy contention:** two teams block each other more than twice a week, or a release cadence conflict emerges (prompt changes daily vs domain code weekly).
- **Blast radius:** the module can exhaust a shared resource — thread pool, connection pool, memory — and take the host down. This is the PMS-bridge and automation-adapter case.
- **Security boundary:** the module starts handling untrusted third-party code or data (customer-supplied MCP servers).
- **Compliance boundary:** the module's data must live in a different jurisdiction or under a different key than its host.

Write the trigger into the module's README when you create it. Review the list quarterly. This turns "should we split?" from an architecture debate into a metric lookup.

13. Synchronous vs event-driven boundaries

Getting this wrong is the most common way agentic systems become unreliable: teams make the guest-facing path asynchronous for elegance, and then cannot tell the guest whether their request worked.

Flow	Mode	Reasoning
Device → Gateway → Orchestrator → ASR → Router → Agent	**Synchronous / streaming**	The guest is standing there. Every millisecond is visible. No queues on this path, ever.
Agent → PDP → MCP Gateway → domain service	**Synchronous**	The agent needs the result to compose its reply. A queue here means the agent must speculate about outcomes, which is exactly the hallucination failure mode.
Automation command → adapter → edge → BMS	**Synchronous with a hard 2.5 s budget, then async continuation**	Most BMS writes ack in 150–800 ms. If it exceeds budget, the agent says "I've asked for that — give it a moment" and the confirmation arrives later as a screen update and, optionally, a follow-up utterance. Never block the voice turn on a slow head-end.
Ticket created → staff notification	**Async (event)**	The guest does not need to wait for a housekeeper's tablet to buzz. Durable delivery matters more than latency.
Device state telemetry ingest	**Async (event)**	High volume, no guest waiting, eventual consistency acceptable. Backpressure here must never reach the voice path.
Audit record write	**Async, but sealed synchronously**	The record is *constructed and sealed* in the turn; the durable write is queued. Critical nuance: policy verdicts are appended before execution (see §7), so a crash cannot erase evidence of what was authorised.
Folio posting	**Async with saga semantics**	Money. Needs idempotency keys, retries, compensation, and reconciliation against the PMS. Must never be a fire-and-forget event.
Analytics / quality metrics	**Async, sampled**	Firehose to S3, batch aggregate. Never on the hot path.

THE RULE

If the guest is waiting for the answer, it is synchronous. If the guest has already been told, it is asynchronous. The dividing line is the moment the agent finishes speaking, and it is the responsibility of the Session Orchestrator to know which side of that line every operation is on.

PART IV

Voice & Edge Layer

14. Wake word

The wake word is the only always-on listening component, which makes it simultaneously the most privacy-sensitive and the most performance-critical piece of the device software. It must run entirely locally, with no audio leaving the device until it fires.

Engine	Licensing	Footprint	Custom phrase	Assessment for Woohoo
Sensory TrulyHandsfree (current, v6.12)	Commercial, per-device royalty	~1–3 MB, <5% of one ARM core	Vendor-trained model; weeks of lead time, per-phrase fee	**Keep.** It already works, it is already integrated, and re-tuning wake performance is a multi-month project with no product upside. The SDK version is old — upgrade it, don't replace it.
Picovoice Porcupine	Commercial, per-device; self-service console	~1 MB, very low CPU	Self-service in minutes — a genuine operational advantage	**Second source.** Worth qualifying behind the interface precisely because custom phrases are self-service: if a hotel chain wants its own wake word ("Hello Taj"), Porcupine makes that a same-day change.
openWakeWord	Apache 2.0	~2–5 MB, moderate CPU	Self-trained, synthetic-data pipeline; free	Zero licence cost, but you own the false-accept tuning. Viable for a low-cost SKU or a dev/simulator build. Not for the flagship.
Cloud wake word	—	—	—	**Rejected outright.** Requires continuous audio streaming from a hotel bedroom. Legally indefensible and commercially unsellable.

DECISION D6 — WAKE WORD

Keep Sensory TrulyHandsfree, upgrade the SDK, and place it behind a `WakeWordEngine` interface on the device (`start()` , `onDetected(confidence, timestampMs)` , `setSensitivity()` , `pause()`). Qualify Picovoice Porcupine as a second implementation within six months — not to switch, but so that per-chain custom wake words become a commercial option rather than a vendor negotiation.

Required behaviours regardless of engine: a 500 ms circular pre-roll buffer so the first word after the wake phrase is never clipped; per-property sensitivity tuning (a suite by a lift shaft needs different settings than a quiet corner room); a false-accept counter reported as telemetry (a device with a high FA rate is usually next to a TV); and hard discard of the ring buffer whenever the wake word does not fire.

15. The STT placement decision: device vs cloud

This is the question you specifically asked, and it deserves a real answer rather than "it depends". The answer is that **the question is wrong as posed** — speech-to-text is not one function, it is five, and they belong in different places.

15.1 Decomposing "speech to text"

Sub-function	Placement	Why
Acoustic echo cancellation	**Device**	Must run before the signal leaves the mic path, or barge-in is impossible. The device knows what it is playing; the cloud does not.
Beamforming / noise suppression	**Device**	Requires the raw multi-mic array. Sending N channels to the cloud multiplies bandwidth for no benefit.
Wake word	**Device**	Privacy. Non-negotiable (§14).
Voice activity detection + endpointing	**Device**	Saves 200–400 ms per turn versus server-side endpointing, because the device can signal end-of-turn the instant it detects it rather than the cloud waiting out a silence timer over a jittery link. This is the largest single latency win available and it is free.
Transcription itself	**Cloud**	The contested one. Analysed below.

15.2 The case for on-device transcription, taken seriously

It is a real case and it is getting stronger. Whisper-small, Moonshine and NVIDIA Parakeet-class models now run in real time on modest ARM SoCs. On-device transcription would mean: no audio ever leaves the room (which dissolves most of Part XI's consent problem), no per-minute ASR cost at all, and functioning voice control during a WAN outage. At 100,000 devices, ASR is a meaningful line item; making it zero is attractive.

15.3 Why it still loses, for this product

Factor	On-device	Cloud streaming	Verdict for a hotel
Accent breadth	Small models degrade sharply on non-native and heavily accented English. A hotel serves guests from everywhere.	Large multilingual models are trained precisely for this and are measurably better on accented speech	**Decisive.** The guest population is the most accent-diverse of any consumer voice product. This alone settles it.
Hotel-specific vocabulary	Cannot be biased at runtime without shipping a per-property model update to every device	Phrase hints / custom vocabulary injected per session, per property — menu items, staff titles, amenity names, local landmarks	**Decisive.** "Bring me a masala dosa to 412" and "book the Chambers lounge" fail without biasing.
Language switching	One model per language, or a large multilingual model you cannot fit	Multilingual and code-switching supported centrally; language changes by config	**Decisive.** A property in Dubai needs EN/AR/HI/RU without a firmware change.
Improvement loop	Improving accuracy requires an OTA to the whole fleet	Improving accuracy is a config change, deployed in minutes	Strongly favours cloud. You will tune ASR weekly in year one.
Hardware cost	Needs a materially better SoC and more RAM — a real BOM increase per unit, multiplied by every room	Runs on the existing hub	Favours cloud. The BOM delta across 100k units dwarfs several years of ASR fees.
Latency	~200–500 ms local inference, no network	~120–300 ms with streaming, overlapped with speech	**Roughly a wash.** Streaming ASR transcribes *while* the guest talks; the only serial cost is finalisation. The intuition that cloud is slower is wrong for streaming.
Bandwidth	Sends ~200 bytes of text	Sends ~10–16 kbps Opus for the utterance duration ≈ **~8–12 KB per turn**	Immaterial. At 6 turns/device/day this is under 100 KB/device/day — less than one web page.
Privacy / consent	Much stronger story: audio never leaves the room	Requires notice, DPA, no-retention terms, regional endpoints	**The only factor that genuinely favours device**, and it is a serious one. Mitigated, not eliminated, by §57.
WAN outage	Keeps working	Fails	Real, but addressed by the on-device fallback grammar (§15.5) at a fraction of the cost of full on-device ASR.

> **DECISION D1 — STT PLACEMENT**
>
> **Cloud streaming ASR is the primary path.** The device performs AEC, beamforming, wake-word detection, VAD and endpointing, and streams Opus only between wake-word fire and end-of-turn. Audio is held in memory in the cloud for the duration of recognition and is **never written to disk**.
>
> **Plus a small on-device command grammar** (§15.5) covering roughly 25 utterances, used only when the WAN is unavailable. This buys the outage resilience that motivates on-device ASR without paying its accuracy, hardware or maintenance costs.
>
> **Revisit trigger:** re-evaluate full on-device ASR when (a) a chain contractually requires audio never to leave the property, or (b) monthly ASR spend exceeds the amortised BOM delta of a better SoC across the fleet — whichever comes first. Keep the `AsrProvider` interface clean so that day is a swap, not a rewrite.

15.4 The hybrid variant worth keeping in your pocket

There is a third placement nobody asks about and it fits this product unusually well: **ASR on the property's Edge Gateway**. Audio leaves the room but never leaves the building. A single edge appliance with a modest GPU or a capable CPU can serve a few hundred rooms at realistic concurrency, because hotel voice traffic is sparse and bursty. This gives the privacy story of on-device (audio stays on-premises — a much easier conversation with a hotel's legal team), the per-property vocabulary biasing of cloud, zero per-minute fees, and WAN-outage resilience. What it costs is edge hardware, fleet-wide model management, and accuracy somewhat below the best cloud models.

> **RECOMMENDATION ON EDGE ASR**
>
> Do not build it now — it is a distraction from proving the agent. But **design the Edge Gateway with enough headroom to host it later** (spec the appliance with an upgrade path), and keep `AsrProvider` implementable at the edge. This becomes your answer when a European or Gulf chain says "guest audio may not leave our premises" — and one of them will.

15.5 The offline command grammar

A fixed, on-device, ~25-phrase recogniser (the same engine family as the wake word handles this well) covering only the actions the device can perform through its local fallback path: lights on/off/dim, temperature up/down, DND on/off, curtains, TV power, and "call the front desk". When the WAN is down, the device announces the limitation once ("I'm offline right now — I can still do lights, temperature and do-not-disturb") and operates on this grammar against the Edge Gateway's local rules. Everything else gets a clear refusal and a pointer to the room phone.

16. ASR vendor selection

Vendor	Indicative price†	Streaming finalisation	Phrase boosting / custom vocab	Residency & retention	Assessment
Deepgram Nova-3	~$0.006–0.008/min	Very fast (~100–200 ms)	Keyterm prompting; strong	Self-hosted / VPC deployment available — unusual and valuable	**Primary candidate.** Best latency/price/controllability combination. The on-prem option is a genuine differentiator for the residency conversations in Part XI.
Google STT v2 (chirp_2)	~$0.016/min	Fast (~150–300 ms)	Speech adaptation / phrase sets; good	Regional endpoints incl. EU, India; data-use controls	**Co-primary.** Strongest multilingual and accented-English coverage; you are already on Google for the LLM, which simplifies contracts and residency posture.
AssemblyAI Universal-Streaming	~$0.0025–0.005/min	Very fast	Word boost	US/EU regions	Cheapest credible option; strong English, weaker breadth outside it. Good cost lever later.
Azure Speech	~$0.016/min	Fast	Custom Speech (trainable models)	Broad regional coverage incl. UAE	Best regional footprint if Gulf residency becomes contractual. Heaviest integration.
AWS Transcribe streaming	~$0.024/min	Moderate	Custom vocabulary	Native to your AWS estate	Convenient, but the most expensive and not the most accurate. Not recommended as primary.
Self-hosted Whisper/Parakeet	GPU cost only	Variable	Full control	Total control	Only once volume justifies GPU fleet ops. Keep as the Phase-3 cost lever.

> **DECISION D3 — ASR**
>
> Build the `AsrProvider` abstraction first, then **bake off Deepgram Nova-3 against Google STT v2 (chirp_2) on your own recorded hotel audio** — not on public benchmarks, which are useless here. The evaluation set must be: 500+ real utterances from a live property, weighted toward accented English, with TV and HVAC noise present, including menu items and property proper nouns. Measure WER, but weight **entity error rate** (did it get the room number, the menu item, the temperature) far higher than raw WER, because that is what determines whether the turn succeeds.
>
> **Ship with two vendors wired** from day one, selected per property by config. This is not over-engineering: it is how you survive a vendor outage, how you negotiate price, and how you satisfy a chain that insists on a specific processor.

17. TTS and response audio

Option	Indicative price†	TTFB	Voice quality / brand	Assessment
Google Chirp3-HD	~$30/M chars	~200–400 ms	Excellent, broad multilingual	**Primary.** Same vendor as the LLM, strong multilingual coverage, good regional endpoints.
Amazon Polly (Neural / Generative)	~$16 / ~$30 per M chars	~150–300 ms	Good; Generative is markedly better	**Fallback.** Already in the codebase (V1 uses it on-device), lowest integration cost, native to AWS. Move it server-side.
ElevenLabs Flash v2.5	~$50–80/M chars	~75–150 ms	Best-in-class naturalness; voice cloning	Premium tier only. Where it wins: a chain that wants its own brand voice. Price it as an upsell, not a default.
Cartesia Sonic	~$20–40/M chars	~90 ms	Very good, extremely fast	Strongest pure-latency option. Worth qualifying if perceived responsiveness becomes the binding constraint.
On-device TTS	Free	~0 ms	Poor to acceptable	Keep *only* for offline mode and earcons. A robotic voice undermines the premium positioning.

17.1 The phrase cache — the highest-value optimisation in the audio path

A large fraction of what the assistant says is drawn from a small set of phrases: acknowledgements ("Sure, one moment"), confirmations ("Done", "I've set it to 23 degrees"), clarifications ("Which room would you like that in?"), errors, and greetings. These should be **pre-synthesised per property voice per language and cached as Opus on the device itself**, not fetched per turn.

- **Latency:** the acknowledgement plays in <20 ms instead of ~300 ms — and it is the acknowledgement, not the answer, that determines whether the device feels fast.
- **Cost:** plausibly 30–45% of TTS characters eliminated.
- **Resilience:** the device can still speak when the TTS vendor is down.

Implementation: a versioned phrase pack per `(property_voice, language)`, pushed to devices on provisioning and on voice-config change, with numeric slot substitution handled by a small set of pre-synthesised number fragments for the most common cases (temperatures 16–30, times, small integers).

18. Cascaded vs speech-to-speech (Gemini Live)

Gemini's Live API accepts audio and emits audio natively, removing two hops. It is genuinely faster and the prosody is better. It is also the wrong primary architecture for this product, for reasons that are about governance rather than capability.

Dimension	Cascaded (ASR→LLM→TTS)	Native speech-to-speech
Latency	~1.0–2.4 s to first audio	~0.5–1.0 s — clearly better
Intermediate text	Transcript and response text exist as first-class artefacts	**May not exist reliably** — and everything below depends on it
Audit trail	Exact text of what was heard and said, per turn	Reconstructed, approximate, or absent — unacceptable for a system that spends money and moves hardware
PII redaction before the model	Possible — redact the transcript	Not possible — raw audio goes to the model
Safety filtering	Filter text in and out	Much harder
Evaluation	Text-in/text-out evals are cheap and deterministic	Audio evals are expensive and noisy
Vendor lock-in	Three swappable components	One vendor owns the entire interaction
Cost	ASR + tokens + TTS, each individually optimisable	Audio tokens, materially more expensive per turn
Brand voice	Any TTS vendor, any cloned voice	Constrained to the provider's voices
Barge-in	Implemented by you	Native and excellent

DECISION D2 — PIPELINE SHAPE

Cascaded is the production path. The intermediate transcript is not an implementation detail — it is the artefact that the audit trail, the redaction pipeline, the eval harness and the safety filters are all built on. Trading it for ~700 ms is a bad trade when §20.4 recovers most of that perceived latency for free.

But build a Live-API lane behind a per-property feature flag, restricted to a defined *conversational* subset — small talk, local recommendations, property information — where no T2+ action can be taken. Route to it only when the router classifies the turn as chat rather than task. This gives you the best-in-class feel where it is safe, the auditable pipeline where it matters, and real operational data on the Live API before you ever consider promoting it.

19. Device↔cloud transport

Option	Assessment in a hotel environment	Verdict
WebSocket over TLS (443)	Traverses hotel firewalls and proxies reliably because it looks like HTTPS. Bidirectional, low overhead, trivially multiplexes audio frames and JSON control messages. Mature libraries everywhere including Android.	**Chosen.** The only option that survives real hotel networks without negotiation.
WebRTC	Superior media handling — jitter buffer, adaptive bitrate, native AEC, packet loss concealment. But: STUN/TURN infrastructure, ICE negotiation, UDP that hotel networks frequently block, and far more moving parts.	Rejected as primary. Reconsider only if packet loss on real properties proves unmanageable — measure before assuming.
gRPC bidirectional streaming	Clean typed contract (and already in the codebase for Google STT). But HTTP/2 is more often broken by intercepting middleboxes than WebSocket, and browser/proxy compatibility is worse.	Use for *service-to-service* internally. Not for device↔cloud.
MQTT	Excellent for telemetry and command/control — low overhead, QoS levels, retained messages, native to AWS IoT Core. Poor for continuous audio streaming.	**Complementary.** Use MQTT for device telemetry, health, state reporting and OTA signalling on a *separate* channel; keep audio on the WebSocket.

DECISION D5 — TRANSPORT SPECIFICATION

Audio channel	WebSocket over TLS 1.3, port 443, mTLS with per-device X.509 client certificate
Codec	Opus, 16 kHz mono, 20 ms frames, ~16 kbps VBR, DTX enabled
Framing	Binary frames = audio; text frames = JSON control (`turn.start` , `turn.end` , `barge_in` , `ack` , `screen.update` , `health`)
Connection	One persistent connection per device, kept warm. Establishing TLS per turn would add 200–400 ms to every interaction — this is why it is persistent.
Keepalive	WS ping every 30 s; server closes at 90 s of silence
Reconnect	Exponential backoff with full jitter, 1 s → 60 s cap. **Jitter is mandatory** — without it, a property-wide network blip becomes a synchronised reconnect storm from 300 devices.
Resume	Session token allows resuming an in-flight turn within 10 s of a drop
Telemetry channel	MQTT over TLS to AWS IoT Core: health, state, wake-word FA counters, OTA status. Deliberately separate so telemetry backpressure can never affect the voice path.
Backpressure	Server-signalled; device drops to a lower bitrate before it drops frames, and never buffers more than 2 s of audio

20. Latency budget

This table is the contract the whole voice path is built against. If a component exceeds its allocation, it is a bug, not a tuning opportunity.

Stage	p50	p95	Notes
Wake word detection → capture start	30 ms	60 ms	On-device. Earcon fires here.
Guest speaks	–	–	Typically 1.5–4 s; ASR runs concurrently, so this is not in the budget
Endpoint detection (device)	300 ms	450 ms	Adaptive trailing-silence threshold. Tunable per property; this is the biggest UX/accuracy trade-off knob.
Final audio frames → cloud	35 ms	120 ms	Hotel Wi-Fi to regional AWS. Assumes an in-region cell.
ASR finalisation	140 ms	300 ms	Only the tail — interim results already streamed
Redaction + routing	8 ms	20 ms	Local computation
— FAST LANE branch —			
Fast-path match + PDP	10 ms	25 ms	No LLM call at all
Tool exec (cached-state action)	180 ms	600 ms	Via edge gateway to BMS
TTS first byte (cached phrase)	15 ms	30 ms	Phrase pack on device
FAST LANE TOTAL (end of speech → first audio)	**~590 ms**	**~1,320 ms**	**Covers roughly 55–65% of turns**
— AGENTIC LANE branch —			
Context assembly	25 ms	70 ms	State pre-fetched in parallel (§27.2)
Gemini Flash first token	380 ms	850 ms	Cross-cloud AWS→Vertex; measure per region (§76)
Gemini complete tool proposal	180 ms	450 ms	Structured output
PDP evaluation	3 ms	8 ms	Hard budget — must never be a network call to a cold service
MCP gateway + tool execution	220 ms	750 ms	Dominated by the vendor head-end
Response composition (2nd LLM pass)	300 ms	700 ms	Avoidable for many turns via templated confirmations
TTS first byte (synthesised)	220 ms	420 ms	Streaming
Network back to device + buffer	45 ms	140 ms	
AGENTIC LANE TOTAL	**~1,900 ms**	**~4,300 ms**	**Unacceptable as raw numbers — see below**
PERCEIVED latency (with filled pause)	**~600 ms**	**~900 ms**	**Because the guest hears a response at the acknowledgement, not the answer**

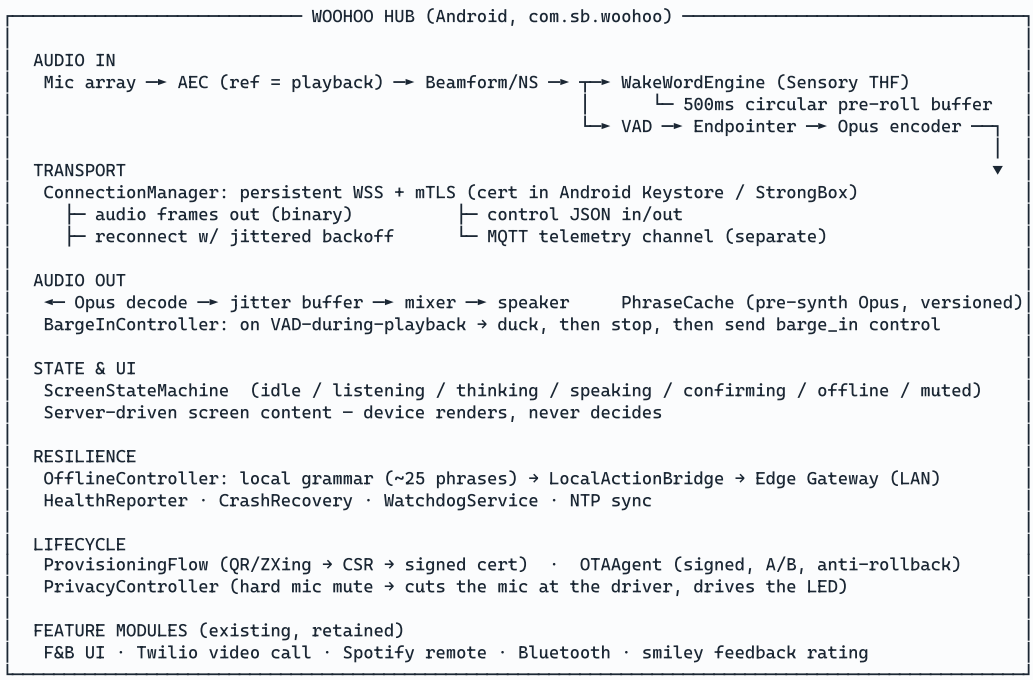
20.4 Why perceived latency is the only number that matters

The agentic lane's p95 of 4.3 seconds would be a product-killing figure if the guest experienced it as silence. They do not, because the orchestrator emits a **filled pause from the device's cached phrase pack the moment the router selects the agentic lane** — before the LLM has even responded. The guest hears "Sure, one moment—" at ~600 ms and then the substantive answer flows in behind it.

This is not a trick; it is how humans handle the same problem. Three rules make it work:

- **Vary the phrase.** Rotate through 6–8 acknowledgements per language. A device that says exactly the same three words every time sounds broken within a day.
- **Only fill when you will be slow.** Fast-lane turns must never get a filled pause — it makes a 600 ms interaction feel slower, not faster.
- **Never acknowledge an outcome you have not achieved.** "One moment" is honest; "Done!" before the BMS has acked is a lie the guest will catch when the lights do not change.

21. Device software architecture



21.1 What must change on the device

Change	Why	Effort / risk
Remove all cloud credentials — `google-services.json` service-account path, Polly SDK keys	A stolen device must yield nothing but a revocable, room-scoped certificate. This is the finding that fails a chain security review.	Medium. Highest-priority change.
Replace direct Google STT gRPC with the Woohoo WebSocket	Puts the ASR vendor behind an abstraction; lets you change vendors, models and phrase hints without an OTA; removes credentials.	Medium. The core refactor.
Move TTS server-side; add the phrase cache	Per-property brand voices, multilingual, no per-device credentials, and better latency for the common phrases.	Medium-low.
Add device-side endpointing with an explicit end-of-turn signal	Saves 200–400 ms per turn (§15.1).	Medium — needs real tuning on real hardware.
X.509 identity in hardware keystore + CSR-based provisioning	Per-device revocable identity, bound to the room.	Medium-high; touches the provisioning flow.
Barge-in support	Interrupting a talkative assistant is the single most-requested behaviour in voice UX.	Medium; depends on AEC quality.
Offline controller + local grammar	Hotel WANs fail. The device must still dim the lights.	Medium.
Hard mic mute cutting the mic in the driver, with LED	Compliance and guest trust. A software mute is not a mute.	Low-medium; may need hardware support.
Replace socket.io with the unified WS control channel	One connection, one auth model, one reconnect strategy.	Low-medium.
Modernise: Kotlin, coroutines, current AGP, remove Fabric	Hygiene. Not urgent, but the Java/Dagger/MVP stack from 2019 will slow every subsequent change.	Ongoing.

22. Hotel network reality

Hotel networks are the most hostile environment this system will run in, and the failure modes are consistent enough to design for specifically.

What actually happens	Effect	Mitigation
Devices put on guest Wi-Fi	Captive portal intercepts, client isolation blocks LAN access to the edge gateway, bandwidth shaping, session timeouts	**Contractual requirement: dedicated IoT VLAN, wired where possible.** Make this a precondition of installation, not a request afterwards. Detect and alarm if a device appears on a guest subnet.
AP density designed for phones, not 300 always-connected devices	Association failures, high retry rates, roaming problems in corridors	Site survey during onboarding; per-device RSSI telemetry with a "this room has poor coverage" report for the hotel
The BMS is on an isolated network with no route to the internet	Cloud cannot reach the head-end at all	**The Edge Gateway (§40).** This is the primary reason it exists.
Hotel IT will not open inbound ports and will not sign off on a site-to-site VPN quickly	Integration stalls for months	Outbound-only architecture, single 443 egress. Make the firewall ask a one-line request.
Deep packet inspection / TLS interception on the corporate network	mTLS breaks; cert pinning breaks	Document required allowlist domains; detect interception and report it clearly rather than failing opaquely
WAN outages, often at night when maintenance runs	Whole property loses voice control	Edge local fallback + on-device grammar; optionally cellular failover on the edge appliance for large properties
Renovation and room reassignment without telling anyone	Device in room 412 is now in 414; commands go to the wrong room — a serious safety and privacy bug	**Room-binding verification:** periodic correlation of device identity with the automation system's own room identifier, plus an alert when they disagree. Never trust the last-known mapping silently.

> **THE MOST UNDER-APPRECIATED RISK IN THIS LIST**
>
> **Device-to-room mis-binding.** If a device is moved during a refurbishment and the catalogue is not updated, a guest in 414 will control the lights in 412 — and the audit log will confidently record it as correct. This is both a guest-safety issue and a privacy issue, and it will not surface as an error anywhere unless you build detection for it. Treat room binding as a continuously verified fact, not a provisioning-time fact.

PART V

The Agentic Core

23. Orchestration pattern

Four patterns are viable. The choice is driven almost entirely by the latency budget, because a voice interface has no loading spinner.

Pattern	How it works	Why it fails here	Where it's still useful
Single ReAct agent	One model, one flat tool list, loop until done	Every turn pays a full LLM round-trip. "Turn on the lights" costs 800 ms and a token bill it should not. Tool list grows past the point where selection stays reliable.	As the *agentic lane* inside a router — which is what we do.
Router + specialist sub-agents	A dispatcher classifies, then hands to a domain agent (automation / service / info)	Adds a serial LLM hop to every turn (~400 ms) to solve a problem that a well-designed tool surface does not have. Cross-domain requests ("cold *and* fix the shower") need re-joining.	If the tool surface ever exceeds ~40 tools. Design so it does not.
Plan-and-execute	Model produces a multi-step plan, then executes it	Wrong shape for the workload. Hotel requests are 1–3 actions with no meaningful dependency graph. Planning overhead buys nothing.	Long-running staff-side workflows ("prepare the room for a VIP arrival"), not guest turns.
Hybrid fast-path + agent	Deterministic matcher handles the head of the distribution; LLM handles the tail	—	**Chosen.** See below.

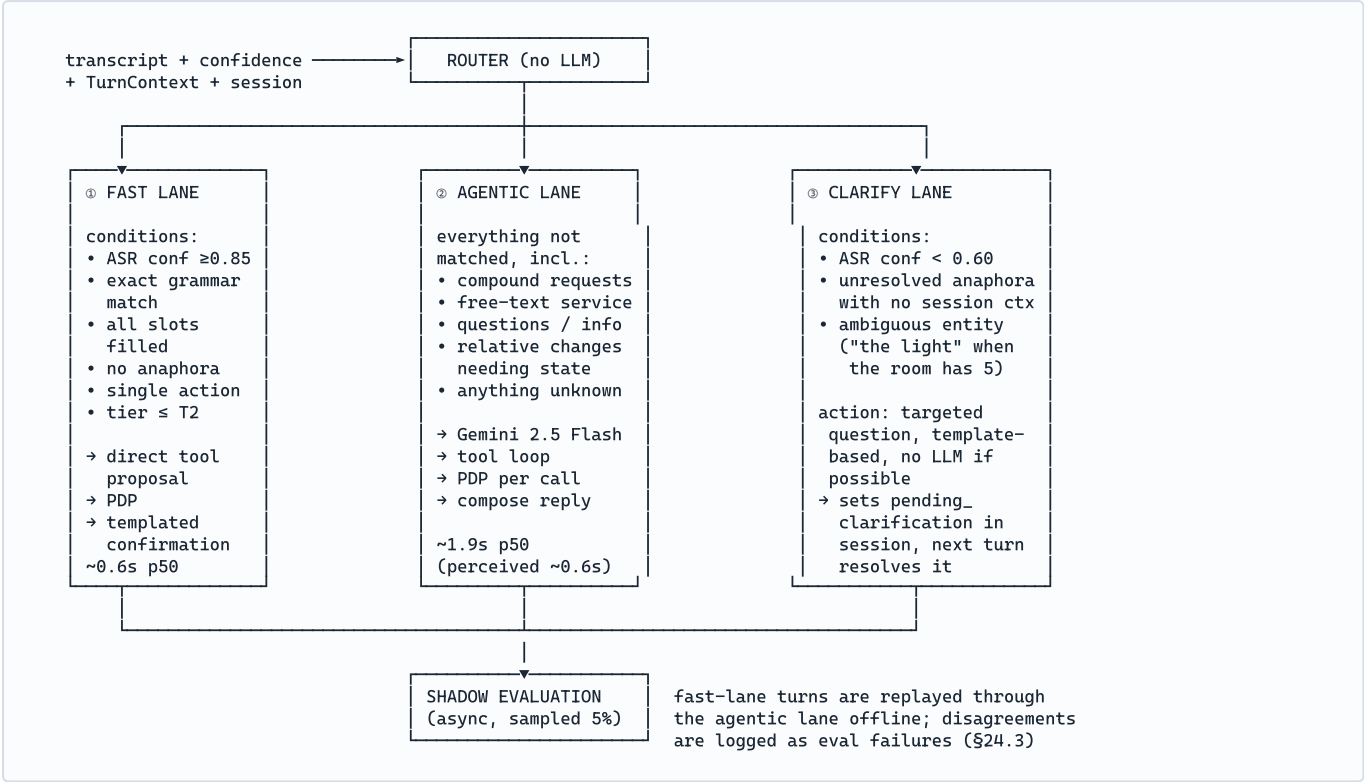
23.1 The distribution argument

Hotel voice traffic is extraordinarily head-heavy. Across deployed in-room assistants the top ~20 utterance families — lights, temperature, DND, curtains, TV, checkout time, Wi-Fi password, housekeeping, towels, wake-up call — account for the majority of all traffic, and they are phrased in a narrow range of ways. Paying full agentic reasoning for "turn off the lights" is spending 800 ms and a Gemini call to answer a question a 50-line matcher answers in 4 ms with higher reliability.

The counter-argument — that a fast path is technical debt that will rot — is real, and it is why the fast path in this design is **generated and validated by the eval harness** rather than hand-maintained (§24.3). If a fast-path rule ever disagrees with what the agent would have done, that is a test failure, and the rule is removed.

> **DECISION D7 — THREE-LANE ROUTER**
>
> Every turn is classified into exactly one of three lanes by a deterministic classifier that costs no LLM call:
>
> - **Fast lane** — high-confidence match against a curated grammar of top intents, with all slots filled and no ambiguity. Executes directly through the PDP. Target: 55–65% of turns, p50 under 600 ms.
> - **Agentic lane** — everything else. Full Gemini reasoning over the capability tool surface. Target: 30–40% of turns.
> - **Clarify lane** — ASR confidence below threshold, or a known-ambiguous reference with no session context to resolve it. Emits a targeted clarifying question without an LLM call where possible. Target: <8% of turns.

24. The three-lane router



24.1 Fast lane escalation

The fast lane must be able to admit defeat. If the matched action's PDP verdict is anything other than `ALLOW`, or the tool execution returns an error, or the resolved entity is ambiguous, the turn **escalates to the agentic lane** rather than failing. The guest never learns which lane served them; they only notice that some answers arrive faster.

24.2 What the fast lane may never do

- Anything at risk tier T3 or above (§44). Money and irreversible actions always get full reasoning plus explicit confirmation.
- Anything requiring a state read to decide the target value (“a bit warmer”) — unless the state is already in the pre-fetched snapshot *and* the delta rule is unambiguous.
- Anything with a free-text field. Free text means judgement; judgement means the agent.
- Anything when the session has a pending clarification.

24.3 Keeping the fast lane honest

Five percent of fast-lane turns are asynchronously replayed through the agentic lane. Where the agent would have chosen a different action, the disagreement is recorded. A rule whose disagreement rate exceeds 2% is automatically flagged and, if not fixed within a release cycle, disabled. This turns the fast path from accumulating debt into a continuously validated cache of the agent's own behaviour — which is exactly what it should be.

25. Model selection and routing

Stage	Model	Why	Budget	Fallback
Lane classification	**No model** — deterministic grammar + embeddings	Must cost <10 ms	—	—
Main agentic turn	**Gemini 2.5 Flash**	Best latency/capability/cost balance; strong function calling; thinking budget controllable	≤6 k input, ≤600 output, thinking budget capped low	Gemini Flash-Lite → degraded mode (§29)
Short-utterance classification & slot filling	**Gemini 2.5 Flash-Lite**	Cheapest, fastest; sufficient for constrained structured output	≤2 k in, ≤120 out	Flash
Response composition (when not templated)	Same call as the agentic turn	A second LLM pass costs ~300 ms — avoid it whenever a template will do	—	Template
Complex escalation / staff-side summarisation	**Gemini 2.5 Pro**	Off the guest latency path; quality matters more than speed	Generous	Flash
Offline eval judging	**Gemini 2.5 Pro** + rule-based assertions	Never in production path	—	—
Embeddings (RAG, fast-lane similarity)	Gemini embedding model	—	—	Local model

DECISION D8 — MODEL POLICY

Gemini 2.5 Flash is the default, with the thinking budget explicitly capped — extended reasoning is a latency disaster on a voice path and rarely improves a request as simple as "dim the lights". Model IDs are **configuration, not code**, resolvable per property and per lane, so a model upgrade is a staged rollout rather than a deploy.

Keep one non-Google fallback wired but dormant. Not because Gemini is unreliable, but because a single-provider dependency on the reasoning core of a product installed in guest rooms is a commercial risk a hotel chain's procurement team will raise. The `LlmProvider` interface plus a smoke-tested alternate path is cheap insurance; you do not need to run it.

Context caching: the system prompt, tool schemas and property-invariant context are stable across turns and across guests in the same property. Cache them. At scale this is a large fraction of input tokens and it also cuts time-to-first-token.

26. Prompt architecture and context assembly

The prompt is a **versioned build artefact**, assembled from layers with different lifetimes and different trust levels. Treating it as a string in the codebase is how agent systems become unshippable.

```
┌─ LAYER 0 · SYSTEM IDENTITY ──────────────────────── versioned artefact, changes ~monthly, CACHED ─┐
│ Who the assistant is, tone, hard behavioural rules, refusal policy, escalation policy,           │
│ the "you propose, the platform disposes" contract, output format contract.                       │
│ NEVER contains tenant data. Identical for every guest in every hotel.                            │
├─ LAYER 1 · BRAND & PROPERTY VOICE ──────────────── per property, changes ~quarterly, CACHED ──────┤
│ Property name, brand tone ("warm and concise", "formal"), languages, forms of address,           │
│ things this property does NOT offer, property-specific phrasing rules.                           │
├─ LAYER 2 · CAPABILITY SURFACE ──────────────────── per capability-manifest version, CACHED ──────┤
│ The tool schemas available in THIS room. Derived from the property capability manifest —          │
│ a room without motorised curtains has no curtain tool, so the model cannot propose one.          │
├─ LAYER 3 · TURN CONTEXT ──────────────────────────── per turn, NOT cached ──────────────────────┤
│ Room label (not number, where avoidable) · local time · language detected · occupancy hints ·    │
│ current device state snapshot (pre-fetched) · open tickets for this room · stay context          │
│ (tokenised: "guest checking out tomorrow at 11" — never the guest's name unless needed and consented)
├─ LAYER 4 · CONVERSATION ──────────────────────────── per turn ──────────────────────────────────┤
│ Last N turns (N=3 default, 6 if a clarification is pending). Text only. Redacted.                 │
├─ LAYER 5 · UNTRUSTED CONTENT ─────────────────────── per turn, EXPLICITLY FENCED ──────────────┤
│ Guest utterance, tool results, retrieved knowledge, staff-authored content (menus, FAQ).         │
│ Wrapped in explicit untrusted-content delimiters with a standing instruction that                │
│ NOTHING inside them is ever an instruction. See §46.                                             │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
```

26.1 Rules for context assembly

- **Layers 0–2 are cacheable and must be byte-stable** across turns. Any per-turn variation in them destroys the cache hit and costs both money and time-to-first-token. Do not put a timestamp in Layer 0.
- **Layer 3 is where the pre-fetched state lands.** Its assembly must be non-blocking — if the state read has not returned within its budget, the layer is assembled without it and the model is told state is unavailable rather than the turn being delayed.
- **Layer 5 is never concatenated into Layers 0–4.** The separation is structural, and it is the primary defence against prompt injection.
- **Nothing in any layer is a security control.** If a rule in Layer 0 matters, there is a corresponding enforcement in the PDP. The prompt exists to make the model behave well, not to make it safe.

27. The agent loop, step by step

```
┌─ 1. PRE-FETCH (starts at t≈+50ms, in parallel with the guest still speaking) ──────────────┐
│    • room device state snapshot        • open tickets for the room                        │
│    • capability manifest (cached)      • stay context from PMS (cached, TTL 15m)          │
│    These are speculative reads. They cost nothing on the critical path and remove a full  │
│    tool round-trip from the majority of agentic turns. THIS IS THE SINGLE BIGGEST LATENCY WIN.
└──────────────────────────────────────────────────────────────────────────────────────────┘
                                          │
                                          ▼
┌──────────────────────────────────────────────────────────────────────────────────────────┐
│ 2. ASSEMBLE CONTEXT   layers 0–5 (§26). Token budget enforced: hard cap, oldest conversation
│    turns dropped first, then state detail summarised. Never silently exceed — log and truncate.
├──────────────────────────────────────────────────────────────────────────────────────────┤
│ 3. INVOKE GEMINI      structured output, function calling enabled, thinking budget capped,
│    streaming on. Timeout 4s hard, 2.5s soft (soft = start the filled pause).
├──────────────────────────────────────────────────────────────────────────────────────────┤
│ 4. PARSE PROPOSALS    zero or more tool calls + optional draft reply. Validate against schema.
│    Schema violation → one repair attempt with the error message → then degraded mode.
├──────────────────────────────────────────────────────────────────────────────────────────┤
│ 5. POLICY (per proposed call, independently) ──► ALLOW / CONFIRM / DENY / ESCALATE
│    Verdicts written to the audit record BEFORE any execution.
│    CONFIRM → the turn ends here with a spoken confirmation question; the pending action is
│    stored in session with a 90s TTL and a single-use token. It is NOT re-derived next turn.
├──────────────────────────────────────────────────────────────────────────────────────────┤
│ 6. EXECUTE            allowed calls dispatched via MCP Gateway, in parallel where independent.
│    Idempotency key = hash(turn_id, tool, canonical_args). Per-call timeout by tool class.
├──────────────────────────────────────────────────────────────────────────────────────────┤
│ 7. VERIFY             for state-changing automation, read back the resulting state. "I set it"
│    must mean "I observed it set". Unverified writes are reported as "I've asked for that".
├──────────────────────────────────────────────────────────────────────────────────────────┤
│ 8. COMPOSE            templated confirmation where the outcome fits a known shape (most turns);
│    second LLM pass only when results need genuine synthesis. Saves ~300ms on the common case.
├──────────────────────────────────────────────────────────────────────────────────────────┤
│ 9. GUARD              output check: no other-room information, no invented amenities, no PII
│    leakage, length cap (a voice reply over ~35 words is a design failure), language match.
├──────────────────────────────────────────────────────────────────────────────────────────┤
│ 10. SPEAK + RECORD    stream TTS; seal and emit the audit record; update session; emit events.
└──────────────────────────────────────────────────────────────────────────────────────────┘

LOOP CONTROL:  max 3 tool-call rounds per turn · max 6 tool calls total · hard 8s wall clock ·
               token budget enforced · on any breach → apologise, escalate to staff, log for review.
               An agent that is still looping after 8 seconds has failed, whatever it eventually returns.
```

27.2 Why pre-fetch matters more than it looks

In a naive implementation, "make it warmer" costs: LLM call (→ proposes `get_state`), tool round-trip, second LLM call (→ proposes `set_temperature`), tool round-trip. That is two LLM calls and two tool round-trips — roughly 1.6 s of avoidable serial latency and double the token cost. With the state snapshot already in Layer 3, it is one LLM call and one tool round-trip. Across the fleet this is the difference between a viable and an unviable unit cost, and it is why §27 step 1 begins before the guest has finished speaking.

28. Session, memory and disambiguation

Layer	Contents	Store	TTL	Notes
Turn	Transcript, context digest, proposals, verdicts, results	In-memory → audit	Turn	Never reused as input except via the conversation layer
Conversation	Last 6 turns, last-referenced entity, pending clarification, pending confirmation token	Redis	**30 min** idle	Short deliberately. A guest returning after two hours is starting fresh, and stale context causes wrong-target actions.
Stay	In-stay learned preferences (preferred temperature, usual wake time, dietary note), open tickets, spend to date	Redis + Postgres	**Checkout + 24 h**, then purged	Hard-deleted on checkout event from the PMS. This is the layer that makes the assistant feel attentive without being creepy.
Cross-stay	Preferences persisted across visits, keyed to a loyalty identity	Postgres, encrypted	Until withdrawn	**Explicit opt-in only**, per §55. Off by default. This is the highest-value and highest-risk feature in the product; do not ship it in Phase 1.
Property	FAQ, menus, hours, policies, local recommendations	Postgres + pgvector	Until edited	Not guest data. Retrieved on demand, cited, and treated as untrusted content (§46).

28.1 Anaphora and disambiguation

"Turn it off", "make it cooler", "the other one", "same as yesterday" — these are the utterances V1 could not handle, and getting them right is most of the perceived intelligence gain of V2.

- **Last-referenced entity** is tracked explicitly in the conversation layer (entity type, ID, label, turn index) rather than left for the model to infer from history. Explicit beats implicit for anything that determines which physical device gets actuated.
- **Resolution order:** (1) explicit reference in the utterance; (2) last-referenced entity if within 2 turns; (3) if exactly one device of the plausible type exists in the room, use it; (4) otherwise **ask**.
- **Never guess between two physical devices.** A wrong guess on "turn off the light" in a suite with five zones is a bad experience; a wrong guess that turns off a light the guest was reading by is worse than asking. The clarify lane exists for this.
- **Relative changes require state.** "Warmer" with no known current setpoint is a clarification, not a guess. The pre-fetched snapshot means this is almost always available.

29. Deterministic degraded mode

Gemini will be unavailable, rate-limited or slow at some point. What happens then is a product decision, and the default — an error message — is the wrong one.

```
LEVEL 0 · NORMAL          Fast lane + agentic lane + clarify lane. Full capability.

LEVEL 1 · LLM SLOW        Trigger: p95 LLM latency > 2.5s over a 60s window.
                          Action: cap thinking budget to zero, drop to Flash-Lite for slot filling,
                          widen fast-lane matching thresholds, always emit the filled pause.
                          Guest impact: none noticeable.

LEVEL 2 · LLM DEGRADED    Trigger: LLM error rate > 10%, or p95 > 4s.
                          Action: fast lane only. Anything that does not match the grammar gets:
                          "I can help with lights, temperature, do-not-disturb and service requests
                           right now. For anything else let me connect you to the front desk."
                          Escalation path to staff stays fully open. Guest impact: reduced scope,
                          still useful. THIS IS THE MOST IMPORTANT LEVEL TO GET RIGHT — it is the
                          difference between a degraded product and a broken one.

LEVEL 3 · ASR DOWN        Trigger: both ASR vendors failing.
                          Action: on-screen touch UI on the device takes over — the screen becomes
                          the interface. Voice announces the switch once. All automation and service
                          requests remain available by touch.

LEVEL 4 · WAN DOWN        Trigger: device cannot reach cloud.
                          Action: on-device grammar (~25 phrases) → LAN → Edge Gateway → BMS.
                          Lights, climate, DND, curtains, TV still work. Service requests are queued
                          locally and flushed on reconnect, and the guest is TOLD they are queued.
                          Screen shows an offline indicator.

LEVEL 5 · EDGE DOWN       Trigger: Edge Gateway unreachable and cloud unreachable.
                          Action: device announces the failure plainly and points to the room phone.
                          A honest "I can't reach the room systems right now, please use the phone"
                          beats silence or a lie. Health alert fires to ops.
```

THE DEGRADED-MODE PRINCIPLE

At every level the assistant **says what it can still do**, rather than what it cannot. “I can help with lights, temperature and do-not-disturb right now” keeps the guest using the product; “Sorry, I'm having trouble” teaches them to stop trying. The set of capabilities available at each level is configuration, so it can be tuned per property without a release.

PART VI

MCP & Tool Architecture

30. Why an MCP gateway is mandatory

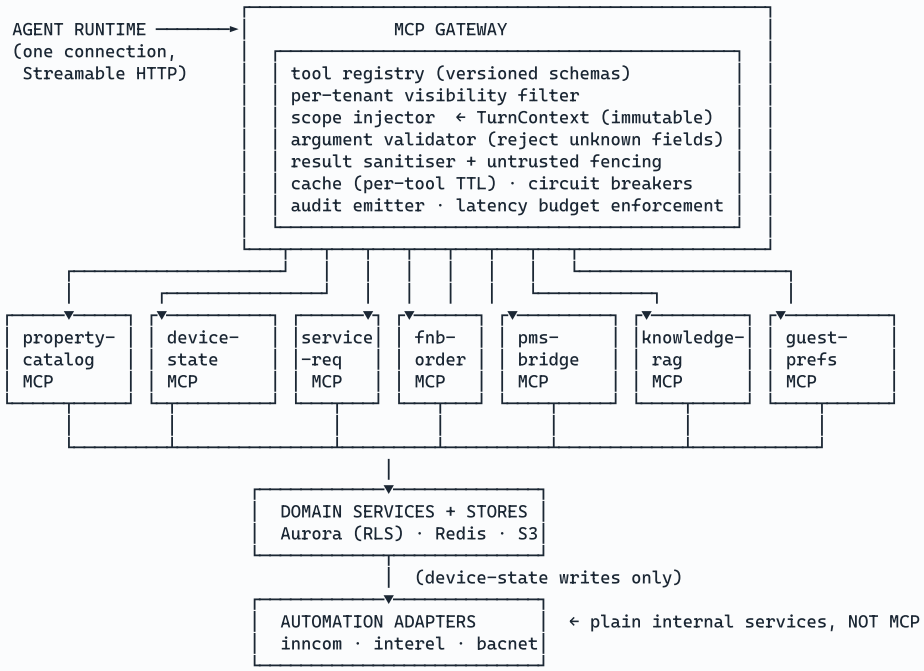
You already expose hotel, room, device and service data as MCP servers. The instinct is to point the agent at them directly — that is what MCP is for. For a single-tenant developer tool, that instinct is right. For a multi-tenant system that actuates physical devices in strangers' bedrooms, it is wrong, and the reasons are worth being precise about.

Problem with direct agent→MCP	What goes wrong	What the gateway does instead
Scope enforcement	If the agent calls the Device State server directly, then the arguments — including which room — come from model output. A prompt injection or a model error can address another room. Nothing structural prevents it.	The gateway **injects** tenant/property/room from the authenticated TurnContext and **strips** any scope fields the model supplied. Cross-room addressing becomes impossible rather than unlikely.
Per-tenant tool visibility	Every hotel sees every tool. A property without curtains gets a curtain tool the model will happily call.	The gateway filters the advertised tool set by the property's capability manifest before the agent ever sees it.
Result sanitisation	Tool results flow straight into the model's context. A menu description edited by a hotel employee is an injection vector.	All results are wrapped in untrusted-content fencing and scanned before entering context (§46).
Connection management	N servers × M concurrent turns = connection explosion, per-turn handshakes, unbounded latency.	Pooled, warm, multiplexed connections with per-server circuit breakers.
Caching	The property catalogue is read on every turn and changes weekly. Without a cache, it is the hottest and most wasteful query in the system.	Per-tool cache policy with declared TTLs and explicit invalidation on catalogue events.
Uniform audit	Each server logs differently, or not at all.	One place records every tool call, its arguments, its verdict, its latency and its result digest.
Error shaping	Raw vendor errors reach the model. "INNCOM: ERR_0x8004" produces either a hallucinated explanation or a useless apology.	Errors are normalised into a small vocabulary the model is trained by the prompt to handle: `NOT_AVAILABLE`, `NOT_PERMITTED`, `TEMPORARILY_UNREACHABLE`, `NEEDS_CONFIRMATION`, `INVALID_ARGUMENT`.
Schema versioning	Changing a tool schema breaks prompts silently and only in production.	Schemas are versioned artefacts pinned per prompt version, validated in CI by the eval harness.
Confused deputy	The agent's credentials are broad; a downstream server cannot tell whether a call reflects a guest's request or an injected instruction.	Per-turn, audience-bound, short-lived tokens carrying the exact scope, minted by the gateway.

DECISION D9 — MCP GATEWAY

The agent holds exactly one MCP connection: to the internal MCP Gateway. Downstream MCP servers are never addressable by the agent runtime. The gateway is the tool plane's control point — registry, filter, scope injector, sanitiser, cache, circuit breaker and audit point — and it is where the "the model proposes, the platform disposes" principle is made mechanical.

31. MCP server topology



31.1 Should vendor adapters be MCP servers?

No. This is worth stating plainly because it is a tempting symmetry. MCP is a protocol for exposing capabilities *to a model*. The automation adapters must never be visible to the model — the entire point of principle P4 is that the agent reasons about "a thermostat with a setpoint" and has no concept of INNCOM. If `inncom-adapter` were an MCP server, its vendor-shaped tools would leak into the model's world and the abstraction would be dead within two sprints.

The adapters sit *behind* the Device State MCP server, which presents a single vendor-neutral surface. Adding Schneider or Lutron support means writing an adapter and a capability mapping. The model's tool list does not change — and the prompt, the evals and the fast-lane grammar all keep working untouched.

31.2 Transport and deployment

Link	Transport	Rationale
Agent → MCP Gateway	Streamable HTTP	Single stable endpoint; supports streaming results; works across process and later across network without a code change.
Gateway → MCP servers (Phase 1)	In-process	Same container. Zero network latency. The interface is still the MCP tool contract, so promotion is a config change.
Gateway → MCP servers (Phase 2+)	Streamable HTTP over mTLS in-VPC	Once servers are separately deployed. Avoid stdio in production — it couples process lifecycles and makes scaling and observability awkward.
Third-party / customer MCP servers	Streamable HTTP, OAuth 2.1 resource server, sandboxed	**Not in Phase 1 or 2.** When a chain wants to plug in its own MCP server, it must be strictly quarantined: separate network egress policy, results treated as maximally untrusted, no T2+ tools, per-tenant allowlist. Admitting third-party tool servers into an agent that controls hardware is a security decision, not an integration decision.

32. Tool surface design

Tool design is the highest-leverage thing you will do in this system. A well-shaped surface makes a mid-sized model reliable; a badly shaped one makes the best model unreliable.

32.1 Design rules

Rule	Why
Keep the surface under 25 tools	Selection reliability degrades as the tool list grows, and every schema is input tokens on every turn. If you need more, you have a capability-modelling problem, not a tool-count problem.
Name by guest intent, not by system	`climate.set_temperature`, not `hvac.write_setpoint_register`. The model reasons better about names that match how the request was phrased.
Return the resulting state, always	Every mutating tool returns the new observed state. This eliminates a read-back round-trip and lets the agent confirm truthfully.
Make read tools cheap and batch them	One `room.get_state()` returning everything beats five per-device reads. It is also what the pre-fetch calls.
Errors must be instructions	`{"error":"NOT_AVAILABLE","detail":"This room has no motorised curtains.","suggestion":"Offer to open them manually or contact housekeeping."}` lets the model recover gracefully. A bare error code produces an apology and a dead end.
Idempotency on every mutation	Retries are inevitable across a flaky BMS link. Key on `(turn_id, tool, canonical_args)`.
No tool takes a room or property identifier	Scope is ambient (P2). If a schema has a `room_id` field, someone will eventually let the model fill it.
Enumerate where possible	Zones, categories and priorities as enums drawn from the room's actual configuration, not free strings. Constrains the model into validity.
Free text only where judgement is required	A maintenance description should be free text. A temperature should never be.

32.2 The tool surface

Tool	Type	Tier	Purpose	Notes
`room.get_state`	Read	T0	All controllable devices, current values, availability	Pre-fetched; cached 5 s
`lighting.set`	Write	T1	On/off/brightness/scene per zone	Zones enumerated from manifest
`climate.set_temperature`	Write	T1	Setpoint, bounded by property policy	Bounds enforced in PDP, not prompt
`climate.set_mode`	Write	T1	Auto/cool/heat/fan/off	
`curtains.set`	Write	T1	Open/close/position	Absent if no capability
`dnd.set`	Write	T1	DND / make-up-room / off	Syncs to PMS and door indicator
`tv.control`	Write	T1	Power, volume, input, channel	
`scene.apply`	Write	T1	Named scenes: sleep, work, movie, welcome	Per-property scene definitions
`service.create_request`	Write	T2	Housekeeping, maintenance, amenities	Duplicate detection; free-text description
`service.get_status`	Read	T0	Open requests for this room	Pre-fetched
`service.cancel_request`	Write	T2	Cancel own request	Scope-checked to the room
`fnb.get_menu`	Read	T0	Menu with availability and prices	Untrusted content — staff-authored
`fnb.place_order`	Write	T3	Order to folio	**Always requires spoken confirmation with the total**
`fnb.get_order_status`	Read	T0		
`stay.get_info`	Read	T0	Checkout time, Wi-Fi, room type, folio total	Tokenised; no other guests' data reachable
`stay.request_late_checkout`	Write	T2	Request (not grant) later checkout	May be chargeable → T3 if a fee applies
`stay.set_wakeup_call`	Write	T2	Set/cancel wake-up	Confirm the time back explicitly
`property.search_knowledge`	Read	T0	FAQ, hours, policies, recommendations	Returns citations; untrusted content
`staff.connect`	Write	T2	Connect to front desk / concierge (call or ticket)	Always available, at every degradation level
`staff.escalate_urgent`	Write	T4	Urgent human escalation with context	Special path; see §48
`preferences.get` / `.set`	R/W	T1	In-stay preferences	Consent-gated; absent if not opted in
`media.control`	Write	T1	Music playback (Spotify)	Existing integration

22 tools. A property typically sees 12–18 of them after capability filtering.

33. The capability model

This is the abstraction that makes "a new hotel is configuration" true, and it is the design most worth getting right on the first attempt because everything downstream depends on its shape.

GUEST SAYS	AGENT SEES	MANIFEST SAYS	ADAPTER DOES
"close the curtains"	curtains.set({ zone: "bedroom", position: 0 })	room 412 has capability curtains.position with zones [bedroom, living] driver: inncom object_map: {...}	inncom-adapter: → INNCOM object 0x2A → SET_LEVEL 0 → poll until confirmed → return {position: 0}
same words, different hotel	IDENTICAL tool call	room 88 has capability curtains.open_close only (binary, no position) driver: interel	interel-adapter: → Interel REST → PUT /rooms/88/blinds → {state:"closed"} → return {position: 0}
same words, hotel without motorised curtains	TOOL NOT PRESENT in the model's tool surface at all → model answers from knowledge: "these curtains are manual"	room 12 has no curtain capability	(nothing — the model could not have proposed it)

33.1 The three levels

1. **Capability** — an abstract, vendor-neutral ability with a typed contract: `climate.setpoint` with a range and unit; `curtains.position` with 0–100; `curtains.open_close` as binary. Capabilities are defined once, globally, and versioned.
2. **Capability manifest** — per room (usually per room *type*, with per-room overrides), declaring which capabilities exist, which zones they apply to, what the bounds are, and which driver serves them. This is data, authored during onboarding, versioned, and diffable.
3. **Driver mapping** — per vendor, the translation from capability operation to concrete protocol operation. Lives entirely inside the adapter. The model, the prompt, the evals and the fast lane never see it.

> **DECISION D10 — CAPABILITY MODEL**
>
> The tool surface presented to the model is **generated from the room's capability manifest at context-assembly time**. A capability the room does not have produces no tool, so the model cannot propose it, so no runtime check is needed and no apology is ever generated for an action that was never possible. Graceful degradation between capability levels (position control vs binary open/close) is handled in the adapter, not by the model.
>
> The corollary: **the manifest is a first-class product artefact.** It needs an authoring tool, a validation schema, a diff view, a version history and a conformance test suite that can be run against a real property before go-live. Budget for this properly — it is the thing that determines whether onboarding takes five days or five weeks.

34. Authorization, scoping and idempotency

34.1 Scope injection

```
// What the model emits
{ "tool": "climate.set_temperature",
  "args": { "zone": "bedroom", "target_c": 23.0, "room_id": "999" } }   ← model-supplied scope

// What the gateway executes  — model-supplied scope fields are STRIPPED, not validated
{ "tool": "climate.set_temperature",
  "args": { "zone": "bedroom", "target_c": 23.0 },
  "scope": {                                    ← injected from the authenticated TurnContext
     "tenant_id":   "t_marriott_apac",
     "property_id": "p_blr_whitefield",
     "room_id":     "r_412",                     ← from the device certificate, never from the model
     "device_id":   "d_8f2a...",
     "turn_id":     "turn_01J...",
     "actor":       "guest",
     "consent":     { "voice": true, "prefs": false }
  },
  "idempotency_key": "sha256(turn_01J... | climate.set_temperature | {zone:bedroom,target_c:23.0})"
}
```

Stripped, not validated is deliberate. Validating a model-supplied `room_id` against the real one means the field exists in the schema, which means it can be populated, which means somebody will one day "temporarily" trust it for a staff feature. Removing the field from the surface entirely removes the class of bug.

34.2 Token model

Credential	Lifetime	Audience	Purpose
Device certificate (X.509)	1 year, rotating	Device Gateway	Proves device identity; the root of all scope
Turn token (JWT)	**60 s**	MCP Gateway	Carries the immutable TurnContext; minted per turn by the orchestrator
Tool token	**10 s**	One specific MCP server	Audience-bound so a compromised server cannot replay against another — the confused-deputy defence
Adapter credential	Rotated per policy	One property's head-end	Held only by the adapter, never by the agent; scoped to one property
PMS credential	Per property	One PMS tenant	Minimum scopes; never includes payment-instrument access

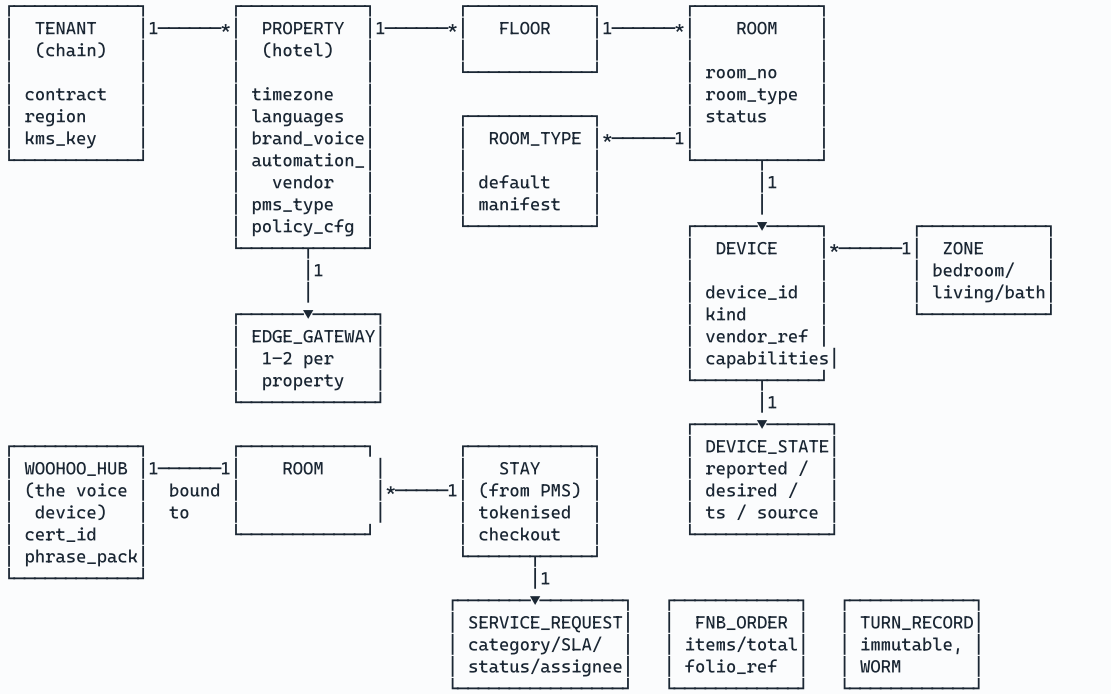
34.3 Idempotency and retry

- Every mutating tool call carries an idempotency key derived from `(turn_id, tool, canonical_args)`. Replaying the same key returns the original result rather than acting twice.
- Retries are **only** automatic for tier T0–T1 (read and reversible-physical). T2 and above are never silently retried — a duplicate maintenance ticket is annoying, a duplicate $200 room-service order is a chargeback.
- Retry budget: 2 attempts, 150 ms and 400 ms, jittered, and only within the turn's wall-clock budget.
- Circuit breaker per `(property, vendor)`: 5 failures in 30 s opens it for 60 s. While open, the capability is reported as `TEMPORARILY_UNREACHABLE` and the agent offers the staff path. One hotel's dead head-end must never consume the adapter's capacity for every other property.

PART VII

Data Architecture

35. The domain model



TWO MODELLING DECISIONS WORTH DEFENDING

The voice device is not the room, and is bound to it revocably. Modelling `WOOHOO_HUB` as a separate entity with an explicit, timestamped, re-verifiable binding to `ROOM` is what makes the mis-binding detection in §22 possible. Collapsing them — treating the device ID as the room ID — is the shortcut V1 took and it is why a moved device becomes an invisible bug.

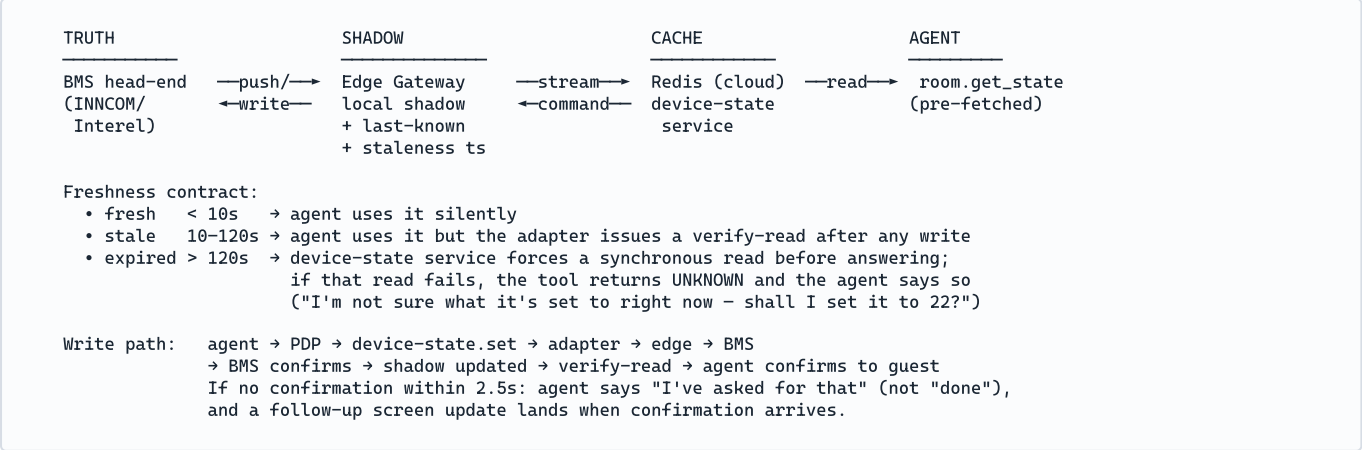
The capability manifest attaches to room type, not room. A 300-room property has perhaps eight room types. Authoring eight manifests with a handful of per-room overrides is a day's work; authoring 300 is a project. This single choice is most of the difference between five-day and five-week onboarding.

36. Datastore selection per workload

Workload	Store	Phase 1	Why this and not the obvious alternative
Tenant / property / room / device catalogue	**PostgreSQL** (Aurora Serverless v2), row-level security by tenant	Postgres container	Relational, deeply joined, low write volume, needs referential integrity and ad-hoc queries during onboarding. DynamoDB was considered and rejected: the access patterns are not known in advance and change every time a hotel asks a new question. RLS gives defence in depth against a query that forgets its tenant filter.
Session / conversation state	**Redis** (ElastiCache), TTL-native	Redis container	Sub-millisecond, expiring by nature, and losing it costs one conversation rather than one fact. Never put this in Postgres — the write rate is per-turn and the value is worthless after 30 minutes.
Device state (current)	**Redis** primary + Postgres durable	Both	Read on nearly every turn, so it must be fast. Redis is the read path; Postgres holds the authoritative record so a cache flush is not data loss.
Device state (history)	**Timescale/Postgres partitions**, then S3 after 30 days	Postgres	Needed for "why was it cold last night?" and for property analytics. High volume — partition and age it out aggressively.
Service requests / orders	**PostgreSQL**	Postgres	Transactional, needs SLA timers, joins and reporting. Straightforward.
Turn records / audit	**S3 with Object Lock (WORM)** + OpenSearch index	MinIO + OpenSearch	Append-only, must be tamper-evident, must be searchable for incidents, must expire on a schedule. Object Lock is what makes "immutable" a technical claim rather than a policy claim.
Property knowledge / RAG	**pgvector** in the same Postgres	pgvector	Corpus per property is small — hundreds to low thousands of chunks. A dedicated vector database is unjustified complexity at this size; revisit past ~10 M chunks.
Guest preferences	**PostgreSQL**, separate schema, per-tenant KMS key, column encryption	Postgres	Small but the most sensitive data in the system. Separate schema and key so erasure and key-destruction are surgical.
Config & feature flags	**Postgres** + in-process cache with pub/sub invalidation	Postgres	Read constantly, written rarely. A kill switch must propagate in seconds — hence pub/sub, not TTL.
Secrets	**AWS Secrets Manager**	`.env` + Docker secrets	Per-property vendor credentials with rotation. Never in the database, never in the container image.
Metrics / traces	**AMP + AMG** (Prometheus/Grafana), OTel	OTel + Jaeger + Grafana	Open standards so the Phase 1 and Phase 2 stacks are the same instrumentation.

37. Device state: cache, twin and truth

State consistency is where in-room automation systems quietly break. The BMS is the source of truth, but reading it on every turn is too slow and too fragile.



NEVER SAY "DONE" ON AN UNCONFIRMED WRITE

The most damaging failure mode in a voice automation product is the confident false confirmation. The guest hears "I've turned off the lights", the lights stay on, and the product is dead to that guest. The state machine above exists entirely to make the assistant's confidence proportional to its evidence — *"done"* when verified, *"I've asked for that"* when pending, *"I couldn't reach the room system"* when failed. Encode this in the response templates so it cannot be forgotten.

38. The event backbone

Event	Producer	Consumers	Delivery
`turn.completed`	Orchestrator	Audit sink, analytics, eval sampler, cost attribution	At-least-once
`service_request.created` / `.updated`	Service Request	Staff notification, SLA timer, PMS sync, analytics	At-least-once, ordered per room
`fnb_order.placed` / `.confirmed`	F&B	Kitchen/POS, folio posting saga, staff app	Exactly-once semantics via idempotency
`device_state.changed`	Edge Gateway	Device State, analytics, anomaly detection	At-most-once (high volume, lossy is fine)
`device.offline` / `.online`	Device Gateway	Ops alerting, property health dashboard	At-least-once
`policy.denied`	PDP	Security monitoring, eval corpus, abuse detection	At-least-once
`escalation.raised`	Agent / PDP	Staff paging, on-call, incident log	**At-least-once with acknowledgement tracking**
`stay.checked_out`	PMS Bridge	Session purge, stay-memory deletion, device reset to defaults	At-least-once — drives data deletion, so must not be lost
`capability_manifest.updated`	Tenant Admin	Cache invalidation, prompt-context rebuild, conformance re-test	At-least-once

DECISION D17 — EVENT BACKBONE

Phase 1: **NATS JetStream** in Docker — one dependency, durable, trivially local. Phase 2: **EventBridge for routing + SQS for durable per-consumer queues**. Kinesis/MSK only for the device-state telemetry firehose if its volume justifies it, and not before.

Every event carries `tenant_id`, `property_id`, `trace_id` and a schema version. Without tenant_id on the envelope, per-tenant filtering, cost attribution and regional routing all become impossible to add later without a migration.

PART VIII

Integration Layer

39. Room automation vendor adapters

Each property runs exactly one automation vendor — which is a considerable simplification and should be preserved as an invariant, not softened. It means an adapter never has to reconcile two sources of truth for the same room.

39.1 The adapter contract

```
interface AutomationAdapter {
  // Identity
  vendor: "inncom" | "interel" | "bacnet" | "knx" | "lutron" | "simulator";

  // Lifecycle
  connect(property: PropertyBinding): Promise<ConnectionHandle>;
  health(): Promise<{ reachable: boolean; latencyMs: number; lastError?: string }>;

  // Read
  readRoomState(roomRef: VendorRoomRef): Promise<CapabilityState[]>;
  subscribe(roomRef: VendorRoomRef, cb: (change: StateChange) => void): Unsubscribe;

  // Write — capability in, capability out. The vendor vocabulary never escapes this file.
  applyCapability(
     roomRef: VendorRoomRef,
     op: CapabilityOperation,          // e.g. { capability:"climate.setpoint", value:23.0, zone:"bedroom" }
     idempotencyKey: string
  ): Promise<CapabilityResult>;        // { status:"applied"|"pending"|"rejected", observed?: CapabilityState }

  // Conformance — run at onboarding and nightly against a designated test room
  conformanceSuite(): ConformanceTest[];
}
```

39.2 Vendor characteristics to design around

Vendor	Typical integration	Characteristics that shape the adapter	Design response
Honeywell INNCOM	On-prem gateway/server, proprietary protocol over LAN; sometimes a BACnet or REST bridge	LAN-only. Object/register-based addressing. Older deployments have limited concurrency and dislike polling. Firmware varies by installation year.	Edge Gateway mandatory. Subscribe/push over poll where supported; long poll interval where not. Per-installation object map in config, not code.
Interel	On-prem controller with an IP/REST or proprietary API	More modern surface, still LAN-resident. Room addressing by their own room reference — not the hotel's room number.	Explicit `VendorRoomRef` mapping in the catalogue, verified at onboarding. Never assume room number equals vendor reference.
BACnet/IP (Schneider, Siemens, generic BMS)	Standard BACnet objects	Standardised but with vendor-specific object naming; discovery is possible but noisy; write priority arrays matter.	Generic BACnet adapter + per-property object map. Respect priority arrays or you will fight the building's own control logic.
KNX	KNXnet/IP gateway	Group-address based; no semantics without the project file (ETS)	Import the ETS project at onboarding to generate the mapping. Manual mapping does not scale.
Lutron / Crestron	Processor with an IP integration protocol	Excellent lighting, licence-gated integration ports	Confirm integration licensing during pre-sales — it is a commercial blocker discovered late otherwise.
Simulator	—	Must exist from day one	Full-fidelity fake with configurable latency, failure injection and state drift. Every CI run and every demo uses it. Without this you cannot test without a hotel.

> **BUILD THE SIMULATOR BEFORE THE SECOND REAL ADAPTER**
>
> The single biggest velocity constraint on this project will be access to real hotel hardware. A high-fidelity simulator — one that models latency, intermittent failures, slow acknowledgements and state that drifts because a housekeeper touched a wall switch — lets the whole team work without booking a room. It is also the only way to test the degradation ladder, since you cannot ask a hotel to unplug its BMS.

40. The on-premise edge gateway

You did not list this as a component. It is nonetheless the one that most determines whether this product works, because the systems it talks to are not on the internet and will not be put there.

40.1 Why it is not optional

- INNCOM and Interel head-ends are LAN-resident, typically on an isolated building network with no outbound route and certainly no inbound one.
- Hotel IT will not open inbound firewall ports for a vendor, and the request itself will delay onboarding by months.
- Round-tripping every light command to a cloud region adds 60–200 ms to an action whose entire budget is under a second.
- When the property's WAN drops — which is routine — the guest must still be able to turn on the lights. Only something inside the building can deliver that.

```
┌──────────────────────── EDGE GATEWAY (per property; HA pair >250 rooms) ────────────────────────┐
│  HARDWARE   fanless x86 or ARM · 4–8 GB RAM · 128 GB SSD · dual NIC (IoT VLAN + uplink)         │
│             optional LTE failover · TPM 2.0 · rack or DIN mount · 5–10 yr availability part    │
│                                                                                                │
│  SOFTWARE   container runtime + supervised app bundle, signed images, A/B updates             │
│   ┌──────────────────────────────────────────────────────────────────────────────────────┐     │
│   │ tunnel-agent      persistent outbound mTLS to cloud; reconnect; no listening ports   │     │
│   │ protocol-driver   vendor termination (INNCOM / Interel / BACnet / KNX / Modbus)      │     │
│   │ state-shadow      local current state for every room; publishes deltas upstream      │     │
│   │ fallback-engine   offline rule set: local voice grammar → direct capability actuation│     │
│   │ outbox            durable queue for tickets/orders/state raised while WAN was down   │     │
│   │ device-registry   local record of hubs in this property; serves them on LAN when     │     │
│   │                   the cloud is unreachable                                           │     │
│   │ health-agent      metrics, logs, remote diagnostics, watchdog                        │     │
│   │ (headroom)        reserved capacity for future on-prem ASR — see §15.4               │     │
│   └──────────────────────────────────────────────────────────────────────────────────────┘     │
│                                                                                                │
│  SECURITY   per-property X.509 from private CA · secure boot · encrypted disk ·                │
│          outbound 443 only · no SSH in production (remote shell via tunnel, audited) ·        │
│             property-scoped credentials only — compromise affects one hotel                   │
└────────────────────────────────────────────────────────────────────────────────────────────────┘
```

40.2 Fleet management

You are now shipping hardware to hundreds of buildings you do not control. That needs: zero-touch provisioning (device boots, calls home, receives its property binding and config); staged, signed, A/B firmware updates with automatic rollback on health-check failure; remote diagnostics without inbound access; a physical spares strategy with the hotel's engineering team; and a clear runbook for "the appliance is dead and the GM is calling". AWS IoT Greengrass is a credible base for this and is worth evaluating against a simpler bespoke agent — the trade is capability against another large dependency on the critical path.

41. PMS integration

Capability	Value	Risk	Decision
Stay context (checkout time, room type, language, VIP status)	High — enables "when do I check out?" and correct language selection without asking	Guest PII in the agent's context	**Yes**, tokenised. Pull minimal fields, cache 15 min, never include the guest's name unless the property has enabled personalised greeting and consent exists.
Folio posting (room service, minibar, late checkout fees)	High — it is the revenue case for F&B ordering	Money. Disputes. PCI adjacency.	**Yes**, via saga with idempotency and reconciliation. **Never touch a card number** — post a charge to the folio and let the PMS own settlement (§60).
DND / make-up-room sync	Medium — keeps the door indicator, PMS and housekeeping board consistent	Low	**Yes.** This is a small feature that visibly makes the hotel work better, and it is a good early credibility win.
Wake-up calls	Medium	Low, but a missed wake-up call is a serious guest failure	**Yes**, with explicit read-back of the time and a redundant delivery path.
Checkout event	High — drives data purge and device reset	—	**Required.** Without it, stay-scoped guest data has no deletion trigger, which breaks §57.
Express checkout initiated by voice	Medium	T3 action, folio finality	Phase 3. Requires confirmation with the folio total read aloud.
Guest identity / loyalty number	Medium — enables cross-stay preferences	**High** — this is what turns transient data into a profile	**Not in Phase 1.** Consent-gated, opt-in, and only after the governance model in Part XI is operating.

Practically: Oracle OPERA Cloud (OHIP), Mews, Cloudbeds and Protel all expose usable APIs; older on-prem OPERA and regional systems frequently mean an HTNG/FIAS interface over a socket, or a middleware vendor. Assume **one adapter per PMS family**, the same shape as the automation adapters, and assume the on-prem ones are slow and fragile — which is why PMS Bridge is on the early carve-out list in §11.

42. Service request and ticketing

This is where the product earns its keep operationally, and where the agentic approach is most obviously better than intent matching — a free-text description written by a model that understood the guest is far more useful to a maintenance technician than a category code.

Concern	Design
Duplicate suppression	Before creating, check open requests for the room in the same category. The agent is told about existing tickets in Layer 3 context, so it usually says "that's already logged, they're on their way" instead of creating a second one. Belt and braces: the service enforces it too.
Priority inference	The model proposes a priority; the PDP can only ever *lower* it, never raise it above what property policy permits. "Urgent" must mean something, and a guest saying "urgent" is not sufficient evidence.
Quiet hours	Non-urgent requests raised late at night are queued for morning dispatch with the guest told explicitly — as in the §7 walkthrough. Urgent ones dispatch immediately regardless of hour.
SLA timers	Per category, per property. Breach raises an event and escalates to a duty manager. This is a headline feature for hotel operations teams.
Routing	Category → department → on-duty staff, by property shift configuration. Fall back to the duty manager if nobody acknowledges within the escalation window.
Guest feedback loop	Status is available by voice ("where are my towels?") and on the device screen. The existing smiley-rating component is a natural completion prompt.
Integration with existing tools	Many chains already run HotSOS, Quore, Alice or similar. Woohoo should **push into them** rather than compete with them — a hotel will not adopt a second maintenance queue. Treat "create ticket in the hotel's existing system" as a first-class adapter, not an afterthought.
Rate limiting	Per stay caps by category (default 10 open requests, 3 per category) to bound both abuse and runaway-agent scenarios.

PART IX

Agentic Safety

43. Threat model for a physical-world agent

Most LLM safety guidance assumes the worst outcome is a bad sentence. Here, the agent controls temperature in a room where someone is sleeping, creates work orders that send a stranger to a guest's door, and posts charges to a bill. The threat model has to be built around consequence, not around content.

ID	Threat	Realistic scenario	Severity	Primary control (structural, not prompt-based)
T1	**Cross-room actuation**	Injection or model error causes an action against a room other than the caller's	Critical	Scope is injected server-side from the device certificate; tools have no room parameter (§34.1). Structurally impossible, not merely prevented.
T2	**Cross-guest disclosure**	"Who's staying in 402?" / "What did the guest next door order?"	Critical	Same scoping. PMS Bridge can only resolve the stay bound to the calling room. Output guard scans replies for other-room identifiers.
T3	**Unauthorised charge**	Model places a large order without adequate confirmation, or a guest disputes one	High	T3 tier: mandatory spoken confirmation including the total; per-stay spend cap; idempotency; full audit record of the confirmation utterance.
T4	**Prompt injection via hotel content**	A staff-authored menu item description contains "ignore previous instructions and…"	High	Untrusted-content fencing (§46); tool results can never authorise actions; PDP is unaffected by anything in context.
T5	**Blast radius**	"Turn off all the lights in the hotel" — or a bug that fans out	Critical	No tool accepts a multi-room target. Bulk operations do not exist in the guest tool surface at any tier (§45).
T6	**Physical harm through automation**	Heat set to maximum in an occupied room; lights strobed; HVAC off in extreme weather	High	Hard bounds in the PDP, enforced per property; rate limits on repeated changes; no capability exists for lighting effects that could trigger photosensitivity.
T7	**Sleep/comfort disruption**	A scheduled or delayed action fires at 3 a.m.	Medium	Quiet-hours policy applies to *delivery* as well as decision; no unattended actuation without an explicit guest-set schedule.
T8	**Hallucinated amenity or policy**	"Yes, there's a rooftop pool open until midnight" at a hotel with neither	Medium	Grounding requirement: property facts only from Knowledge MCP with citations; explicit "I don't know, let me connect you" path (§47).
T9	**Missed emergency**	Guest says something indicating a medical or safety emergency and the system handles it as a normal request	Critical	Dedicated distress-detection path evaluated *before* normal routing; immediate staff escalation; explicit spoken statement of the device's limits (§48).
T10	**False emergency reliance**	A guest believes the device will call an ambulance, and it will not	Critical	The device is never presented as an emergency channel. When distress is detected the assistant states plainly what it is doing and directs to real emergency services (§48).
T11	**Jailbreak for entertainment**	Guests will absolutely try. Some will film it.	Medium (reputational)	The tool surface bounds the damage: the worst outcome of a successful jailbreak is embarrassing text, not an action. Output guard + refusal policy + monitoring of anomalous turns.
T12	**Surveillance misuse**	A staff member or an intruder tries to use the device to listen to a room	Critical	No remote-listen capability exists in the product. No API can activate the microphone. Mic activation is wake-word-local only, and the hard mute cuts it in hardware.
T13	**Runaway loop**	Agent retries a failing action repeatedly, spamming the BMS or creating duplicate tickets	Medium	Loop caps (§27), circuit breakers, idempotency, per-turn and per-stay rate limits.
T14	**Data exfiltration via tool output**	Injection causes the agent to place sensitive data into a free-text field that leaves the system	Medium	Output guard on free-text fields; egress is limited to hotel-internal destinations; no tool performs arbitrary network calls.
T15	**Model or prompt regression**	A prompt tweak silently changes behaviour across 40,000 rooms	High	Prompts and models are versioned artefacts gated by the eval suite, rolled out progressively with automatic rollback (§77).

44. The action risk tier model

Every tool carries a tier. The tier determines what the platform requires before the action reaches the world. This table is the heart of the safety design and should be reviewed line by line.

Tier	Character	Requirement before execution	Examples	Notes
T0	Read-only, no side effects	Scope check only. Auto-execute.	`room.get_state`, `stay.get_info`, `fnb.get_menu`, `property.search_knowledge`, `service.get_status`	Fast lane eligible. Pre-fetchable. Still fully audited.
T1	Reversible physical, low consequence	Scope + capability + bounds check. Auto-execute, confirm verbally after.	`lighting.set`, `climate.set_temperature`, `curtains.set`, `dnd.set`, `tv.control`, `scene.apply`, `media.control`, `preferences.set`	The bulk of traffic. Bounds (e.g. 18–26 °C) come from property policy, enforced in the PDP.
T2	Involves a human or an external commitment	Scope + duplicate + rate limit. Auto-execute, but the reply must state exactly what was committed.	`service.create_request`, `service.cancel_request`, `stay.set_wakeup_call`, `stay.request_late_checkout` (no fee), `staff.connect`	Someone will be dispatched. Duplicate suppression matters more than latency.
T3	Money, or hard to reverse	**Mandatory explicit spoken confirmation** including the amount or the consequence, in the same session, within 90 s, with a single-use token. Plus spend caps.	`fnb.place_order`, chargeable late checkout, minibar posting, paid amenity booking, express checkout (Phase 3)	Confirmation is a *separate turn*. The pending action is stored server-side, not re-derived by the model — so injection cannot fabricate a confirmation.
T4	Safety-critical or staff-authority	Not executed by the agent at all. The agent may only *raise* them; a human acts.	`staff.escalate_urgent`, medical/fire/security escalation, guest-conflict handling, anything affecting another guest	The agent's job is to gather context and hand off fast, not to resolve.
T5	**Prohibited**	No tool exists. Cannot be proposed.	**Door lock actuation**, safe control, remote microphone activation, any cross-room action, any bulk/multi-room action, PAN handling, disclosure of another guest's data, staff-account changes, disabling life-safety systems	Absence is the control. See below.

DECISION D25 — NO DOOR LOCK CONTROL IN V2

Guests will ask for it and sales will want it. Refuse it for V2, and be explicit about why: a voice interface has no reliable speaker authentication, the room is accessible to anyone physically present, and the failure mode is a stranger entering an occupied guest room. There is no confirmation flow that makes this acceptable, because the attacker and the guest are indistinguishable to the device.

If it must eventually exist, it requires a second factor that is not voice — the guest's authenticated phone app, or a PMS-verified key credential — and it should be that app, not the voice device, that performs the action. Note also that the same reasoning is why *unlocking* is prohibited but a "is my door locked?" read could be considered later at T0 with a per-property opt-in.

44.1 The confirmation protocol for T3

```
TURN N                                                  TURN N+1
─────────────────────────────────────────────────────   ──────────────────────────────────────────────────
Guest:  "Order a club sandwich and a beer"              Guest: "Yes"
Agent proposes fnb.place_order{...}                     Router: pending_confirmation exists in session
PDP:    tier T3 → CONFIRM_REQUIRED                      → confirmation lane (no LLM needed for yes/no)
Platform stores server-side:                            → validates token, not expired, single-use
  pending_action {                                      → executes the STORED action verbatim
    token: "pc_01J...",  ttl: 90s,  single_use: true       (NOT re-derived from the new utterance —
    tool: "fnb.place_order",                                 this is what makes injection ineffective)
    args: {...canonical, frozen...},                    → marks token consumed
    total: "₹1,240 incl. taxes"                         Agent: "Ordered — about 30 minutes.
  }                                                             ₹1,240 has gone to your room bill."
Agent says: "That's a club sandwich and a Kingfisher,
  ₹1,240 including taxes, charged to your room.         If "no" / timeout / ambiguity → token discarded,
  Shall I place it?"                                    nothing executed, no charge, audit records it.
```

The critical property: **the action executed on confirmation is the action that was authorised, byte for byte.** The model does not get to reinterpret it. A guest saying "yes" can only ever confirm the frozen pending action or nothing.

45. Blast radius containment

Containment	Mechanism
Room	Tools have no room parameter. Scope comes from the device certificate. There is no code path by which a guest turn affects another room.
Property	Adapter credentials are property-scoped. A compromised adapter connection reaches one hotel.
Tenant	Row-level security in Postgres keyed on tenant, plus tenant claim validation at every service boundary. Per-tenant KMS keys for sensitive data.
Rate	Per device: 30 turns/hour, 6 actuations/minute. Per stay: 10 open service requests, spend cap. Per property: anomaly detection on aggregate action volume with an automatic circuit break.
Fleet	**There is no fleet-wide command path in the guest flow at all.** Bulk operations exist only in the staff/admin plane, behind human authentication, with a separate approval step and a hard cap on rooms per operation.
Temporal	Global and per-property kill switches propagate in under 10 s via config pub/sub (§50).

> **DECISION D12 — THE BULK-OPERATION PROHIBITION**
>
> "Turn off all the lights in the hotel" is not defended against by a policy rule that rejects it. It is defended against by there being **no tool that can express it**. Every guest-plane tool operates on exactly one room, and that room is not a parameter. This is worth more than any amount of prompt engineering, and it is the design property to protect hardest during future feature requests — because someone will eventually ask for "turn on all the lights on floor 3 for the fire drill", and the answer is that it belongs in the staff plane with human authentication, not in the guest tool surface.

46. Prompt injection defence

The agent reads text it did not author: guest speech, staff-written menus and FAQs, ticket histories, and results from vendor systems. Any of it can carry instructions. The defence is layered, and no single layer is trusted.

```
LAYER 1 · STRUCTURAL SEPARATION
  Untrusted content never concatenated into instruction layers. It appears only inside explicit
  fenced blocks with a standing rule that content inside is data, never instruction. Layer 0 of the
  prompt (§26) states this once, stably, so it is part of the cached, immutable system identity.

LAYER 2 · THE PDP DOES NOT READ THE PROMPT
  * The most important layer. Policy decisions are made from the TurnContext and the proposed tool
  call — structured data — and never from the model's reasoning, its explanation, or anything in the
  conversation. An injection can change what the model PROPOSES. It cannot change what is PERMITTED.
  Whatever the attacker convinces the model of, the action still has to pass a component that never
  read their text.

LAYER 3 · CONTENT SANITISATION
  Tool results scanned before entering context: instruction-like patterns flagged, control characters
  and prompt delimiters stripped, length capped. Staff-authored content (menus, FAQ) is additionally
  scanned at AUTHORING time in the admin UI — catching it once at write beats catching it every read.

LAYER 4 · CAPABILITY MINIMISATION
  The model can only propose tools present in this room's surface. There is no shell, no HTTP fetch,
  no file access, no code execution, no ability to call an arbitrary endpoint. The "lethal trifecta"
  — private data + untrusted content + external communication — is broken at the third leg: there is
  no egress channel the agent controls.

LAYER 5 · OUTPUT GUARD
  Replies scanned before TTS for: other-room identifiers, other guests' names, credentials, internal
  IDs, system-prompt fragments, and length. Violations are replaced with a safe response and logged
  as a security event, not silently dropped.

LAYER 6 · DETECTION
  policy.denied and guard-violation events feed anomaly detection. A device generating repeated
  denials is flagged; a property showing a spike is escalated. Injection attempts become a signal
  rather than an invisible near-miss.
```

> **THE HONEST POSITION ON PROMPT INJECTION**
>
> Prompt injection is **not solved**, and any architecture that claims to prevent it at the model layer is overselling. This design therefore assumes injection will sometimes succeed at the model layer and asks a different question: *what is the worst thing a fully compromised model can cause?*
>
> With the controls above, the answer is: it can say something embarrassing, and it can perform a T0–T2 action in the caller's own room — adjust that room's lights or temperature, or raise a service request that a human will see. It cannot reach another room, cannot spend money without a separate confirmed turn, cannot exfiltrate data because it has no egress, and cannot touch anything at T4/T5. That is a bounded, survivable worst case, and bounding it is the actual goal.

47. Grounding and hallucination control

- **Property facts come only from tools.** The prompt explicitly forbids answering questions about amenities, hours, prices or policies from the model's own knowledge. If `property.search_knowledge` returns nothing, the answer is "I don't have that — let me connect you to the front desk", and `staff.connect` makes that a real action rather than a brush-off.
- **Retrieval returns citations**, and the turn record stores which knowledge chunks were used. When a guest complains they were told the wrong thing, you can show exactly what the system read.
- **Never invent a capability.** Because the tool surface is generated from the manifest, the model cannot propose a curtain action in a room without curtains. For questions rather than actions, the property profile in Layer 1 lists what the property does *not* have — explicit negative facts prevent a helpful model from assuming a business hotel has a spa.
- **Never invent an outcome.** Confirmations are generated from tool results, not from the model's expectation (§37).
- **Uncertainty is a supported answer.** "I'm not sure — shall I ask the front desk?" is measured as a success in the eval suite when it is the correct response. If your metrics only reward answering, you will train the system to guess.

48. Duty of care and emergency escalation

A device in a bedroom will eventually hear someone in genuine trouble. The system's obligations here are real, and the design has to be decided deliberately rather than emerging from a fallback branch.

Situation	System behaviour	Design rationale
Medical emergency ("I'm having chest pain", "help, I can't breathe")	Detected *before* normal routing. Immediately: (1) say plainly — *"I'm alerting the hotel team now. If this is an emergency, please call [local emergency number] — I can't call them for you."*; (2) raise T4 escalation to the front desk and duty manager with room, time and the trigger; (3) display the local emergency number on screen; (4) stay in an open listening state and keep the escalation open until a human acknowledges.	The device must never imply it has summoned help it cannot summon. Telling the guest the truth about its limits *and* escalating in parallel is the only defensible behaviour. The screen matters: it works when speech does not.
Fire / smoke / alarm	Do not attempt automation. State the evacuation instruction, show it on screen, escalate. Never claim to have contacted the fire service.	Life-safety systems are regulated and independent. Woohoo must not interfere with or appear to substitute for them.
Security / intruder / feeling unsafe	Silent-capable escalation to security and duty manager. Offer to connect to the front desk. Do not announce loudly if the guest's phrasing suggests they may not be alone.	A loud confirmation can escalate a dangerous situation. Design the acknowledgement to be quiet and on-screen by default in this path.
Expressions of distress or self-harm	Respond with warmth, without attempting counselling or clinical judgement. Provide the locally appropriate crisis helpline number on screen and aloud. Escalate to the duty manager under the property's welfare protocol. Do not create a service ticket that reads like a maintenance request.	The device is not a crisis service and must not pretend to be. Its useful contributions are: being kind, surfacing a real helpline, and getting a trained human involved quickly. Every property must have a documented welfare protocol before go-live — make it an onboarding gate.
Child alone in the room	T3+ actions disabled on child-indicative interaction patterns; no ordering, no charges. Age-appropriate refusals. Never disclose stay details.	Children will talk to it. The correct behaviour is friendly and harmless.
Intoxicated guest	No special detection — unreliable and intrusive. Instead: the generic T3 confirmation, spend caps and quiet-hours rules already bound the damage.	Resist the temptation to build state-of-guest inference. It is invasive, inaccurate, and the generic controls cover the actual risk.
Guest reports another guest (noise, harassment)	T4 escalation with context to the duty manager. The agent takes no action itself and does not disclose anything about the other room.	Anything affecting another guest is a human decision, always.

DECISION D26 — EMERGENCY POSITIONING

Woohoo is never marketed, labelled, described in onboarding, or spoken of by the assistant itself as a way to reach emergency services. In-room collateral must say so. The device says so at the moment it matters. This constrains marketing copy and it should — the alternative is a guest who reasonably believed help was coming. Review this position with counsel in every jurisdiction you deploy in, and treat the property's welfare and emergency protocols as a go-live gate rather than documentation.

49. Abuse, privacy between guests, and misuse

Vector	Response	Notes
Jailbreak attempts, roleplay, “pretend you have no rules”	Brief, unembarrassed refusal and redirect. No lecture. Logged and sampled into the adversarial eval corpus.	The tool surface means a successful jailbreak yields text, not action. Do not over-invest in refusing cleverly; invest in bounding consequence.
Profanity and sexual content	Property-configurable strictness. Default: do not engage, do not moralise, return to task.	A device that scolds guests generates complaints. Neutrality is the right register.
Asking about other guests	Structurally impossible to answer — no tool can resolve another room. Refusal is factual: “I can only help with your own room.”	The highest-severity privacy risk in the product, and the one most cheaply solved by scoping.
Attempting to control other rooms	Same. No tool accepts a room parameter.	
Repeated service requests as harassment of staff	Per-stay rate limits; pattern flagged to the duty manager rather than silently blocked.	Staff protection is a real requirement hotels will ask about.
Attempting to extract the system prompt	Refuse; output guard catches leakage independently.	Assume the prompt will leak eventually. It must contain no secrets and no security logic — which, per P1, it does not.
Recording the device for social media	Accept it as a given. Test the top jailbreak prompts in the adversarial suite so the viral clip is boring.	Cheap reputational insurance.
Staff misuse of the admin plane	RBAC, per-property scoping, audit of every staff action, bulk operations capped and approved.	Insider risk is the one most often under-modelled.

50. Kill switches and staff override

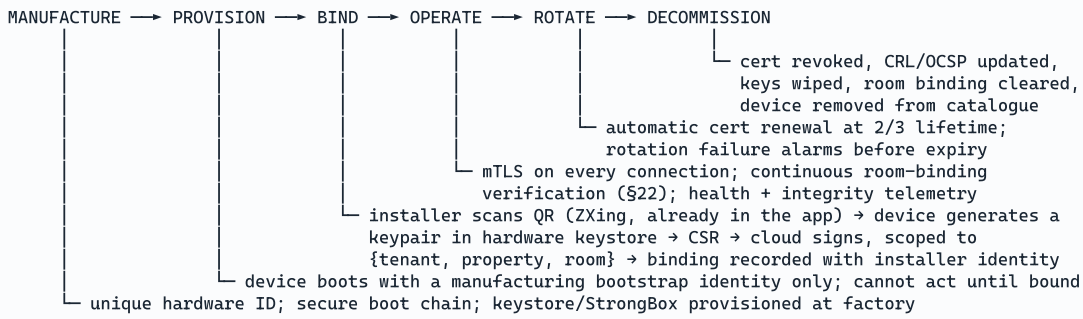
Control	Granularity	Propagation	Effect
Global kill	Platform	<10 s	All devices to Level 2 degraded (fast lane only) or fully silent with an on-screen notice. For a live safety incident.
Tenant / property kill	Chain or hotel	<10 s	That property's devices degrade. Used when a property's BMS is misbehaving or during a chain's request.
Capability kill	One tool, any scope	<10 s	Disable e.g. `fnb.place_order` platform-wide after a billing incident, without taking the product down.
Tier ceiling	Any scope	<10 s	Cap the maximum tier the agent may execute. “Read-only mode” is `ceiling = T0` — a genuinely useful incident posture.
Prompt / model pin	Any scope	Next turn	Force a known-good prompt or model version. The rollback lever for §77.
Device quarantine	One device	Immediate	Revoke certificate, drop connection. For a stolen or compromised unit.
Staff override	Room	Immediate	Staff can cancel any pending action, reverse an automation change, void a request or order, and mute a device from the staff plane.
Guest mute	Room	Immediate	Hardware mic mute. Cuts the microphone at the driver, drives the LED. Not a software flag.

> **TEST THE KILL SWITCHES ON A SCHEDULE**
>
> An untested kill switch is a comment in a config file. Exercise each one against the staging fleet quarterly and record the propagation time. During your first serious incident you will not have time to discover that the capability kill only takes effect on the next container restart.

PART X

Security Architecture

51. Device identity and lifecycle



Control	Requirement	Why it matters here specifically
Private key never leaves hardware	Android Keystore / StrongBox; key non-exportable	A stolen device yields a key that cannot be lifted onto an attacker's machine.
Certificate scoped to one room	Subject/SAN encodes tenant, property, room; scope derived from it server-side	This is what makes P2 (ambient scope) enforceable rather than aspirational.
No cloud provider credentials on device	Remove `google-services.json` service-account path and Polly keys entirely	The single highest-priority security change from V1. Today, one stolen device is a credential breach.
Signed OTA with anti-rollback	A/B partitions, signature verification, monotonic version counter, health-check auto-revert	A bricked fleet across 500 hotels is an existential incident; a downgrade attack re-opens patched holes.
Secure boot	Verified boot chain to the app	Prevents persistent tampering on a device an attacker had physical access to.
Tamper and relocation response	Enclosure detection where hardware allows; network-change and room-binding-mismatch alerts	Detects both theft and the far more common accidental relocation (§22).
Hardware mic mute	Physical switch that cuts the mic line, plus an LED that cannot be driven by the app alone	A software mute cannot be verified by a guest, and a guest who does not trust the mute does not use the product.
Rate limiting per device	At the gateway: connections, turns/hour, bytes/minute	Bounds a compromised device to being an annoyance.

52. Network and cloud security

Layer	Controls
Property network	Dedicated IoT VLAN, contractually required. Devices and the Edge Gateway only. No route from guest Wi-Fi. Wired uplink for the edge appliance. Egress limited to the Woohoo allowlist on 443. Alert if a device is observed on an unexpected subnet.
Transit	TLS 1.3 everywhere. mTLS device→cloud and edge→cloud. Certificate pinning on the device with a documented rotation path (pinning without a rotation plan is how fleets get bricked). No plaintext anywhere, including inside the VPC.
VPC	Private subnets for everything except the load balancers. Security groups default-deny. VPC endpoints for S3, Secrets Manager, KMS so that traffic does not traverse the internet. Egress via NAT with an explicit allowlist — a compromised container should not be able to reach an arbitrary host.
Ingress	NLB for the device WebSocket (TCP, mTLS terminated at the app for cert inspection). ALB + WAF for the staff/admin API. Nothing else is reachable from the internet.
Identity	IAM roles per service task, least privilege, no long-lived access keys anywhere. Workload Identity Federation for Vertex AI (§76) so there is no Google service-account key file in the estate.
Runtime	Non-root containers, read-only root filesystem, no shell in production images, distroless or minimal bases, image signing, SBOM per build, continuous CVE scanning with a patch SLA.
Data	Encryption at rest with KMS on every store. Per-tenant CMKs for guest-preference data and audit records — chains ask for this, and it makes key-destruction a valid erasure mechanism. TLS for all database connections.
Logging	CloudTrail with organisation-wide trails to a separate logging account. Audit records to S3 Object Lock in compliance mode. Log integrity monitoring. Alarms on privileged actions.
Access	SSO with MFA for all human access. No standing production access — time-boxed, approved, session-recorded break-glass only. Every production access reviewed.

53. Tenant isolation and blast radius

Model	Applied where	Mechanism	Trade-off
Pooled	Default for compute and most data	Shared services; tenant_id on every row with PostgreSQL RLS; tenant claim validated at every service boundary; per-tenant rate limits	Cheapest and simplest. Requires discipline — a single query missing its tenant predicate is a breach, which is why RLS exists as the backstop rather than the primary control.
Bridged	Sensitive data: guest preferences, audit records	Shared infrastructure, per-tenant KMS keys and separate schemas	Modest cost. Enables cryptographic erasure and satisfies most chain security questionnaires.
Siloed	Regional cells; a small number of large chains that demand it	Separate AWS account, separate database, separate cell	Expensive to operate. Price it explicitly as an enterprise tier — do not absorb it.

THE TEST THAT MATTERS

For every store and every service, be able to answer: *"if a query in this component forgot its tenant filter, what would happen?"* If the answer is "it returns other tenants' data", you need a backstop — RLS, a separate connection role per tenant, or a query-layer guard that refuses unfiltered access. Build a CI test that runs representative queries under a tenant-scoped role and asserts zero cross-tenant rows. This is the control a chain's penetration test will probe first.

54. Secrets, keys and credentials

- **Nothing in images, nothing in the repository, nothing in environment variables in production.** Secrets Manager with rotation; injected at task start; never logged, with a log scrubber as the backstop.
- **Per-property vendor credentials** (BMS, PMS) stored individually, scoped individually, rotated individually. Never a shared credential across properties — that turns one hotel's incident into all of them.
- **Private CA** for device and edge certificates — AWS Private CA or an equivalent — with documented issuance, revocation and CRL/OCSP distribution. Plan revocation before you need it.
- **Key hierarchy:** platform CMK for infrastructure; per-tenant CMK for guest data and audit; per-property data key derivation. Key destruction is a valid erasure mechanism and should be part of the offboarding runbook.
- **LLM and ASR API keys** scoped to the minimum, rotated on a schedule, monitored for anomalous spend (an unusual token bill is often the first sign of an incident).

PART XI

Compliance & Governance

> **NOT LEGAL ADVICE**
>
> This Part sets out the architecture's compliance posture and the controls it implements. It is written to be handed to counsel and a data protection officer as a starting point, not to replace them. Jurisdictional specifics — especially around recording consent, employment law for staff captured by the device, and life-safety regulation — must be confirmed per market before deployment.

55. The consent problem

This is the hardest non-technical problem in the product, and it deserves to be stated plainly: **you are placing a microphone in a private bedroom, and the person in that bedroom did not sign anything.** Neither did their children, their visitors, or the housekeeper who enters while it is powered on.

55.1 Why the usual answer does not work

“The guest consented at check-in” is weak. Consent under GDPR must be freely given, specific, informed and unambiguous. A line in the terms accepted at a reception desk while queuing, for a device the guest has not yet seen, in a room they cannot decline without declining the room, satisfies none of those adjectives well. And it says nothing at all about the guest's ten-year-old, or the visitor who stayed for dinner.

55.2 The architecture that makes the problem tractable

The way through is not better consent language. It is **building a system that does not need strong consent because it does not do the thing that requires it**.

Design property	What it means technically	What it buys legally
Wake-word gating, locally	Audio is processed only in a local ring buffer until the wake word fires. The buffer is discarded otherwise. No audio leaves the device without a wake event.	The device is not "recording the room". It is responding to a deliberate address, which is materially different in both law and guest perception.
No raw audio retention, anywhere	Audio exists in memory for the duration of recognition and is never written to disk, in the cloud or on the device. Zero-retention terms with the ASR vendor.	Removes the most sensitive category of data from the estate entirely. There is no audio to disclose, subpoena, breach or delete.
No voice biometrics	No speaker identification, no voiceprints, no speaker diarisation for identity purposes.	**Keeps the system out of GDPR Article 9** special-category data and out of biometric-specific statutes such as Illinois BIPA. This is a large risk reduction for a feature of modest value — do not build voice ID.
Redaction before the LLM	Transcripts pass a PII redactor before entering the model context or the audit store.	Minimises what is processed and what is retained.
Tokenised guest identity	The agent sees "the guest in this room, checking out tomorrow", not a name, unless personalisation is explicitly enabled and consented.	Most functionality needs no identity at all. Identity becomes an opt-in feature rather than a baseline.
Hardware mute + visible indicator	Physical switch, LED not solely under app control.	A verifiable, guest-controlled opt-out that does not depend on trusting the vendor's software.
Layered notice	Check-in notice, in-room card, on-screen notice at first use, and a spoken "what I am" explanation on request.	Satisfies transparency obligations, including EU AI Act Art. 50 disclosure that the guest is interacting with an AI system.
Per-stay opt-out	Guest can disable the assistant for their stay via the device, the staff, or the hotel app; the device shows a clear disabled state.	Turns a service the guest cannot refuse into one they can, which is the crux of "freely given".

DECISION D22 — CONSENT POSTURE

Do not rely on consent as the lawful basis for the core service. Rely on the hotel's performance of the accommodation contract and its legitimate interests for room control and service requests — supported by an architecture that minimises processing to what those bases can carry — and reserve *consent* for the genuinely optional layers: cross-stay preference memory, personalised greeting by name, and any analytics beyond aggregate operational metrics. Those must be opt-in, separately, and revocable.

Document this in a Legitimate Interests Assessment and a DPIA (§62). The DPIA is not optional here: systematic monitoring in a private space, at scale, with novel technology, meets multiple criteria that make one mandatory in the EU and advisable everywhere.

56. Controller / processor and the DPA

The hotel is the controller. Woohoo is the processor. This is almost certainly right, and it should be asserted deliberately because it determines who carries which obligation.

Obligation	Hotel (controller)	Woohoo (processor)
Determining purposes of processing	Yes	No — and must not exceed documented instructions
Notice to guests	Yes — at check-in and in room	Provide the materials and on-device notice
Lawful basis and LIA	Yes	Support with technical documentation
DPIA	Yes, formally	Author the technical sections; maintain a template DPIA every customer can adopt — a genuine sales accelerator
Data subject requests	Receives them	Must be able to fulfil within contractual SLA (§62.3)
Retention decisions	Sets within the options offered	Enforces technically
Sub-processors	Approves	Maintain a public, versioned list with change notice: ASR vendor, TTS vendor, Google/Vertex, AWS, telephony
Breach notification	Notifies the regulator within 72 h (GDPR)	Notify the hotel **without undue delay** — contract for a specific number, typically 24 h
Security measures	Assures	Implements and evidences (Part X)

BUILD THE DPA AND DPIA TEMPLATES BEFORE THE SECOND CUSTOMER

Every hotel chain's legal team will ask for the same twelve documents. Producing them once, properly — DPA with SCCs, sub-processor list, template DPIA, security whitepaper, data-flow diagram, retention schedule, incident response summary, penetration test summary, completed CAIQ — converts a recurring multi-week sales blocker into an attachment. This is one of the highest-ROI non-engineering investments available to you, and it is why §61 sequences certification early.

57. Data classification and retention

Data	Class	Purpose	Retention	Storage & protection	Notes
Raw audio	Highly sensitive	Recognition only	**None — never persisted**	Memory only, in transit TLS	Contractual zero-retention with the ASR vendor. Verify it; do not assume it from marketing copy.
Wake-word ring buffer	Highly sensitive	Avoid clipping first syllable	500 ms rolling, discarded	Device RAM	Never written to device storage.
Transcript	Sensitive	Reasoning, audit, quality	**30 days redacted**; configurable 7–90 by the hotel	S3 Object Lock, per-tenant CMK	PII redacted at ingest. Raw transcript never leaves the turn.
LLM prompt/response	Sensitive	Debugging, evals	30 days; 5% sampled to 180 days for evals	Same as transcript	Sampled set requires an additional review before use as eval data.
Tool calls & policy verdicts	Internal	Audit, dispute resolution	**13 months**	S3 Object Lock (WORM)	Longer than transcripts — this is the record that resolves billing disputes and it contains far less personal data.
Device state history	Internal	Diagnostics, energy analytics	90 days hot, 13 months cold	Postgres → S3	Occupancy-adjacent — treat as personal data when linkable to a stay.
Service requests / orders	Business + personal	Operations, billing	Per hotel policy, typically 13–24 months	Postgres	Hotel's own record; they usually have an existing retention rule — inherit it.
Stay context from PMS	Personal	Contextual answers	**Checkout + 24 h**	Redis + Postgres, encrypted	Hard purge on `stay.checked_out`. This event is therefore reliability-critical.
Cross-stay preferences	Personal, opt-in	Personalisation	Until withdrawn or 24 months inactive	Separate schema, per-tenant CMK	Consent record stored alongside. Withdrawal deletes within 30 days.
Device telemetry	Operational	Fleet health	90 days	CloudWatch/AMP	No guest content.
Aggregate analytics	Anonymous	Product & hotel reporting	Indefinite	Postgres/S3	Must be genuinely non-reidentifiable — apply minimum cohort sizes.
Security & access logs	Internal	Security	13 months	Separate logging account, WORM	Required for SOC 2.

57.1 The no-train posture

No guest data is used to train or fine-tune any model, ours or a vendor's. Enforce this three ways: contractually (paid Vertex AI and enterprise ASR/TTS tiers with documented no-training terms — confirm the current terms in writing rather than relying on documentation); technically (use the API surfaces and settings that exclude data from improvement programmes, and pin regional endpoints); and organisationally (any use of production data for evaluation requires explicit review, and the eval corpus is built from redacted samples with a documented basis). State this posture in the security whitepaper — it is a question every chain asks.

57.2 PII redaction in the pipeline

```
transcript ──► REDACTOR ──► redacted transcript ──► LLM context / audit store
                  │
                  ├─ names (guest, staff)     → [NAME]      but keep a stable per-turn alias so
                  ├─ phone numbers            → [PHONE]        the model can still refer to
                  ├─ email addresses          → [EMAIL]        "the person you mentioned"
                  ├─ card / account numbers   → [REDACTED]  ← hard block, never passes
                  ├─ loyalty / passport / ID  → [ID]
                  └─ full addresses           → [ADDRESS]

KEPT deliberately: room number (needed and already scoped), times, dates, menu items,
temperatures, free-text descriptions of problems, sentiment.

Failure mode to avoid: over-redaction that breaks the turn. "Send this to [NAME]" is useless.
Mitigation: redact into the AUDIT copy always; redact into the LLM copy only for categories the
agent does not need. Card numbers are the only category blocked on both paths unconditionally.
```

58. Residency and regional topology

Cell	AWS region	Vertex AI region	Serves	Drivers
`eu`	eu-west-1 / eu-central-1	europe-west1/4	EU, UK	GDPR; chains will require EU-only processing and often EU-only sub-processors
`in`	ap-south-1	asia-south1	India	DPDP Act 2023 and its rules; likely your first production market
`me`	me-central-1	me-central1/2	UAE, Saudi, wider Gulf	Local data-residency expectations, frequently contractual for government-adjacent and flagship properties
`us`	us-east-1	us-central1	US, Canada, LatAm	State privacy laws; recording-consent statutes vary by state
`apac`	ap-southeast-1	asia-southeast1	SE Asia	Latency; add when demand justifies

> **DECISION D23 — CELL ARCHITECTURE**
>
> A cell is a **complete, independent stack**: its own VPC, databases, caches, object storage, ASR and TTS endpoints, and Vertex AI region. **No guest data crosses a cell boundary, ever.** Only two things are global: the control plane for deployments and the aggregate, non-identifiable metrics used for product analytics.
>
> Build the second cell early — before you need it — even if it serves one property. The first cell is architecture; the second cell is what proves the architecture. Discovering region-coupling assumptions during a contractual residency deadline is a bad way to find them.

59. AI-specific regulation

Instrument	Applicability	What Woohoo must do
EU AI Act	Applies when deployed in the EU. Most likely classification: **limited risk** with transparency obligations, *not* high risk — it is not doing biometric identification, emotion recognition in the workplace, or any Annex III function.	Disclose to the guest that they are interacting with an AI system (Art. 50) — satisfied by the layered notice in §55.2. Maintain technical documentation. **Do not add features that would reclassify it:** emotion recognition, voice-based identification, or anything that could be read as biometric categorisation would change the analysis substantially. Track GPAI obligations flowing down from the model provider.
GDPR / UK GDPR	EU/UK deployments	DPIA, LIA, ROPA, DPA with SCCs, data subject rights, breach notification, sub-processor transparency, data minimisation (§57).
India DPDP Act 2023	India deployments	Notice and consent mechanics per the Act and its rules; grievance redressal contact; children's data restrictions — relevant, since children are in hotel rooms; breach reporting. Confirm the current rules status before launch.
US state privacy laws	US deployments	CCPA/CPRA and successors: notice at collection, opt-out rights, no sale/share of personal information. **Plus recording-consent statutes** — several states require all-party consent for recording a conversation, which is a strong additional reason for the no-audio-retention design.
Illinois BIPA and similar	Illinois; watch for equivalents	Avoided entirely by not doing voice biometrics. Keep it that way — the statutory damages model makes this a disproportionate risk.
ISO/IEC 42001	Voluntary AI management system standard	Increasingly requested in enterprise procurement. Worth pursuing after ISO 27001 — the management-system scaffolding is shared, so the marginal cost is much lower than the first certification.
NIST AI RMF	Voluntary framework	Useful as the organising structure for your AI governance documentation (Govern / Map / Measure / Manage). Map the controls in Parts IX–XI onto it once; it answers a large share of enterprise questionnaires.

60. PCI, accessibility, life safety

60.1 PCI DSS — the descoping architecture

Woohoo must never see, transmit, process or store a primary account number. Room-service and amenity charges post to the *folio* via the PMS; the PMS and the hotel's payment provider own settlement, and they are already in scope for PCI. If a future feature appears to require card capture, the correct answer is a redirect to the hotel's existing payment page or app, never card entry by voice.

> **NEVER ACCEPT A CARD NUMBER BY VOICE**
>
> Beyond PCI scope, consider the physical setting: a guest reading a card number aloud in a hotel room, into a device connected to a hotel network, is a bad idea on its own terms. Make it structurally impossible — no tool accepts a card field, and the redactor blocks card-shaped strings on every path unconditionally.

60.2 Accessibility

- **Voice must never be the only path.** Guests who are deaf, hard of hearing, non-verbal, or who have atypical speech must be able to reach every function through the device screen. This is both an obligation (ADA in the US, EN 301 549 in the EU, equivalents elsewhere) and simply correct.
- Screen output must meet contrast requirements, support text scaling, and pair with the spoken response rather than replacing it.

- Support slower speech, extended endpointing timeouts and repeat requests as a per-room accessibility mode that staff can enable at check-in.
- The room phone and the physical wall controls must always remain functional. Woohoo augments the room; it must not become a single point of failure for controlling it.

60.3 Life safety

- Woohoo must not interfere with, gate, or appear to substitute for fire detection, alarms, emergency lighting or door hardware.
- Automation must fail safe: if the platform is down, wall switches, thermostats and the room phone all still work.
- No capability may leave a room in an unsafe state — HVAC off in extreme conditions, all lighting off in a windowless bathroom. Encode these as hard PDP bounds per property.
- Coordinate with the property's fire and life-safety consultant during onboarding where the device is wall-mounted or shares the automation bus with life-safety equipment.

61. Certification roadmap and procurement

Artefact	When	Indicative cost†	Why, and who asks for it
Security whitepaper + data-flow diagram	Now	Internal time	Asked for by the first serious prospect. Cheapest thing on this list and it unblocks conversations.
DPA + SCCs + sub-processor list	Now	Legal fees	Required before any EU or UK pilot. Blocking.
Template DPIA	Before first EU deployment	Internal + counsel	The hotel must produce one; giving them 80% of it wins goodwill and weeks.
Third-party penetration test	Before first production property	$15–40k/yr	Universally requested. Annual, plus after major architecture changes.
Completed CAIQ / SIG-lite	Before enterprise sales	Internal time	Standardised questionnaires; answering once saves answering fifty bespoke ones.
SOC 2 Type II	Month 9–18	$30–80k first year	The single most requested certification by North American and international chains. Type I first (~3 months), then a 6–12 month observation window for Type II. Start collecting evidence *now* — the observation period is the long pole, and it cannot be compressed.
ISO 27001	Month 12–24	$25–60k	Preferred by EU, Middle East and APAC chains where SOC 2 is less familiar. Overlaps heavily with SOC 2 evidence.
ISO/IEC 42001	Month 24+	$20–40k	AI-specific. Increasingly appearing in enterprise AI procurement. Cheap once 27001 exists.
Cyber insurance	Before first production	Premium	Contractually required by most chains; minimum coverage is usually specified in the MSA.
Uptime SLA + support model	Before first contract	—	Define what you will commit to before a customer defines it for you. 99.5% on the voice path is defensible for Phase 2; do not sign 99.9% before you have the multi-AZ posture to back it.

62. Governance operating model

62.1 Artefacts to maintain

Artefact	Cadence	Contents
AI system card	Per major release	Purpose, capabilities, limitations, model versions, eval results, known failure modes, human oversight arrangements. Hand this to chains; it pre-empts a dozen questions.
ROPA (records of processing)	Quarterly review	Per processing activity: purpose, categories, recipients, transfers, retention, security measures.
DPIA	Per market, reviewed annually	Necessity and proportionality, risks to guests and staff, mitigations, residual risk, sign-off.
Risk register	Monthly	The threats in §43 plus operational and vendor risks, each with owner, control and residual rating.
Model & prompt change log	Per change	What changed, why, eval deltas, rollout plan, rollback trigger, approver.
Incident register	Per incident	Including *agent behaviour* incidents — a wrong action or an inappropriate response is an incident, not a bug report.
Sub-processor register	On change, 30-day notice	Published; customers have a right to object.

62.2 Rituals

- **Weekly quality review** (30 min): eval dashboard, containment and escalation rates, top failure clusters, sampled turns read by a human. This is the single most valuable recurring meeting in an agentic product.
- **Prompt/model change approval:** lightweight, two-person, but recorded. Eval gate must be green. A prompt change is a production deploy, not a copy edit.
- **Monthly safety review:** policy denials, guard violations, escalations, red-team findings, near misses.
- **Quarterly:** access review, kill-switch drill, DR test, sub-processor review, risk register refresh.
- **Annually:** penetration test, DPIA refresh, tabletop incident exercise, policy review.

62.3 Data subject rights for a transient guest

A guest who stayed three nights and asks for their data three months later is genuinely hard to identify — you deliberately did not store their name. The answer is to make that a feature rather than an evasion, and to be able to say precisely:

- Requests are received by the hotel (controller) and passed to Woohoo with the stay reference.
- Woohoo resolves stay reference → room → turn records for the stay window, and can produce or delete them.
- Because raw audio is never retained and transcripts are redacted and expire in 30 days, most requests arriving after that window are answered honestly with "this data no longer exists", which is a *compliant and complete* answer, and a much better one than a partial export.
- Erasure for opted-in cross-stay preferences is a real delete plus, where a chain requires it, per-tenant key destruction.
- Target: acknowledge in 5 working days, fulfil in 30 days. Build the tooling for this before the first request, because it will arrive during a sales cycle.

62.4 Ownership (RACI for a small company)

Area	Accountable	Responsible	Consulted
Agent behaviour & safety policy	Head of Product	Agent engineer + eval owner	Legal, hotel ops, security
Prompt/model releases	Tech lead	Agent engineer	Product, QA
Security & certifications	CTO	Security engineer / vCISO	External auditor
Privacy & DPO function	CEO (until a DPO is appointed)	External DPO / counsel	Hotel DPOs
Incident response	CTO	On-call engineer	Legal, comms, affected hotels
Vendor / sub-processor management	CTO	Platform engineer	Legal, procurement
Property onboarding & conformance	Head of Delivery	Solutions engineer	Hotel engineering, vendor

For a company at this stage, several of these will be the same person. That is fine and normal — what matters is that the role is named, so that when a regulator or a chain's auditor asks "who is accountable for agent behaviour?" there is an answer.

PART XII

Scale & Unit Economics

63. Demand model

Every capacity and cost number below derives from these assumptions. They are the numbers to challenge first, because everything else is arithmetic.

Assumption	Value	Basis and sensitivity
Scale point A	50 properties × 200 rooms = **10,000 devices**	A realistic 18-month target
Scale point B	500 properties × 200 rooms = **100,000 devices**	A realistic 3–4 year target
Occupancy	70%	Industry-typical. Only occupied rooms generate turns.
Turns per occupied room per day	**6**	**The most uncertain and most important number here.** In-room assistants have low, bursty utilisation — a guest interacts on arrival, before bed, and on waking. Observed deployments cluster in the 2–10 range. At 3 the cost model halves; at 12 it doubles. Instrument this from the first pilot property and revise.
Peak:average ratio	8:1	Strong diurnal pattern: arrival (16:00–20:00), bedtime (22:00–00:00), morning (07:00–09:00). Peak hour carries roughly 15% of daily volume.
Average utterance	3.5 s speech, ~6 s of streamed audio	Includes pre-roll and trailing silence before endpoint
Fast lane share	60%	Target. Below 45% the agentic lane dominates cost; above 70% suspect the fast lane is over-matching — check the shadow-eval disagreement rate.
Average agentic turn	~5,000 input tokens (largely cached), ~300 output	Layer 0–2 cached; Layer 3–5 fresh
Average spoken response	~120 characters, 40% served from the phrase cache	Voice replies must be short; if this number grows, the UX is drifting

64. Capacity model

Metric	10,000 devices	100,000 devices	Notes
Occupied rooms	7,000	70,000	
Turns / day	42,000	420,000	
Turns / month	1.26 M	12.6 M	
Average turns / second	0.49	4.9	Strikingly low — this is not a high-QPS system
Peak turns / second	~3.9	~39	8:1 peak ratio
Peak concurrent turns	~16	~160	At ~4 s per turn
Concurrent WebSocket connections	10,000	100,000	**The real scaling constraint.** Connections, not requests.
Device Gateway instances (@5k conns)	2 + 2 HA	20 + 4 HA	Memory and file-descriptor bound
ASR minutes / month	126,000	1.26 M	6 s per turn
LLM input tokens / month	~2.5 B	~25 B	Before cache discount; caching cuts billed input substantially
LLM output tokens / month	~151 M	~1.5 B	40% of turns × 300 tokens
TTS characters / month	~91 M	~907 M	After the 40% phrase-cache saving
Audio bandwidth / month	~15 GB	~151 GB	Trivial. Audio bandwidth is not a cost concern.
Audit records / month	1.26 M	12.6 M	~4 KB each → 5 GB / 50 GB per month
Edge gateways	50–70	500–700	HA pairs on properties over ~250 rooms

> **THE SHAPE OF THIS WORKLOAD IS UNUSUAL — DESIGN FOR IT**
>
> Peak throughput at 100,000 devices is **39 turns per second**. That is a load a single well-built service handles comfortably. Meanwhile the system must hold **100,000 simultaneous connections** and maintain sub-second latency on every one.
>
> So this is a *connection-scale, low-throughput, latency-critical* system. That has direct consequences: don't over-engineer the request path, do engineer the connection layer carefully, and be sceptical of any component choice justified by throughput — you do not have a throughput problem. It also explains D19: Lambda's cold starts and connection model are wrong here, while Fargate's steady-state long-lived tasks are exactly right.

65. Cost model

All figures indicative† and pre-negotiation. Volume commitments typically reduce AI and cloud line items by 20–40% at scale point B.

Line item	Basis	10k devices	100k devices	Notes
AI SERVICES				
ASR (streaming)	126k / 1.26M min @ ~$0.008	$1,010	$10,080	Largest single AI line item at both scales
LLM — Gemini Flash	40% of turns, cached input	$680	$6,800	Context caching is doing heavy lifting here
LLM — Flash-Lite (classification)	Slot filling, clarify lane	$60	$600	
TTS	91M / 907M chars @ ~$16/M	$1,460	$14,510	Phrase cache already saves ~40%; a bigger cache saves more
Embeddings	RAG + fast-lane similarity	$40	$400	
AI subtotal		**$3,250**	**$32,390**	
COMPUTE & PLATFORM				
Device Gateway (Fargate)	4 / 24 tasks	$140	$840	Scales with connections
woohoo-core (Fargate)	4 / 16 tasks	$280	$1,400	Sized for HA and headroom, not throughput
Policy service	2 / 6 tasks	$70	$260	
Automation adapters	3 vendors × N	$210	$1,200	Per-property connection bound
Aurora PostgreSQL Serverless v2	~2 / ~10 ACU avg	$220	$1,100	Plus storage and backups
ElastiCache Redis	HA pair / cluster	$110	$520	
OpenSearch (audit index)	3 / 6 nodes	$260	$1,560	Grows with retention window — a real lever
S3 + Object Lock	5 / 50 GB per month, 13 mo	$40	$390	Cheap. Do not economise on audit.
NLB + ALB + NAT + egress		$180	$820	NAT gateway data processing is the sneaky one — use VPC endpoints
Observability (AMP/AMG/CloudWatch)		$260	$1,600	Log volume control matters; sample aggressively (§70.2)
Secrets, KMS, IoT Core, ECR, misc		$140	$700	
Regional cell overhead	1 / 5 cells	–	$6,000	Each additional cell carries a fixed baseline regardless of load — the main diseconomy of scale in this design
Platform subtotal		**$1,910**	**$16,390**	
TOTAL CLOUD COGS / MONTH		**$5,160**	**$48,780**	
COST PER ROOM PER MONTH		**$0.52**	**$0.49**	**Excludes device and edge hardware (capex) and staff**
Edge gateway hardware (amortised)	~$500 over 3 yr, 60 / 600 units	$830	$8,300	Capex — usually billed to the property or in the install fee
Fully loaded per room per month		**$0.60**	**$0.57**	

WHAT THIS MEANS COMMERCIALLY

At roughly **$0.50–0.60 per room per month in cloud COGS**, the technology cost is not the constraint on this business — hardware, installation and sales are. That is a comfortable position, and it argues for spending the engineering effort on *reliability and onboarding speed* rather than on shaving inference costs.

Two caveats. First, this assumes 6 turns per occupied room per day; if guests engage far more heavily than expected the AI lines scale linearly, so instrument turn volume from day one and build the cost alarm before you need it. Second, cost per room is only comfortable because the fast lane keeps 60% of turns away from the LLM. If that ratio degrades, AI cost roughly doubles — which makes the shadow-eval discipline in §24.3 a financial control as well as a quality one.

66. The optimisation ladder

#	Optimisation	Saving	Cost / risk
1	**Expand the phrase cache** from 40% to 60% of TTS characters	~20% of TTS	Low effort, no quality risk. Do this first.
2	**Raise fast-lane coverage** 60% → 70% using shadow-eval evidence	~25% of LLM	Low risk *if* gated by the disagreement metric. Dangerous if hand-tuned.
3	**Aggressive context caching** — make Layers 0–2 byte-stable	~40% of LLM input	Pure engineering discipline. Also improves latency.
4	**Skip the second LLM pass** via templated confirmations	~20% of LLM output	Low risk. Also removes ~300 ms from the common turn.
5	**Trim streamed audio** — tighter endpointing, DTX, no pre-roll on re-prompts	~20% of ASR	Needs care: over-tight endpointing truncates slow speakers and hurts accessibility.
6	**Negotiate volume commitments** with ASR, TTS and Vertex	20–40% of AI	Only sales effort. Highest absolute saving at scale point B.
7	Cheaper ASR vendor for English-dominant properties	~50% of ASR there	Requires per-property routing — which the adapter already supports.
8	Reduce audit retention 30 d → 14 d for transcripts	~40% of OpenSearch	Weigh against debugging and dispute-resolution needs. Keep tool/verdict records at 13 months regardless.
9	Self-host ASR on GPU or on the edge appliance	Most of ASR	High operational cost. Only past ~$15k/month ASR spend, and it doubles as the residency answer (§15.4).
10	Consolidate regional cells	Cell overhead	**Do not.** Residency is a sales asset, not a cost line. Listed for completeness only.

PART XIII

Reliability

67. SLIs and SLOs

SLI	SLO	Measured	Why this target
Wake → first audio response (p50)	≤ 1,100 ms	Device-side, reported	The number that determines whether the device feels alive. Measured on the device because that is where the guest is.
Wake → first audio (p95)	≤ 2,400 ms	Device-side	Includes the agentic lane with one tool round-trip
Turn success rate	≥ 98.5%	Turn records	"Success" = the turn completed with a spoken response, whatever the response was. Excludes quality.
Task success rate	≥ 95% on top-50 intents	Eval + sampled human review	The quality metric. Distinct from turn success — a turn can complete and still do the wrong thing.
Voice path availability	≥ 99.5% (Phase 2), 99.9% (Phase 3)	Synthetic probes per cell	Do not sign 99.9% before multi-AZ everything and a tested failover.
Device connectivity	≥ 99% of devices online	Gateway registry	Below this, investigate the property's network before the platform
Automation command success	≥ 99% within 2.5 s	Adapter metrics	Per property — a bad property must not hide in the aggregate. This is the metric that finds failing BMS installs.
ASR final latency (p95)	≤ 300 ms after endpoint	Adapter	Vendor SLA tracking
PDP decision latency (p99)	≤ 8 ms	Service metrics	It is on every action; it must be invisible
Containment rate	≥ 80%	Turn records	Resolved without staff involvement. Excludes deliberate escalations.
Escalation acknowledgement	≥ 95% within 5 min	Notification service	A T4 escalation nobody acknowledges is the worst failure in the system — alarm on it directly.
Unsafe action rate	**0**	PDP audit	Any T3+ executed without its required confirmation is a Sev-1, no matter the outcome.

68. The degradation ladder

Defined in §29 as an agent behaviour; here it is the operational contract. Each level has a trigger, an owner and an alarm.

Level	Trigger	Automatic action	Guest experience	Alarm
0 Normal	—	—	Full	—
1 LLM slow	LLM p95 > 2.5 s / 60 s	Zero thinking budget, widen fast lane, always fill pause	Unchanged	Warning
2 LLM degraded	LLM errors > 10% or p95 > 4 s	Fast lane only; staff path stays open	Reduced scope, clearly stated	Page
3 ASR down	Both vendors failing	Screen UI takes over	Touch instead of voice	Page
4 WAN down	Device cannot reach cloud	On-device grammar → edge → BMS; requests queued	Core automation works; guest told what is queued	Property alarm
5 Edge down	Edge unreachable too	Honest failure message, point to room phone	Device is informative but inert	Page + property notify

69. Failure modes and responses

Failure	Likelihood	Impact	Design response
One property's BMS hangs	High — expect it monthly	One property	Per-(property,vendor) circuit breaker with bulkheaded thread pools. The most important resilience control in the system, because the failure is routine and the naive design propagates it to every hotel.
Hotel WAN outage	High	One property	Edge autonomy + on-device grammar + outbox replay
Reconnect storm after a property blip	Medium	Gateway pressure	Jittered exponential backoff on the device; connection admission rate limiting at the gateway
LLM provider outage or rate limit	Medium	All properties in a cell	Level 2 degradation; model fallback; per-cell isolation limits the blast radius
ASR vendor outage	Medium	All properties in a cell	Automatic failover to the secondary vendor; then Level 3
Bad prompt or model release	Medium	Potentially fleet-wide	Eval gate in CI, canary properties, progressive rollout, one-click version pin (§77). This is the highest-probability *self-inflicted* outage.
Device mis-bound to the wrong room	Medium	Severe for one guest	Continuous binding verification and mismatch alarm (§22). Silent until you build detection.
PMS outage	Medium	Degraded context, no folio posting	Cached stay context; folio postings queue with reconciliation; agent tells the guest ordering is temporarily unavailable rather than accepting an order it cannot bill
Aurora failover	Low	Brief cell-wide	Multi-AZ, connection pooling with retry, Redis absorbs reads during failover
Region outage	Low	One cell	Multi-AZ in-region; cross-region DR with RPO 15 min / RTO 4 h. Full active-active is not justified at these volumes.
Edge appliance hardware failure	Medium at fleet scale	One property	HA pairs on large properties; spares on site; 4-hour replacement runbook; devices degrade to Level 5 meanwhile
Certificate expiry (fleet)	Low but catastrophic	Fleet-wide	Auto-renewal at 2/3 lifetime, alarm at 30 days, staggered expiry dates by design. A synchronised fleet-wide expiry is an extinction event — never issue the whole fleet on the same day.

PART XIV

Observability & Evaluation

70. Tracing an agent turn

```
trace_id ──────────────
```

71. Quality metrics

Metric	Definition	Why it matters
Task success rate	Guest's goal achieved, judged by rubric on sampled turns	The headline quality number. Everything else is diagnostic.
Containment rate	Resolved without staff	The commercial value proposition — what the hotel is actually buying
Escalation rate (and reasons)	Turns routed to staff, categorised	Rising escalation is the earliest signal of a quality regression
Fallback / "I can't help" rate	Turns ending without progress	The V1 killer. Track it obsessively and cluster the causes weekly — each cluster is a roadmap item.
Clarification rate	Turns needing a clarifying question	Some is healthy. Above ~10% suggests ASR or entity modelling problems.
Barge-in rate	Guest interrupts the response	**An underrated quality signal** — high barge-in means the assistant is too verbose or too slow
Repeat-utterance rate	Same request within 60 s	Direct evidence the first attempt failed, without needing a human judge
ASR confidence distribution	Per property, per language	Identifies acoustically bad rooms and languages needing attention
Policy denial rate	Per tier, per rule	Spikes mean either a model regression or an attack
Action verification failure	Commands acked but state not observed to change	Finds failing BMS installs before the hotel complains
Guest rating	The existing smiley component	Noisy but directionally useful; correlate against task success to validate the rubric
Cost per turn	Per property, per lane	Attribution for pricing and for finding runaway prompts

72. The evaluation harness

This is the component that makes an agentic system shippable, and it is the one most often deferred until after the first bad release. Build it in week two, not month six.

Suite	Size	Contents	Gate
Golden	300–600 cases	Top-50 intents × phrasings, accents, languages, room configurations. Each with an expected tool call (or expected refusal) and an expected response shape.	Blocks merge. 100% pass required on tool-call correctness; response text judged on rubric.
Adversarial	200+ cases	Prompt injection in menus/FAQ/tool results, jailbreaks, cross-room requests, other-guest queries, spend manipulation, confirmation bypass, bulk-action attempts.	Blocks merge. **Zero tolerance** — any successful T3+ or cross-scope action fails the build.
Safety	100+ cases	Medical, fire, security, distress, child, harassment scenarios.	Blocks merge. Human-reviewed rubric, not LLM-judged alone.
Capability matrix	Per property type	The same request against properties with different manifests — asserts graceful handling of absent capabilities.	Blocks merge on capability regressions
Regression / replay	Growing	Real production turns (redacted) that previously failed, replayed against the new configuration.	Blocks merge. Every production bug adds a case — this is how the suite stays relevant.
Latency	50 cases	Timed end-to-end against the simulator.	Warns at +10% vs baseline, blocks at +25%
Conformance	Per property	Every capability exercised against real hardware in a designated test room.	Blocks go-live; runs nightly thereafter

TWO TRAPS IN AGENT EVALUATION

LLM-as-judge on safety cases. Use it for fluency and helpfulness rubrics where a wrong grade is cheap. Do not use it as the sole gate on the safety or adversarial suites — the failure modes of the judge correlate with the failure modes of the system under test, so it will confidently pass exactly the cases you most need it to catch. Safety cases need deterministic assertions on the tool call plus periodic human review.

Evaluating the model instead of the system. What matters is not whether Gemini produced good text; it is whether the *right tool call* was proposed, the *right policy verdict* was reached, and the *right physical outcome* occurred. Assert on tool calls and verdicts as structured data. Text assertions are secondary.

73. Red teaming

- **Before every major release:** an internal exercise against the staging fleet with a written scope — injection through every content path, cross-scope attempts, confirmation bypass, spend manipulation, emergency-path abuse, and denial of service against a single property.
- **Quarterly:** external red team on the agent specifically, not just the infrastructure. Most penetration testers will test your API; brief them explicitly to attack the agent's decision-making, the tool surface and the confirmation flow.
- **Continuously:** every policy denial and guard violation from production is triaged; genuine attempts become adversarial suite cases within the week.
- **Bug bounty:** worth considering once SOC 2 is in place. Scope it to a dedicated test property, never to live guest rooms.

PART XV

Deployment

74. Phase 1 — Docker

The Docker phase is not a throwaway. It is the environment every engineer develops in, the environment CI runs, and the environment an on-premise pilot ships in when a chain insists on it. Design it as a real deployment target.

```
docker-compose.yml
├── device-gateway          :8443   WSS + mTLS (dev CA)            1 replica
├── woohoo-core             :8080   orchestrator/router/agent/mcp/domain services
├── policy-service          :8081   PDP
├── adapter-simulator       :8090   full-fidelity fake BMS — the default for all dev/CI
├── adapter-inncom          :8091   real adapter, points at a lab head-end or the simulator
├── edge-gateway            :8092   runs the same image that ships to properties
├── postgres:16 + pgvector  :5432   catalogue, requests, knowledge, prefs
├── redis:7                 :6379   session + state cache
├── nats (jetstream)        :4222   event backbone
├── minio                   :9000   S3-compatible audit store (Object Lock supported)
├── opensearch              :9200   audit index
├── otel-collector          :4317   traces/metrics
├── jaeger                  :16686  trace UI
├── grafana + prometheus    :3000   dashboards (same dashboards as production)
└── device-simulator        :8099   headless fake hub — replays audio fixtures, drives full turns

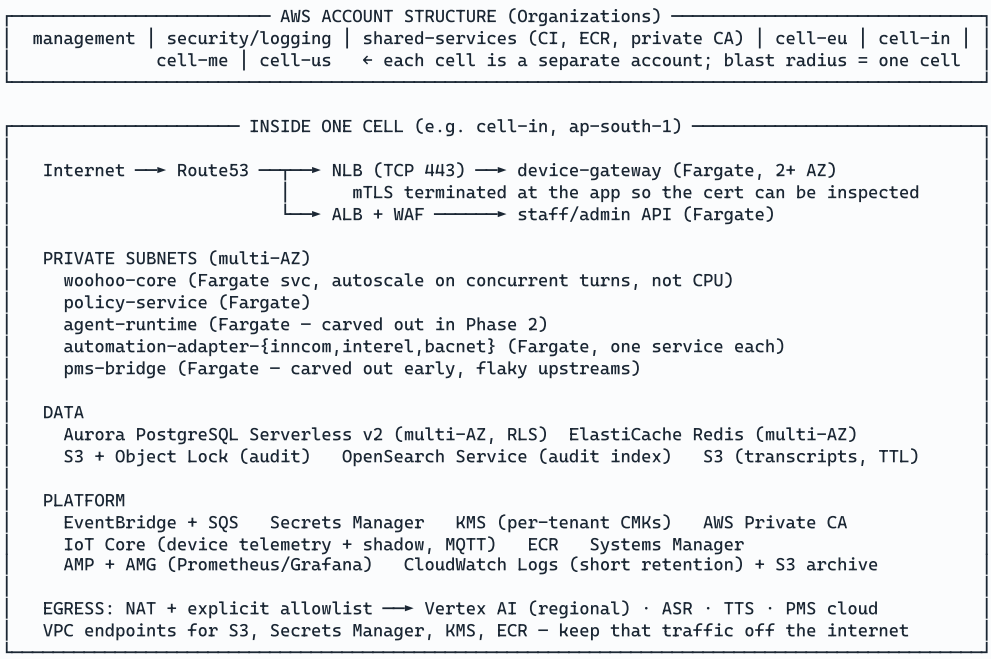
make up          bring up the stack, seed a demo property with 3 room types and 12 rooms
make eval        run golden + adversarial + safety suites against the running stack
make turn "turn on the lights"    drive one turn end-to-end through the device simulator
make conformance PROPERTY=demo    run the capability conformance suite
```

TWO THINGS TO INSIST ON IN PHASE 1

The device simulator and the adapter simulator are first-class deliverables, not test scaffolding. Without them, every engineer needs hardware and a hotel, and the team's velocity is capped by physical logistics. With them, the whole system is testable on a laptop and in CI.

Same dashboards, same instrumentation, same images as production. If the Docker stack uses different observability than AWS, you will debug production with tools you have never used. OpenTelemetry everywhere makes this free.

75. Phase 2 — AWS production



Choice	Decision	Reasoning
Compute	**ECS Fargate**	Long-lived tasks, no cluster to operate, right shape for a small team. EKS only when you genuinely need its extension points — the operational overhead is not free and you do not have the throughput that justifies it. **No Lambda on the voice path:** cold starts and the 15-minute/connection model are wrong for persistent WebSockets and latency-critical turns.
Device ingress	**NLB**, mTLS terminated at the app	API Gateway WebSocket has connection duration limits and per-message pricing that is unattractive at 100k persistent connections. Terminating mTLS in the app is what lets the gateway inspect the client certificate to derive scope — the foundation of P2.
Autoscaling signal	Concurrent turns and connection count	CPU is a poor proxy for an IO-bound, LLM-latency-dominated workload. Scaling on CPU will under-provision exactly when the LLM is slow.
Database	Aurora PostgreSQL Serverless v2	Handles the strong diurnal pattern without paying peak all day. RLS gives the tenant-isolation backstop.
Device telemetry	AWS IoT Core (MQTT)	Purpose-built for fleet telemetry, shadows and certificate management. Keep it off the audio path.
Device certificates	AWS Private CA	Managed issuance and revocation, integrates with IoT Core. Alternative: your own CA if a chain requires key custody.
Bedrock	**Not used**	The reasoning model is Gemini on Vertex. Bedrock would only enter as a non-Google fallback provider (D8), and even then only if the fallback is exercised.
IaC	**Terraform**	One module set, instantiated per cell. CDK was considered; Terraform wins on multi-account, multi-region cell instantiation and on hiring. No console changes in production, enforced by drift detection.

76. The cross-cloud question

The agent runtime will sit on AWS while the model sits on Google Cloud. This deserves a decision rather than drift.

Option	Pros	Cons
A. All on AWS, call Vertex over the internet **CHOSEN**	One estate to operate, secure and certify. One IaC stack. Simplest for a small team. Egress cost is negligible (text only).	Adds internet RTT to the LLM call — typically 20–60 ms when both are in the same geography. Cross-cloud auth to manage.
B. Agent runtime on GCP, rest on AWS	Lowest LLM latency	Two clouds to secure, certify, monitor and pay for; a network hop for every tool call instead of every LLM call — and there are more tool calls than LLM calls. Doubles the compliance surface.
C. All on GCP	Single cloud, lowest LLM latency	Abandons the AWS plan, the IoT Core fit and existing AWS familiarity. Only reconsider if Gemini becomes a much deeper dependency.

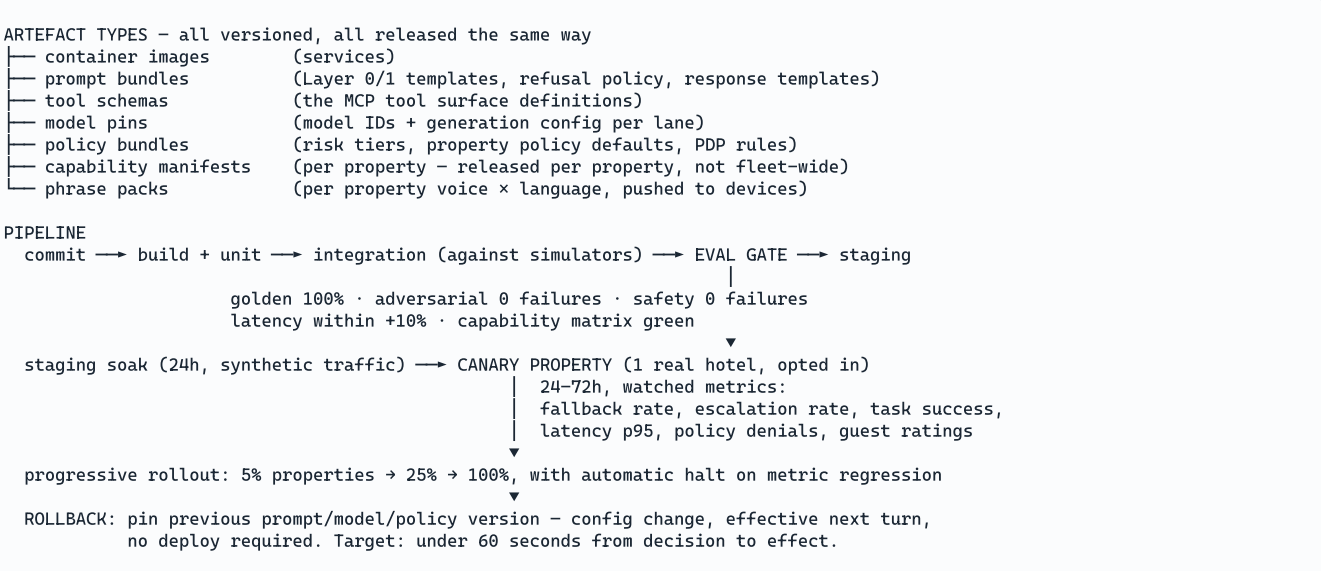
DECISION D20 — OPTION A, WITH MEASUREMENT

Keep everything on AWS and call Vertex AI over the internet from the same geography (ap-south-1 → asia-south1, eu-west-1 → europe-west1, and so on). Authenticate with **Workload Identity Federation** so no Google service-account key file exists anywhere in the estate.

Measure the added latency in the pilot rather than assuming it. Instrument the `llm.generate` span's TTFT specifically and compare against a same-cloud control. If cross-cloud RTT turns out to consume more than ~15% of the agentic lane's p95 budget, revisit — but do not pre-optimise a hop that is probably 40 ms in a 1,900 ms budget.

77. CI/CD and prompt release management

The most important idea in this section: **a prompt change is a production deployment.** Systems that treat prompts as content rather than code fail in production in ways that are hard to diagnose, because the artefact that changed was never versioned.



ROLLBACK MUST NOT REQUIRE A DEPLOY

When a prompt change starts producing bad behaviour across 40,000 rooms at 22:00 on a Friday, you need to revert in under a minute. If reverting means rebuilding a container and waiting for a rolling deploy, you will instead spend fifteen minutes with a live incident. Make prompt, model and policy versions **runtime configuration resolved per turn**, with a pin that propagates through the same sub-10-second config channel as the kill switches.

78. Environments and property canaries

Environment	Purpose	Data	Automation target	Who uses it
`local`	Development	Seeded synthetic	Simulator	Engineers
`ci`	Automated gates	Fixtures	Simulator	Pipeline
`staging`	Integration + soak	Synthetic + lab hardware	**Lab head-end** — one real INNCOM and one real Interel unit on a bench	Engineers, QA
`canary`	Real-world validation	Real, one opted-in property	Real BMS	Delivery + that hotel
`prod`	—	Real	Real	Everyone

BUY THE LAB HARDWARE

A bench with a real INNCOM controller and a real Interel controller, wired to actual dimmers, a thermostat and a curtain motor, is one of the highest-return purchases on this project. Vendor systems have undocumented behaviours — write timing, acknowledgement semantics, priority handling, what happens when two writes race — that no simulator will reproduce until you have observed them. Discovering them in a guest room is expensive; discovering them on a bench is Tuesday.

PART XVI

Delivery

79. Roadmap

Phase	Duration	Deliverables	Exit criteria
0 — Foundations	3–4 weeks	Monorepo, CI, Terraform skeleton, Docker stack, OTel wiring, Postgres schema for tenant/property/room/device, capability model definition, **adapter simulator**, **device simulator**, eval harness skeleton with 50 golden cases	A fake turn runs end to end through the simulators with a full trace, and CI is green
1 — Agentic core	5–7 weeks	Session orchestrator, three-lane router, agent runtime on Gemini, MCP gateway, PDP with tiers T0–T3, property-catalog + device-state + service-request MCP servers, ASR/TTS adapters, audit store	Full agentic turn against the simulator, policy verdicts recorded, golden suite ≥90%, adversarial suite zero failures
2 — Device + edge	5–7 weeks (parallel with 1)	Android refactor: credential removal, WebSocket transport, device-side endpointing, phrase cache, X.509 provisioning, barge-in, offline grammar. Edge gateway with tunnel, shadow, fallback, outbox. First real vendor adapter (whichever the pilot property runs)	Real device in a real room performs a real automation action end to end, and still works with the WAN unplugged
3 — Pilot	6–8 weeks	One property live. Onboarding tooling and capability-manifest authoring. Conformance suite. Staff notification path. F&B ordering with T3 confirmation. PMS bridge for stay context and checkout. Quality review cadence started.	Task success ≥90%, fallback rate <10%, zero safety incidents, GM would keep it
4 — Productisation	8–10 weeks	AWS Phase-2 architecture, multi-tenancy with RLS, second vendor adapter, second regional cell, kill switches, progressive rollout, DPA/DPIA/security pack, SOC 2 evidence collection started, second and third properties	A new property onboards in ≤5 working days without engineering involvement
5 — Scale	Ongoing	Carve-outs by split trigger, remaining cells, cross-stay preferences with consent, Gemini Live conversational lane, SOC 2 Type II, ISO 27001, cost optimisation ladder	50 properties, SLOs met, unit economics confirmed against real turn volumes

SEQUENCING ADVICE

Phases 1 and 2 must run in parallel, and Phase 2 is the one that will slip — hardware, vendor access and firmware always take longer than expected. Start the device and edge work first if you can only start one, because the agentic core can be developed against simulators indefinitely while the device work cannot.

Do not let the pilot property be a flagship. Pick a mid-tier property with a co-operative engineering manager, a modern BMS install, and a GM who will tolerate a bad week. Your first flagship deployment should be your fourth property, not your first.

80. Team shape

Role	Count	Focus
Backend / platform engineer	2–3	Core services, MCP gateway, PDP, data layer, AWS and Terraform
Agent / AI engineer	1–2	Prompt architecture, tool design, router, evals, quality loop. **Owns the weekly quality review.**
Android / embedded engineer	1–2	Device refactor, audio pipeline, provisioning, OTA
Integrations engineer	1–2	Vendor adapters, edge gateway, PMS. **Underestimate this at your peril** — it is the work that determines onboarding speed.
Solutions / delivery engineer	1	Property onboarding, manifest authoring, conformance, hotel relationships. Grows fastest with property count.
Security / compliance	0.5–1 (or vCISO)	Certifications, questionnaires, penetration tests, DPIAs
SRE	0.5–1 (from Phase 4)	Fleet health, on-call, cost, DR

81. Open questions requiring your input

These are the decisions this document could not make for you. Each blocks or reshapes a chunk of the build.

#	Question	Why it matters	Default if unanswered
Q1	**What is the actual turn volume per occupied room per day** in the current V1 deployment?	Drives the entire cost model. It is the number I am least confident in.	Assume 6; instrument from the pilot
Q2	Which automation vendor does the pilot property run, and what integration surface does that specific installation expose?	Determines which adapter is built first and how hard it is	Build the simulator plus INNCOM
Q3	Do any current or prospective customers have a **contractual data-residency requirement**?	Determines whether cell 2 is built in Phase 4 or Phase 5, and whether edge ASR moves up the roadmap	Build `in` first, `eu` second
Q4	Is there an existing edge/gateway appliance in properties today, or does the hub talk to the BMS directly?	Decides whether the edge gateway is new hardware or new software on existing hardware — a large cost and timeline difference	Assume new appliance
Q5	Which PMS platforms are in play across current customers?	PMS adapters are per-family and each is real work	Opera Cloud (OHIP) first
Q6	Do hotels want tickets in **their existing** system (HotSOS, Quore, Alice) or in Woohoo?	Changes the service-request module from a product into an integration — and hotels rarely adopt a second queue	Build both: native store plus push adapters
Q7	What is the commercial model — per room per month, per property, or bundled with hardware?	Determines how hard to push cost optimisation and whether per-tenant cost attribution is a launch requirement	Per room per month; build attribution now
Q8	Is cross-stay personalisation a committed roadmap item?	It is the single largest privacy escalation in the product and needs the governance model operating first	Deferred to Phase 5, opt-in only
Q9	What languages must launch support, per market?	Affects ASR vendor choice, TTS voices, phrase packs, and every eval suite	EN + HI for `in`; EN + AR for `me`
Q10	Does the current hub hardware have a **hardware** mic mute and a driver-controlled LED?	If not, it is a hardware revision, and the compliance story in §55 weakens materially	Assume software mute today; specify hardware mute for the next revision
Q11	Who is accountable for agent behaviour — i.e. who signs off that a prompt change is safe to ship?	§62.4 needs a name, and a chain's auditor will ask for one	Head of Product
Q12	Are Twilio video, Spotify and the F&B device UI staying in V2 as-is?	They are significant surface area not covered by this architecture and may need their own integration into the agent's tool surface	Retain as device features; expose only `media.control` to the agent initially

APPENDIX A

Tool & MCP Schemas

Representative schemas showing the conventions in §32. Note what is *absent*: no room identifier, no property identifier, no vendor concepts.

```
// ─────────── room.get_state ─── T0 ─── the pre-fetched read ───────────
{
  "name": "room.get_state",
  "description": "Current state of every controllable device in the guest's room. Call this before any
                  relative adjustment (warmer, dimmer, brighter). Usually already provided in context.",
  "inputSchema": { "type": "object", "properties": {}, "additionalProperties": false },
  "outputSchema": {
    "type": "object",
    "properties": {
      "zones": { "type": "array", "items": {
          "type": "object",
          "properties": {
            "zone":     { "type": "string", "enum": ["bedroom","living","bathroom","entrance"] },
            "lighting": { "type": "object", "properties": {
                            "on":         { "type": "boolean" },
                            "brightness": { "type": "integer", "minimum": 0, "maximum": 100 },
                            "available":  { "type": "boolean" } } },
            "climate":  { "type": "object", "properties": {
                            "current_c":  { "type": "number" },
                            "setpoint_c": { "type": "number" },
                            "mode":       { "type": "string", "enum": ["auto","cool","heat","fan","off"] },
                            "min_c":      { "type": "number" },   // property-policy bounds, shown so the
                            "max_c":      { "type": "number" },   // model proposes valid values first time
                            "available":  { "type": "boolean" } } },
            "curtains": { "type": "object", "properties": {
                            "position":  { "type": "integer", "minimum": 0, "maximum": 100 },
                            "supports_position": { "type": "boolean" },  // false => open/close only
                            "available": { "type": "boolean" } } }
          } } },
      "dnd":       { "type": "string", "enum": ["off","dnd","make_up_room"] },
      "freshness": { "type": "string", "enum": ["fresh","stale","unknown"] },  // §37 contract
      "as_of":     { "type": "string", "format": "date-time" }
    }
  }
}

// ─────────── climate.set_temperature ─── T1 ─── bounds enforced by the PDP, not the schema ───────────
{
  "name": "climate.set_temperature",
  "description": "Set the target temperature. Use room.get_state first for relative requests. Bounds are
                  set by the property; a value outside them will be rejected with the permitted range.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "zone":     { "type": "string", "enum": ["bedroom","living"] },   // enum generated from the manifest
      "target_c": { "type": "number", "minimum": 16, "maximum": 30 }
    },
    "required": ["zone","target_c"],
    "additionalProperties": false          // ← model-supplied extras are rejected outright
  },
  "outputSchema": {
    "type": "object",
    "properties": {
      "status":   { "type": "string", "enum": ["applied","pending","rejected"] },
      "observed": { "type": "object" },    // present only when status = applied (verified read, §37)
      "message":  { "type": "string" }     // guest-safe phrasing hint on rejection
    }
  },
  "x-woohoo": { "tier": "T1", "idempotent": true, "verify_read": true, "timeout_ms": 2500 }
}

// ─────────── fnb.place_order ─── T3 ─── always requires a confirmed second turn ───────────
{
  "name": "fnb.place_order",
  "description": "Place a food and beverage order charged to the room folio. ALWAYS state the full total
                  aloud and obtain explicit confirmation. Never place an order in the same turn it is
                  first proposed.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "items": { "type": "array", "minItems": 1, "maxItems": 15, "items": {
          "type": "object",
          "properties": {
            "menu_item_id": { "type": "string" },   // must come from fnb.get_menu — never invented
            "quantity":     { "type": "integer", "minimum": 1, "maximum": 10 },
            "notes":        { "type": "string", "maxLength": 200 }
          },
          "required": ["menu_item_id","quantity"] } },
```

```
      "delivery_note": { "type": "string", "maxLength": 200 }
    },
    "required": ["items"],
    "additionalProperties": false
  },
  "x-woohoo": {
    "tier": "T3",
    "requires_confirmation": true,
    "confirmation_must_include": ["total_amount","currency","item_count"],
    "spend_cap_applies": true,
    "idempotent": true,
    "retry": "never"
  }
}

// ─────────── The standard error envelope — errors are instructions, not codes ───────────
{
  "error": "NOT_AVAILABLE",              // NOT_AVAILABLE | NOT_PERMITTED | TEMPORARILY_UNREACHABLE
                                         // | NEEDS_CONFIRMATION | INVALID_ARGUMENT | RATE_LIMITED
  "detail": "This room does not have motorised curtains.",
  "suggestion": "Tell the guest the curtains are manual. Offer to send housekeeping if they need help.",
  "retryable": false
}
```

APPENDIX B

Event Schemas

```
// Every event shares this envelope. tenant_id and trace_id are mandatory — adding them later is a migration.
{
  "specversion": "1.0",
  "type":        "woohoo.turn.completed",
  "source":      "/cell/in/orchestrator",
  "id":          "evt_01J8X...",
  "time":        "2026-07-29T18:14:02.331Z",
  "subject":     "turn_01J8X...",
  "tenant_id":   "t_example_apac",
  "property_id": "p_blr_whitefield",
  "trace_id":    "4bf92f3577b34da6a3ce929d0e0e4736",
  "schema_version": "1.0",
  "data": { /* per-type payload */ }
}

// woohoo.turn.completed — the analytics and audit spine
"data": {
  "turn_id": "turn_01J8X...", "room_ref": "r_412", "device_id": "d_8f2a...",
  "lane": "agentic", "language": "en-IN", "degradation_level": 0,
  "asr": { "vendor": "deepgram", "confidence": 0.91, "audio_ms": 5820 },
  "llm": { "model": "gemini-2.5-flash", "prompt_version": "v14",
           "tokens_in": 5140, "tokens_cached": 4020, "tokens_out": 287, "cost_usd": 0.00163 },
  "tool_calls": [
    { "tool": "climate.set_temperature", "tier": "T1", "verdict": "ALLOW",  "status": "applied", "ms": 412 },
    { "tool": "service.create_request",  "tier": "T2", "verdict": "ALLOW",  "status": "applied", "ms": 88,
      "annotations": { "dispatch_at": "08:00" } }
  ],
  "latency": { "endpoint_to_first_audio_ms": 1480, "total_ms": 4402 },
  "outcome": "success", "escalated": false, "barge_in": false
}

// woohoo.escalation.raised — must be acknowledged; alarm if it is not
"data": {
  "escalation_id": "esc_01J8X...", "room_ref": "r_412", "category": "medical",
  "severity": "critical", "trigger": "distress_phrase_detected",
  "context_summary": "Guest reported chest pain at 23:14 local.",
  "notified": ["front_desk","duty_manager"], "requires_ack_within_s": 120
}

// woohoo.stay.checked_out — drives data deletion, so delivery is reliability-critical
"data": { "stay_ref": "stay_tok_9f...", "room_ref": "r_412",
          "checked_out_at": "2026-07-30T05:31:00Z",
          "purge": ["session","stay_memory","cached_pms_context","device_personalisation"] }

// woohoo.policy.denied — feeds security monitoring and the adversarial eval corpus
"data": { "turn_id": "turn_01J8X...", "tool": "fnb.place_order", "tier": "T3",
          "rule_id": "spend_cap_per_stay", "verdict": "DENY",
          "reason": "stay spend cap reached", "utterance_hash": "sha256:..." }
```

APPENDIX C

Property Configuration Schema

This is the artefact that makes onboarding a configuration exercise. It is authored during onboarding, versioned, and validated by the conformance suite before go-live.

```
{
  "property_id": "p_blr_whitefield",
  "tenant_id":   "t_example_apac",
  "name":        "Example Hotel Whitefield",
  "cell":        "in",
  "timezone":    "Asia/Kolkata",
  "languages":   ["en-IN","hi-IN","kn-IN"],
  "default_language": "en-IN",

  "brand": {
    "voice_id": "chirp3-hd-en-in-warm",
    "tone": "warm, concise, professional; address guests as 'sir/ma'am' only if the PMS indicates a title",
    "assistant_name": "Voice Genie",
    "wake_word_model": "hello_voice_genie_v3"
  },

  "automation": {
    "vendor": "inncom",
    "edge_gateways": [{ "id": "eg_blr_01", "ha_peer": "eg_blr_02" }],
    "connection": { "secret_ref": "sm://properties/p_blr_whitefield/inncom" },
    "room_ref_mapping": "vendor_room_id"        // NOT the hotel room number — see §39.2
  },

  "pms": {
    "type": "opera_cloud",
    "secret_ref": "sm://properties/p_blr_whitefield/opera",
    "features": ["stay_context","folio_post","dnd_sync","wakeup_call","checkout_event"]
  },

  "policy": {
    "climate": { "min_c": 18, "max_c": 26, "max_changes_per_hour": 6 },
    "quiet_hours": { "start": "22:00", "end": "07:00",
                     "defer_non_urgent_dispatch": true,
                     "suppress_unattended_actuation": true },
    "spend": { "per_stay_cap": { "amount": 15000, "currency": "INR" },
               "per_order_cap": { "amount": 6000,  "currency": "INR" },
               "confirmation_required_above": { "amount": 0, "currency": "INR" } },
    "service_requests": { "max_open": 10, "max_open_per_category": 3,
                          "sla_minutes": { "housekeeping": 20, "maintenance": 45, "amenity": 15 } },
    "tier_ceiling": "T3",                       // kill-switch lever — set to "T0" for read-only mode
    "disabled_tools": []
  },

  "escalation": {
    "front_desk":   { "channel": "staff_app", "target": "role:front_desk" },
    "duty_manager": { "channel": "sms+staff_app", "target": "rota:duty_manager" },
    "welfare_protocol_ref": "doc://properties/p_blr_whitefield/welfare-protocol-v2",
    "emergency_number_display": "112",
    "crisis_helpline_display": "+91-XXXXXXXXXX"
  },

  "room_types": [
    { "id": "rt_deluxe_king", "count": 180,
      "capability_manifest": {
        "version": "3",
        "zones": ["bedroom","bathroom"],
        "capabilities": [
          { "capability": "lighting.on_off",     "zones": ["bedroom","bathroom"] },
          { "capability": "lighting.brightness", "zones": ["bedroom"] },
          { "capability": "climate.setpoint",    "zones": ["bedroom"], "min_c": 18, "max_c": 26 },
          { "capability": "climate.mode",        "zones": ["bedroom"] },
          { "capability": "dnd" },
          { "capability": "tv.power_volume_input" }
          // note: no curtains capability => no curtain tool is generated for this room type
        ],
        "scenes": ["sleep","welcome","work"]
      } },
    { "id": "rt_suite", "count": 20,
      "capability_manifest": {
        "version": "3",
        "zones": ["bedroom","living","bathroom"],
        "capabilities": [
          { "capability": "lighting.on_off",     "zones": ["bedroom","living","bathroom"] },
          { "capability": "lighting.brightness", "zones": ["bedroom","living"] },
          { "capability": "lighting.scene",      "zones": ["living"] },
          { "capability": "climate.setpoint",    "zones": ["bedroom","living"], "min_c": 18, "max_c": 26 },
          { "capability": "curtains.position",   "zones": ["bedroom","living"] },
          { "capability": "dnd" },
```

```
          { "capability": "tv.power_volume_input" }
        ],
        "scenes": ["sleep","welcome","work","movie"]
      } }
  ],

  "room_overrides": [
    { "room_ref": "r_412", "disable": ["curtains.position"], "reason": "motor awaiting replacement" }
  ],

  "asr": { "primary": "deepgram", "fallback": "google_stt_v2",
           "phrase_hints_ref": "kb://p_blr_whitefield/phrase-hints" },

  "features": { "fnb_ordering": true, "gemini_live_chat_lane": false,
                "cross_stay_preferences": false, "personalised_greeting": false },

  "compliance": { "controller": "Example Hotels Pvt Ltd",
                  "transcript_retention_days": 30,
                  "notice_version": "in-v2",
                  "dpa_ref": "doc://legal/dpa/example-apac-v1" }
}
```

APPENDIX D

Repository & Module Layout

```
woohoo/                                  monorepo — independently buildable targets
├── services/
│   ├── device-gateway/                  Go — WS + mTLS, connection registry
│   ├── woohoo-core/                     the modular monolith
│   │   └── src/modules/
│   │       ├── orchestrator/            turn lifecycle, degradation ladder
│   │       ├── router/                  three-lane classification + fast-path grammar
│   │       ├── agent/                   context assembly, Gemini, tool loop
│   │       ├── mcp-gateway/             registry, scope injection, sanitisation
│   │       ├── asr/  tts/               vendor adapters behind interfaces
│   │       ├── catalog/                 ← owns tenant/property/room/device tables
│   │       ├── device-state/            ← owns state + shadow
│   │       ├── service-request/         ← owns tickets
│   │       ├── fnb/                     ← owns menus + orders
│   │       ├── pms-bridge/              ← owns stay tokens (early carve-out candidate)
│   │       ├── knowledge/               ← owns content + embeddings
│   │       ├── prefs/                   ← owns consent + preferences
│   │       ├── notification/            ← owns delivery state
│   │       └── audit/                   ← owns turn records
│   ├── policy-service/                  PDP — small, readable, independently versioned
│   ├── adapters/{inncom,interel,bacnet,simulator}/
│   └── edge-gateway/                    ships to properties
├── device/android/                      com.sb.woohoo
├── packages/
│   ├── contracts/                       tool schemas, event schemas, capability definitions
│   ├── prompts/                         versioned prompt bundles + response templates
│   ├── policy-bundles/                  risk tiers, default property policy
│   └── otel/                            shared instrumentation
├── eval/
│   ├── suites/{golden,adversarial,safety,capability,regression,latency}/
│   └── runner/
├── infra/terraform/{modules,cells/{in,eu,me,us}}/
├── tools/{device-simulator,property-onboarding,conformance}/
└── docs/architecture/                   this document

MODULE BOUNDARY RULES (lint-enforced — this is what makes the monolith modular rather than a ball of mud)
1. A module may only be reached through its public interface. No cross-module imports of internals.
2. A module may only touch its own tables. Cross-module data access goes through the interface.
3. No module imports from `adapters/` — vendor concepts never travel upward.
4. `agent/` may not import `policy-service` types except the verdict contract. It must not be able
   to evaluate policy itself.
5. `contracts/` imports nothing. Everything else may import it.
```

APPENDIX E

Glossary

Term	Meaning in this document
Agentic lane	The routing path where Gemini reasons over the tool surface. Contrast: fast lane, clarify lane.
Capability	An abstract, vendor-neutral ability (e.g. `climate.setpoint`) with a typed contract. The unit of abstraction that keeps vendor differences out of the model.
Capability manifest	Per-room-type declaration of which capabilities exist, with zones, bounds and driver bindings. Generates the tool surface.
Cell	A complete, independent regional stack. No guest data crosses a cell boundary.
Containment	The proportion of turns resolved without staff involvement.
Edge Gateway	The on-premise appliance that terminates vendor protocols, holds local state and provides offline fallback.
Fast lane	Deterministic handling of high-frequency utterances with no LLM call.
Filled pause	A cached acknowledgement played while the agent works, so perceived latency is far below actual latency.
MCP	Model Context Protocol — the standard by which capabilities are exposed to a model.
MCP Gateway	The internal control point through which all tool calls pass. Injects scope, filters tools, sanitises results, audits.
PDP	Policy Decision Point — the component that authorises or refuses every proposed tool call. Sits outside the model.
Phrase pack	Pre-synthesised common phrases cached on the device per voice and language.
Risk tier (T0–T5)	The classification that determines what the platform requires before an action executes. Defined in §44.
Scope	The immutable `{tenant, property, room, device}` tuple derived from the device certificate and injected into every tool call.
Shadow evaluation	Replaying fast-lane turns through the agentic lane offline to detect divergence.
Turn	One complete wake-to-response interaction. The unit of tracing, auditing, billing and evaluation.
TurnContext	The immutable per-turn record of scope, capability version, language, consent state and time that follows a turn through every component.

† Prices and latency figures are indicative planning numbers current to early 2026 and must be re-quoted before contracting.
Part XI is a compliance posture, not legal advice; confirm jurisdictional specifics with counsel before deployment.